

第一部分：HTTP/1.1 协议



扫码试看/订阅极客时间

《Web协议详解与抓包实战》视频课程

从上至下的课程安排

以 TCP/IP 协议栈为依托，由上至下、从应用层至基础设施介绍协议

- 应用层
 - 第 1 部分：HTTP/1.1
 - 第 2 部分：Websocket
 - 第 3 部分：HTTP/2.0
- 应用层的安全基础设施
 - 第 4 部分：TLS/SSL
- 传输层
 - 第 5 部分：TCP
- 网络层及数据链路层
 - 第 6 部分：IP 层和以太网

对工具的介绍

- 由浅至深，适时插入课程
 - 首先在演示、实验过程中必须用到的场景里简单介绍用法
 - 在需要深度掌握工具时，再次完整地介绍用法
- 涉及主要工具
 - Chrome 浏览器 Network 面板 (Firefox 类似)
 - Wireshark
 - tcpdump

HTTP/1 课程安排

- 自顶而下、由业务到逻辑
 - HTTP/1 协议为什么会如此设计?
 - 网络分层原理、REST 架构
 - 协议的通用规则
 - 协议格式、URI、方法与响应码概览
 - 连接与消息的路由
 - 内容协商与传输
 - cookie 的设计与问题
 - 缓存的控制

HTTP/1 的协议升级

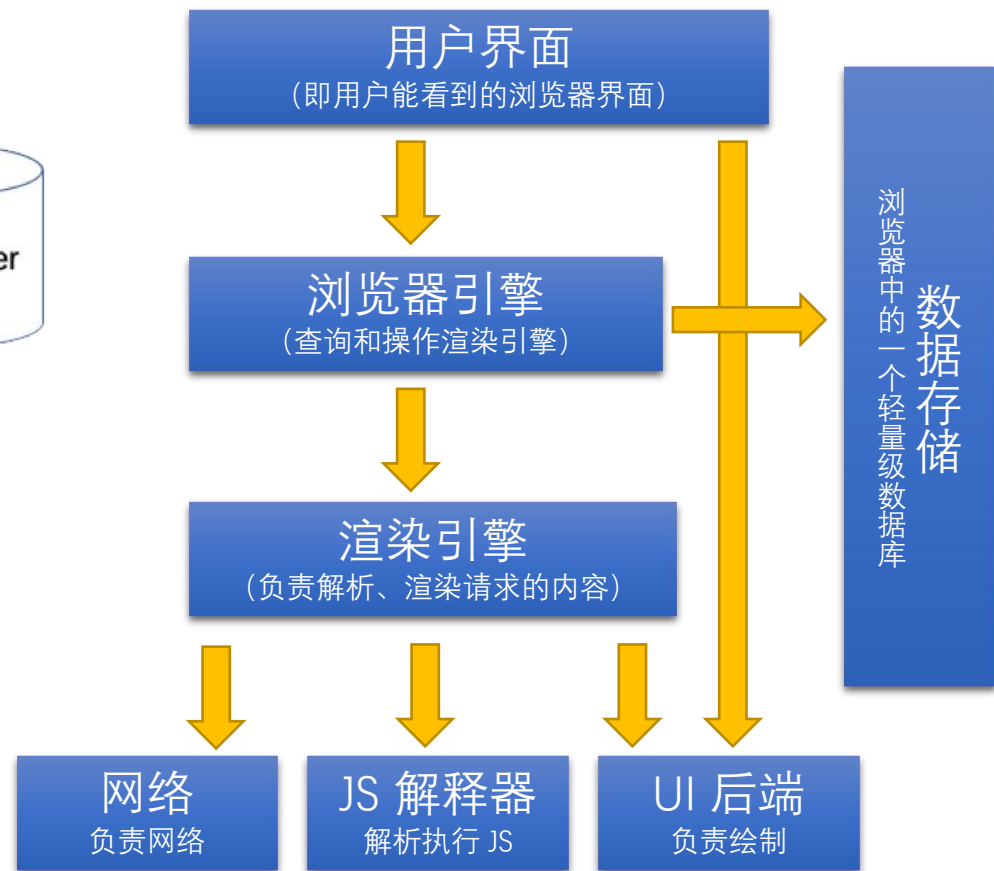
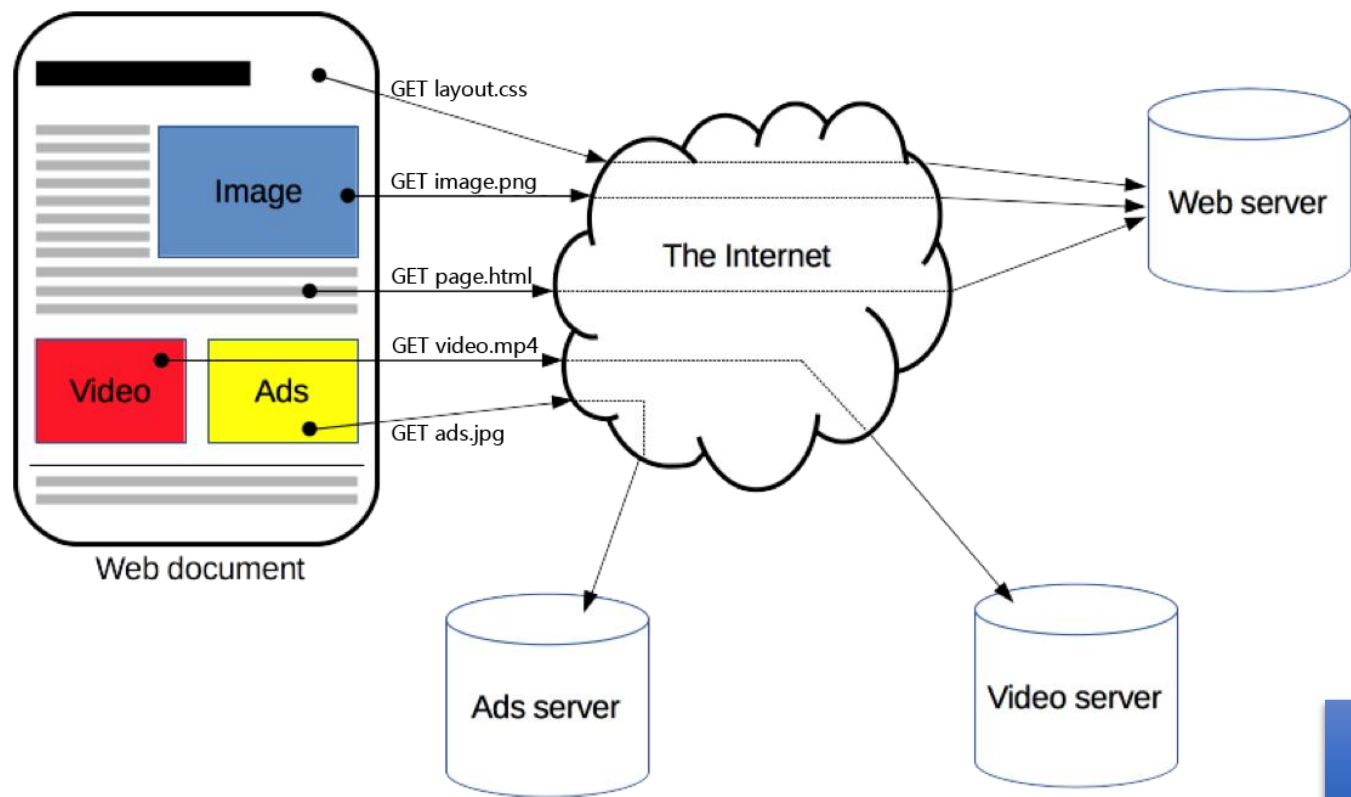
- 支持服务器推送消息的 WebSocket 协议
 - 建立会话
 - 消息传输
 - 心跳
 - 关闭会话
- 全面优化后的 HTTP/2.0 协议
- HTTP/2.0 必须开启的 TLS/SSL 协议

TCP 与 IP 协议

- 传输层的 TCP 协议
 - 建立连接
 - 传输数据
 - 拥塞控制
 - 关闭连接
- 网络层的 IP 协议
 - IP 报文与路由
 - 网络层其他常用协议：ICMP、ARP、RARP
 - IPv6 的区别

第 1 课

浏览器发起 HTTP 请求的典型场景



浏览器发起 HTTP 请求的典型场景



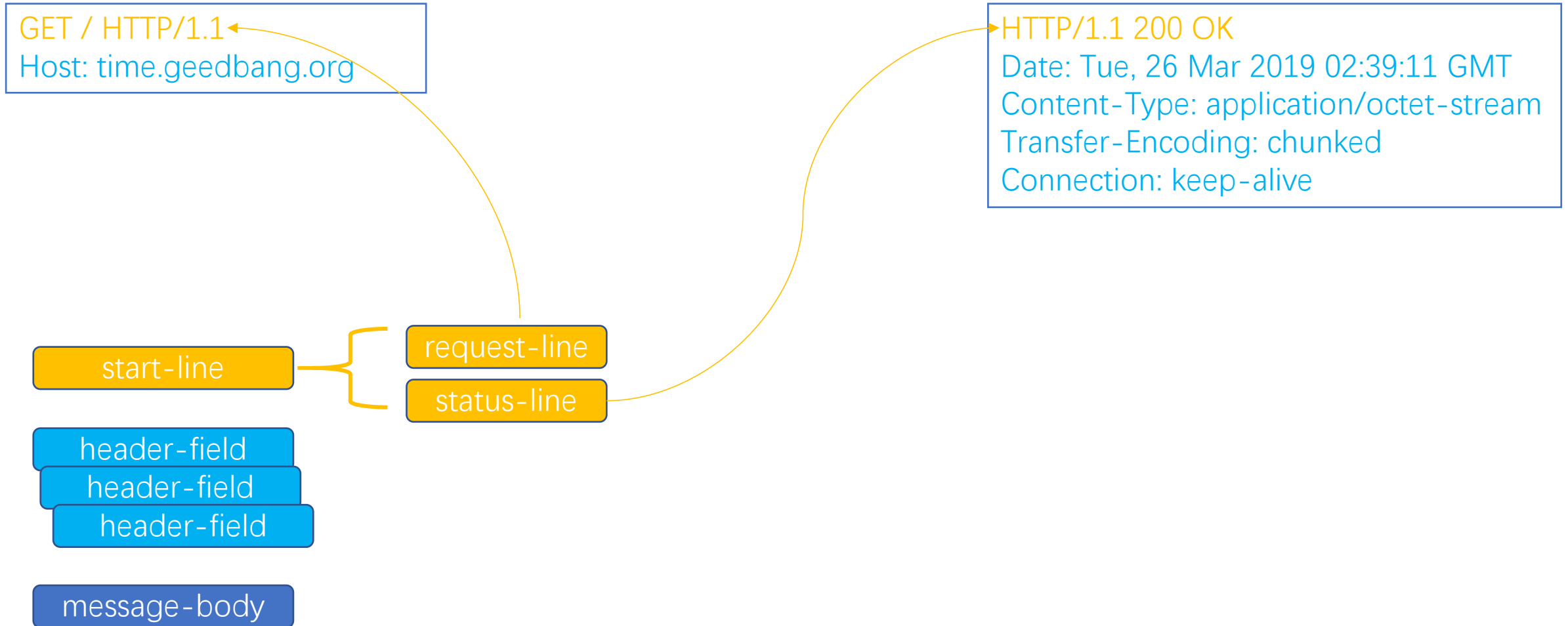
Hypertext Transfer Protocol (HTTP) 协议

a **stateless** application-level **request/response** protocol that uses **extensible semantics** and **self-descriptive** message payloads for flexible interaction with network-based **hypertext information** systems
(RFC7230 2014.6)

一种无状态的、应用层的、以请求/应答方式运行的协议，它使用可扩展的语义和自描述消息格式，与基于网络的超文本信息系统灵活的互动

.

HTTP 协议格式



ABNF (扩充巴科斯-瑙尔范式) 操作符

- 空白字符: 用来分隔定义中的各个元素
 - method SP request-target SP HTTP-version CRLF
- 选择 **/**: 表示多个规则都是可供选择的规则
 - start-line = request-line / status-line
- 值范围 **%c##-##** :
 - OCTAL = "0" / "1" / "2" / "3" / "4" / "5" / "6" / "7" 与 OCTAL = %x30-37 等价
- 序列组合 **()**: 将规则组合起来, 视为单个元素
- 不定量重复 **m*n**:
 - * 元素表示零个或更多元素: *(header-field CRLF)
 - 1* 元素表示一个或更多元素, 2*4 元素表示两个至四个元素
- 可选序列 **[]**:
 - [message-body]

ABNF (扩充巴科斯-瑙尔范式) 核心规则

规则	形式定义	意义
ALPHA	%x41-5A / %x61-7A	大写和小写ASCII字母 (A-Z, a-z)
DIGIT	%x30-39	数字 (0-9)
HEXDIG	DIGIT / "A" / "B" / "C" / "D" / "E" / "F"	十六进制数字 (0-9, A-F, a-f)
DQUOTE	%x22	双引号
SP	%x20	空格
HTAB	%x09	横向制表符
WSP	SP / HTAB	空格或横向制表符
LWSP	*(WSP / CRLF WSP)	直线空白 (晚于换行)
VCHAR	%x21-7E	可见 (打印) 字符
CHAR	%x01-7F	任何7-位US-ASCII字符, 不包括NUL (%x00)
OCTET	%x00-FF	8位数据
CTL	%x00-1F / %x7F	控制字符
CR	%x0D	回车
LF	%x0A	换行
CRLF	CR LF	互联网标准换行
BIT	"0" / "1"	二进制数字

基于 ABNF 描述的 HTTP 协议格式

HTTP-message = **start-line** *(**header-field** CRLF) CRLF [**message-body**]

- **start-line** = request-line / status-line
 - request-line = method SP request-target SP HTTP-version CRLF
 - status-line = HTTP-version SP status-code SP reason-phrase CRLF
- **header-field** = field-name ":" OWS field-value OWS
 - OWS = *(SP / HTAB)
 - field-name = token
 - field-value = *(field-content / obs-fold)
- **message-body** = *OCTET

基于 ABNF 描述的 HTTP 协议格式

```
GET /wp-content/plugins/Pure-Highlightjs_1.0/assets/pure-highlight.css?ver=0.1.0 HTTP/1.1
Host: www.taohui.pub
```

```
HTTP/1.1 200 OK
Server: openresty/1.13.6.2
Date: Thu, 18 Apr 2019 01:21:14 GMT
Content-Type: text/css
Content-Length: 108
Last-Modified: Thu, 27 Dec 2018 07:35:33 GMT
Connection: keep-alive
ETag: "5c2480c5-6c"
Expires: Thu, 25 Apr 2019 01:21:14 GMT
Cache-Control: max-age=604800
Accept-Ranges: bytes
```

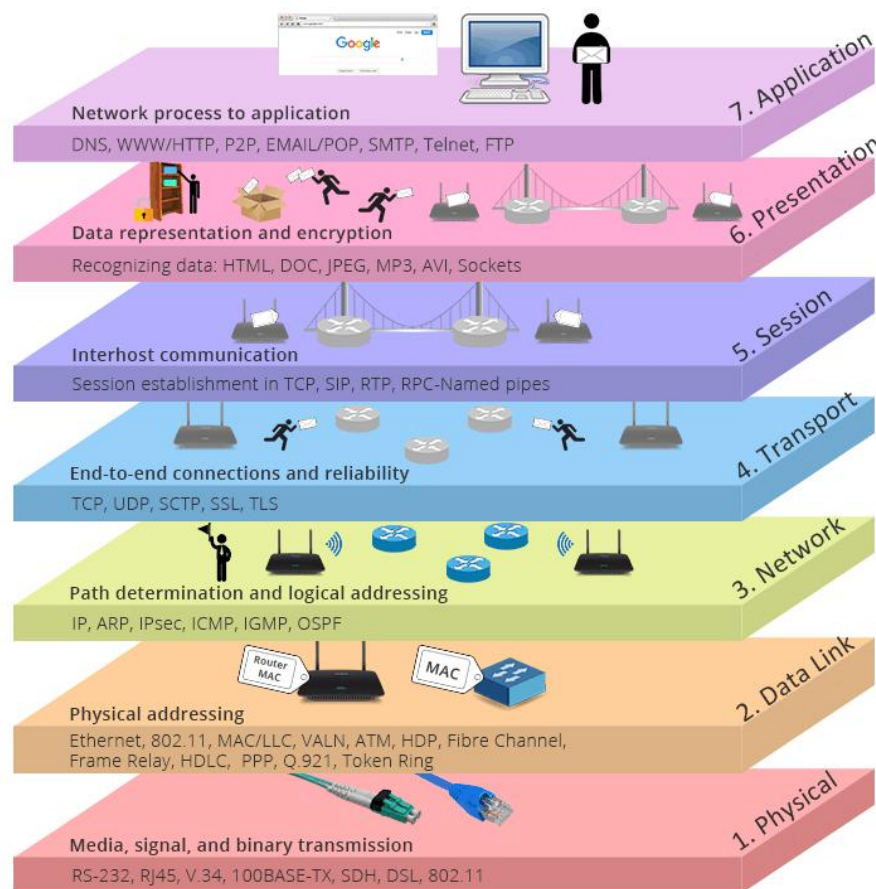
```
pre.pure-highlightjs {
    background-color: transparent; !important;
    border: none;
    padding: 0;
}
```

HTTP-message = **start-line** *(**header-field** CRLF) CRLF [**message-body**]

- **start-line** = request-line / status-line
 - request-line = method SP request-target SP HTTP-version CRLF
 - status-line = HTTP-version SP status-code SP reason-phrase CRLF
- **header-field** = field-name ":" OWS field-value OWS
 - OWS = *(SP / HTAB)
 - field-name = token
 - field-value = *(field-content / obs-fold)
- **message-body** = *OCTET

第 2 课

OSI (Open System Interconnection Reference Model) 概念模型



- 应用层

- 表示层

- 会话层

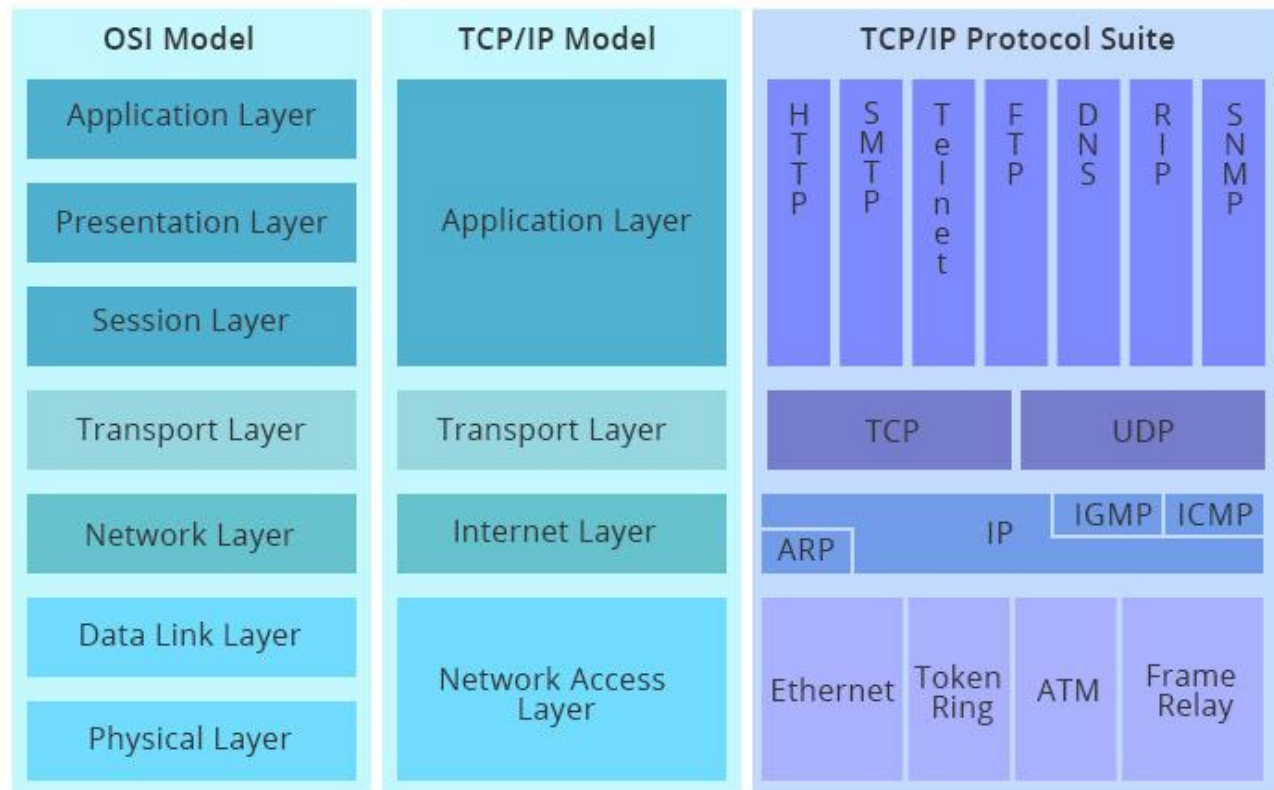
- 传输层

- 网络层

- 数据链路层

- 物理层

OSI 模型与 TCP/IP 模型对照



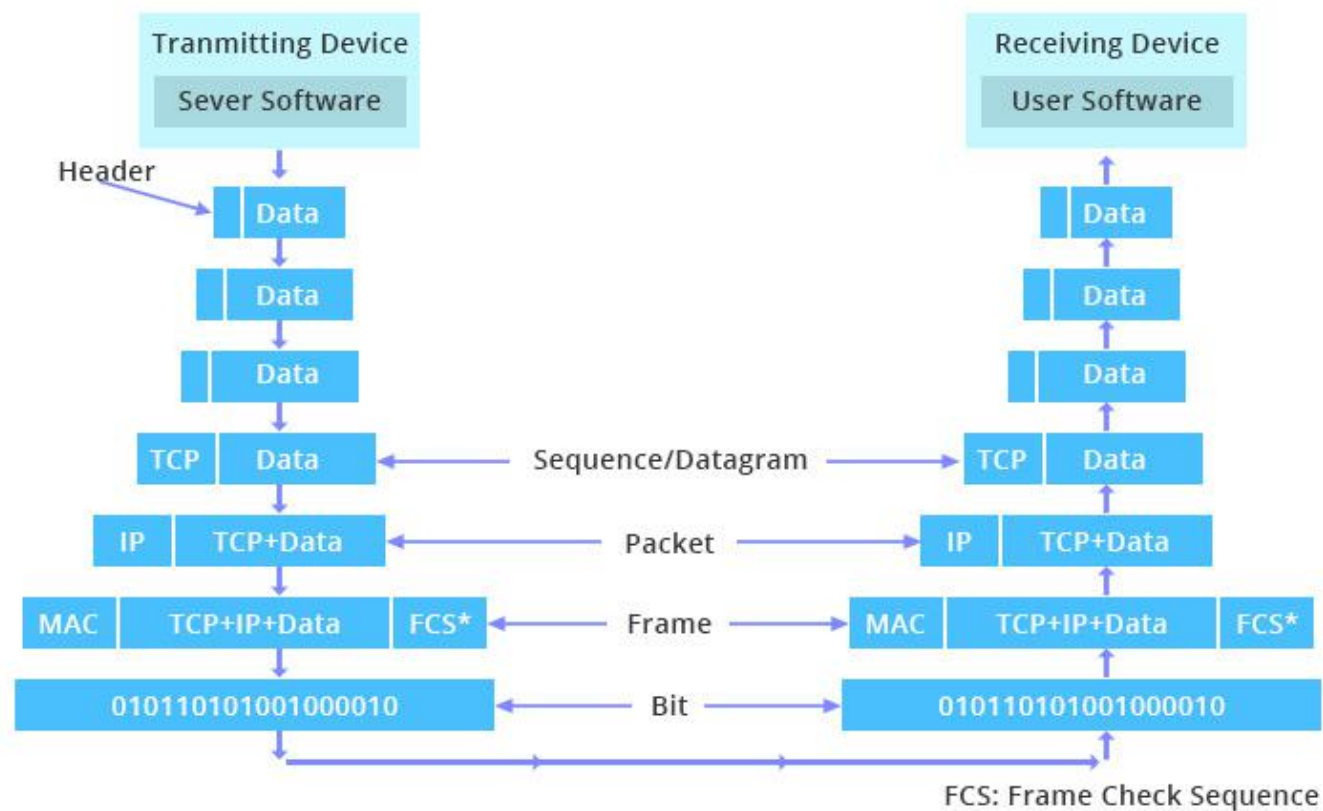
Wireshark 抓包及分析工具

开源、免费

历史

- Ethereal: 1997-2006
- 改名 Wireshark: 2006
 - 1.0 正式稳定版: 2008
 - 2.0 版: 2015
 - 3.0 版: 2019

报文头部



第 3 课

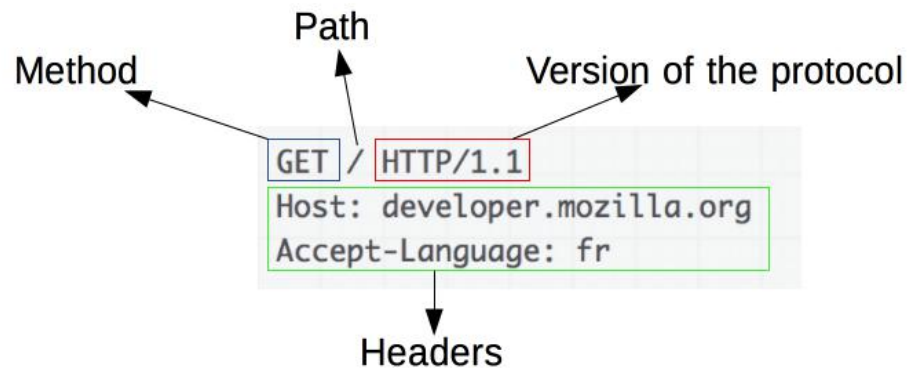
Roy Thomas Fielding 与 HTTP/1.1

- 参与制订 HTTP/1.0 规范 (1996.5)
- 参与制订 URI 规范 (1998.8)
- 主导制订 HTTP/1.1 规范 (1999.6)
- 2000 年发布指导 HTTP/1.1 规范制订的论文
 - 《Architectural Style and the Design of Network-based Software Architectures》, 即我们常谈的Representational State Transfer (REST)架构
- Apache基金会 (The Apache Software Foundation) 共同创始人
 - 参与开发Apache httpd服务

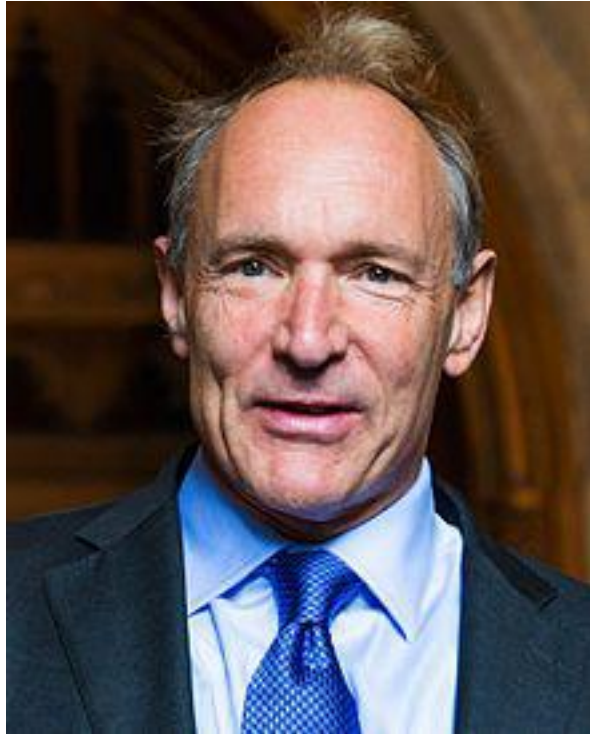


Form Follows Function: HTTP 协议为什么是现在这个样子？

- HTTP 协议
 - Roy Thomas Fielding: HTTP 主要作者, REST 架构作者
- URI: 统一资源标识符



HTTP 解决了什么问题？



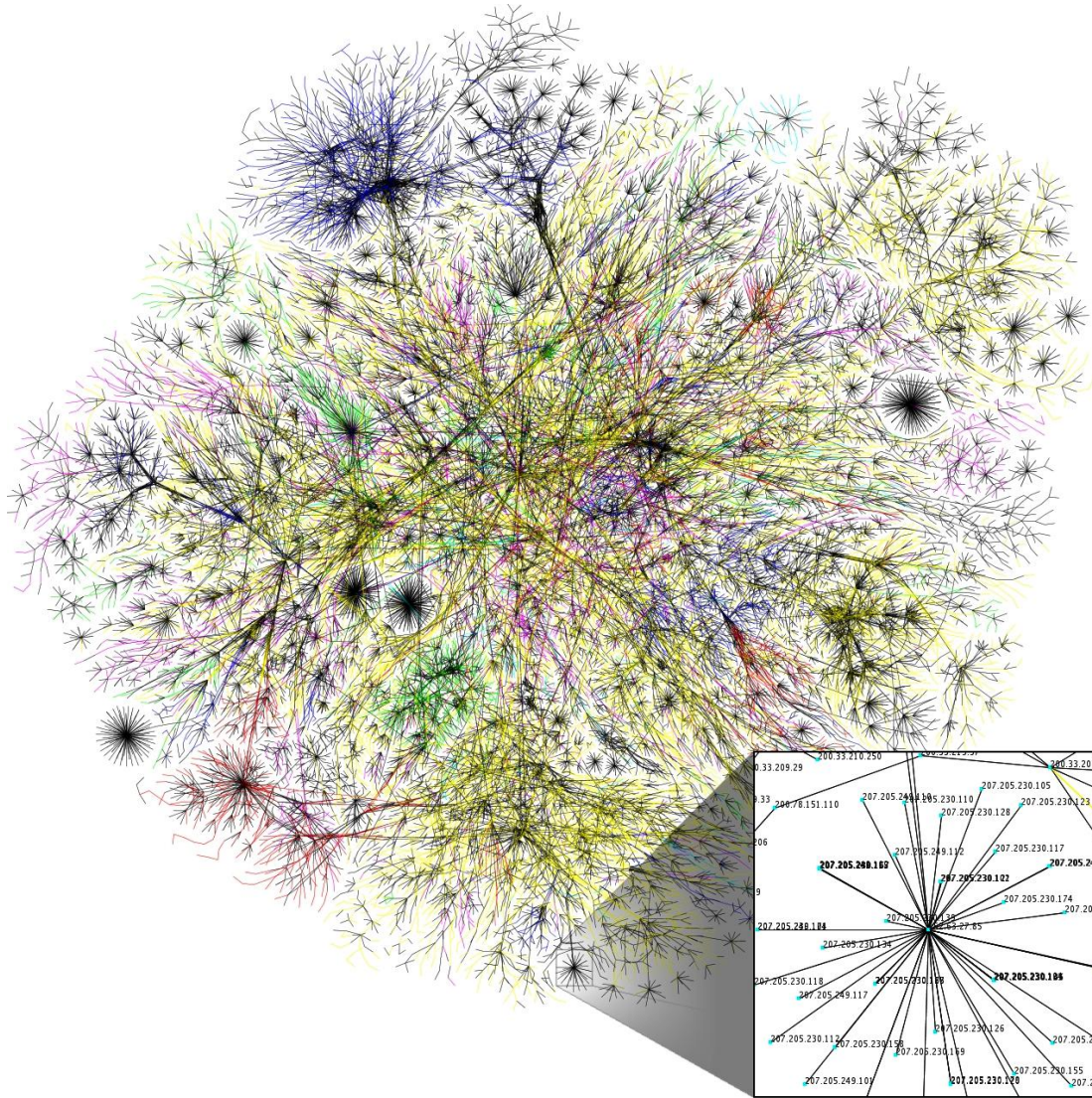
Web's major goal was to be a shared information space through which people and machines could communicate.

--Tim Berners Lee

HTTP 解决了什么问题？

解决 WWW 信息交互必须面对的需求：

- 低门槛
- 可扩展性：巨大的用户群体，超长的寿命
- 分布式系统下的 Hypermedia：大粒度数据的网络传输
- Internet 规模
 - 无法控制的 scalability
 - 不可预测的负载、非法格式的数据、恶意消息
 - 客户端不能保持所有服务器信息，服务器不能保持多个请求间的状态信息
 - 独立的组件部署：新老组件并存
- 向前兼容：自 1993 年起 HTTP0.9\1.0（1996）已经被广泛使用



深蓝色: net, ca, us

绿色: com, org

红色: mil, gov, edu

黄色: jp, cn, tw, au, de

紫红色: uk, it, pl, fr

金黄色: br, kr, nl

白色: 其他

巨大的网络：2005 年初的网络拓扑图

第 4 课

评估 Web 架构的关键属性

HTTP 协议应当在以下属性中取得可接受的均衡：

1. 性能 Performance：影响高可用的关键因素
2. 可伸缩性 Scalability：支持部署可以互相交互的大量组件
3. 简单性 Simplicity：易理解、易实现、易验证
4. 可见性 Visible：对两个组件间的交互进行监视或者仲裁的能力。如缓存、分层设计等
5. 可移植性 Portability：在不同的环境下运行的能力
6. 可靠性 Reliability：出现部分故障时，对整体影响的程度
7. 可修改性 Modifiability：对系统作出修改的难易程度，由可进化性、可定制性、可扩展性、可配置性、可重用性构成

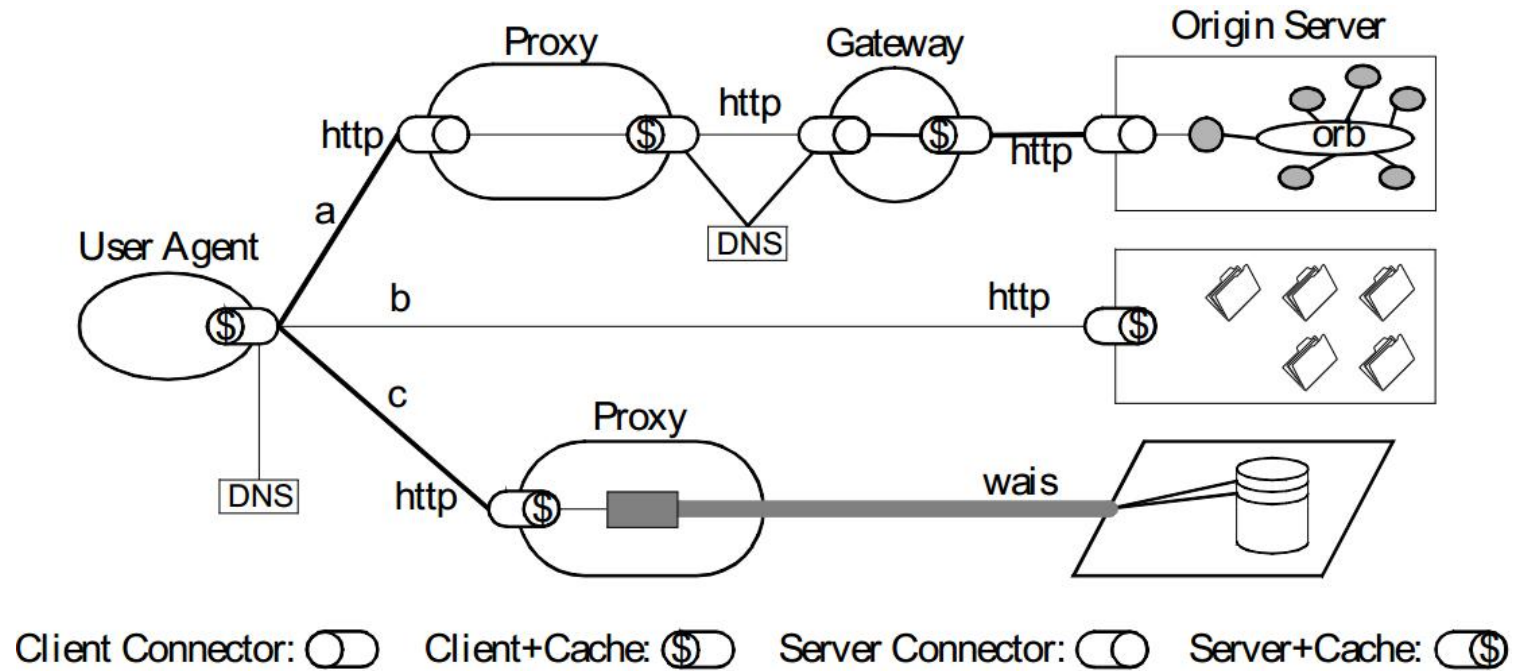
架构属性：性能

- 网络性能 Network Performance
 - Throughput 吞吐量：小于等于带宽 bandwidth
 - Overhead 开销：首次开销，每次开销
- 用户感知到的性能 User-perceived Performance
 - Latency 延迟：发起请求到接收到响应的时间
 - Completion 完成时间：完成一个应用动作所花费的时间
- 网络效率 Network Efficiency
 - 重用缓存、减少交互次数、数据传输距离更近、COD

架构属性：可修改性

- 可进化性 Evolvability：一个组件独立升级而不影响其他组件
- 可扩展性 Extensibility：向系统添加功能，而不会影响到系统的其他部分
- 可定制性 Customizability：临时性、定制性地更改某一要素来提供服务，不对常规客户产生影响
- 可配置性 Configurability：应用部署后可通过修改配置提供新的功能
- 可重用性 Reusabilit：组件可以不做修改在其他应用在使用

REST 架构下的 Web



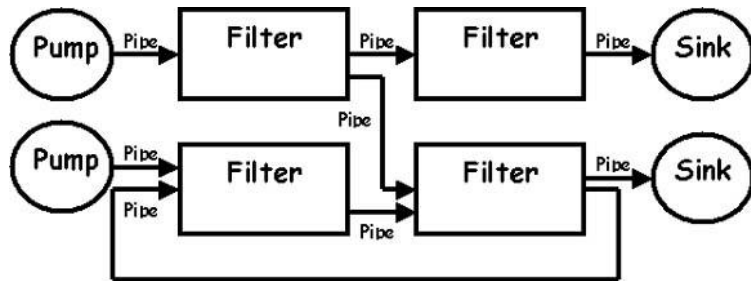
第 5 课

5 种架构风格

- 数据流风格 Data-flow Styles
 - 优点：简单性、可进化性、可扩展性、可配置性、可重用性
- 复制风格 Replication Styles
 - 优点：用户可察觉的性能、可伸缩性，网络效率、可靠性也可以提到提升
- 分层风格 Hierarchical Styles
 - 优点：简单性、可进化性、可伸缩性
- 移动代码风格 Mobile Code Styles
 - 优点：可移植性、可扩展性、网络效率
- 点对点风格 Peer-to-Peer Styles
 - 优点：可进化性、可重用性、可扩展性、可配置性

数据流风格 Data-flow Styles

- 管道与过滤器 Pipe And Filter, PF
 - 每个 Filter 都有输入端和输出端，只能从输入端读取数据，处理后再从输出端产生数据



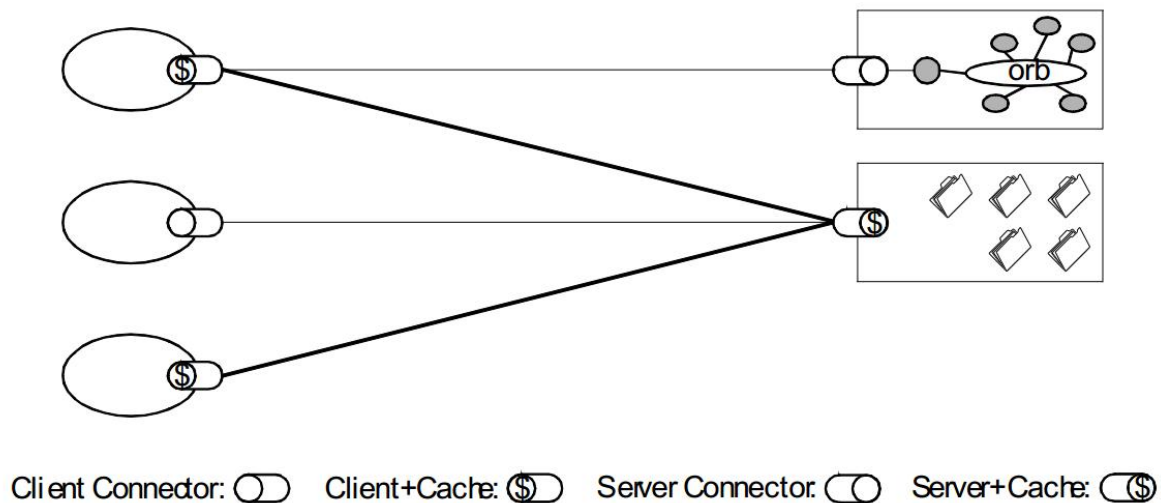
- 统一接口的管道与过滤器 Uniform Pipe And Filter, UPF
 - 在 PF 上增加了统一接口的约束，所有 Filter 过滤器必须具备同样的接口

数据流风格 Data-flow Styles

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
PF			±			+	+	+		+	+			
UPF	PF	-	±			++	+	+		++	++	+		

复制风格 Replication Styles

- 复制仓库 Replicated Repository, RR
 - 多个进程提供相同的服务，通过反向代理对外提供集中服务
- 缓存 \$
 - RR的变体，通过复制请求的结果，为后续请求复用



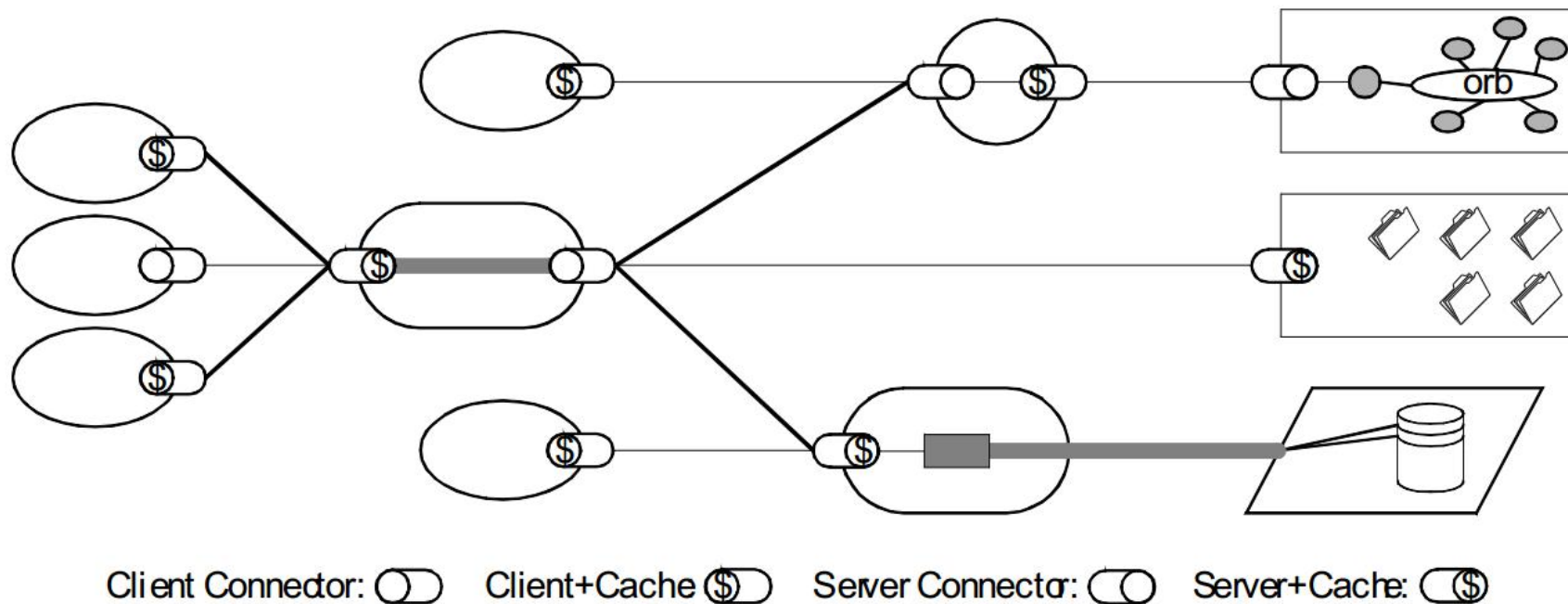
复制风格 Replication Styles

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
RR			++		+									+
\$	RR		+	+	+	+								

分层风格 Hierarchical Styles (一)

- 客户端服务器 Client-Server, CS
 - 由 Client 触发请求, Server 监听到请求后产生响应, Client 一直等待收到响应后, 会话结束
 - 分离关注点隐藏细节, 良好的简单性、可伸缩性、可进化性
- 分层系统 Layered System, LS
 - 每一层为其之上的层服务, 并使用在其之下的层所提供的服务, 例如 TCP/IP
- 分层客户端服务器 Layered Client-Server, LCS
 - LS+CS, 例如正向代理和反向代理, 从空间上分为外部层与内部层
- 无状态、客户端服务器 Client-Stateless-Server CSS
 - 基于 CS, 服务器上不允许有 session state 会话状态
 - 提升了可见性、可伸缩性、可靠性, 但重复数据导致降低网络性能
- 缓存、无状态、客户端服务器 Client-Cache-Stateless-Server C\$SS
 - 提升性能
- 分层、缓存、无状态、客户端服务器 Layered-Client-Cache-Stateless-Server, LC\$SS

LC\$SS 架构



分层风格 Hierarchical Styles (二)

- 远程会话 Remote Session, RS
 - CS 变体, 服务器保存 Application state 应用状态
 - 可伸缩性、可见性差
- 远程数据访问 Remote Data Access , RDA
 - CS 变体, Application state 应用状态同时分布在客户端与服务器
 - 巨大的数据集有可能通过迭代而减少
 - 简单性、可伸缩性差

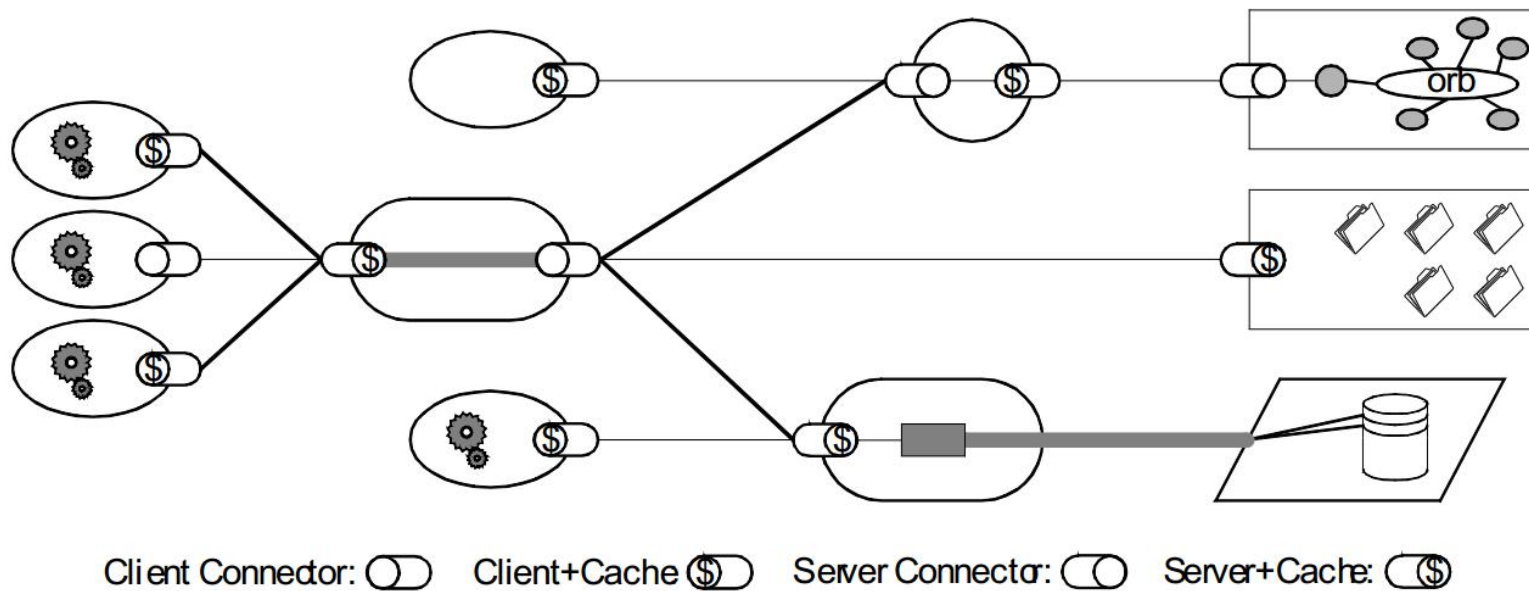
分层风格 Hierarchical Styles

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
CS					+	+	+							
LS			-		+		+				+		+	
LCS	CS+LS		-		++	+	++				+		+	
CSS	CS	-			++	+	+					+		+
C\$SS	CSS+\$	-	+	+	++	+	+					+		+
LC\$SS	LCS+C\$SS	-	±	+	+++	++	++				+	+	+	+
RS	CS			+	-	+	+					-		
RDA	CS			+	-	-						+		-

移动代码风格 Mobile Code Styles

- 虚拟机 Virtual Machine, VM
 - 分离指令与实现
- 远程求值 Remote Evaluation, REV
 - 基于 CS 的 VM, 将代码发送至服务器执行
- 按需代码 Code on Demand, COD
 - 服务器在响应中发回处理代码, 在客户端执行
 - 优秀的可扩展性和可配置性, 提升用户可察觉性能和网络效率
- 分层、按需代码、缓存、无状态、客户端服务器
Layered-Code-on-Demand-Client-Cache-Stateless-Server, LCODC\$SS
 - LC\$SS+COD
- 移动代理 Mobile Agent, MA
 - 相当于 REV+COD

统一接口的 LC\$SS+COD



移动代码风格 Mobile Code Styles

代码与执行过程或者执行结果分离，例如 Javascript

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
VM						±		+				-	+	
REV	CS+VM			+	-	±		+	+			-	+	-
COD	CS+VM		+	+	+	±		+		+		-		
LCODC\$SS	LC\$SS+COD	-	++	++	+4+	+±+	++	+		+	+	±	+	+
MA	REV+COD		+	++		±		++	+	+		-	+	

点对点风格 Peer-to-Peer Styles

- Event-based Integration , EBI:

- 基于事件集成系统, 如由类似 Kafka 这样的消息系统 + 分发订阅来消除耦合
- 优秀的可重用性、可扩展性、可进化性
- 缺乏可理解性
- 由于消息广播等因素造成的消息风暴, 可伸缩性差

- Chiron-2,C2

参

见论文 《A Component- and Message-Based Architectural Style for GUI Software》

- 相当于 EBI+LCS, 控制了消息的方向

- Distributed Objects , DO

- 组件结对交互

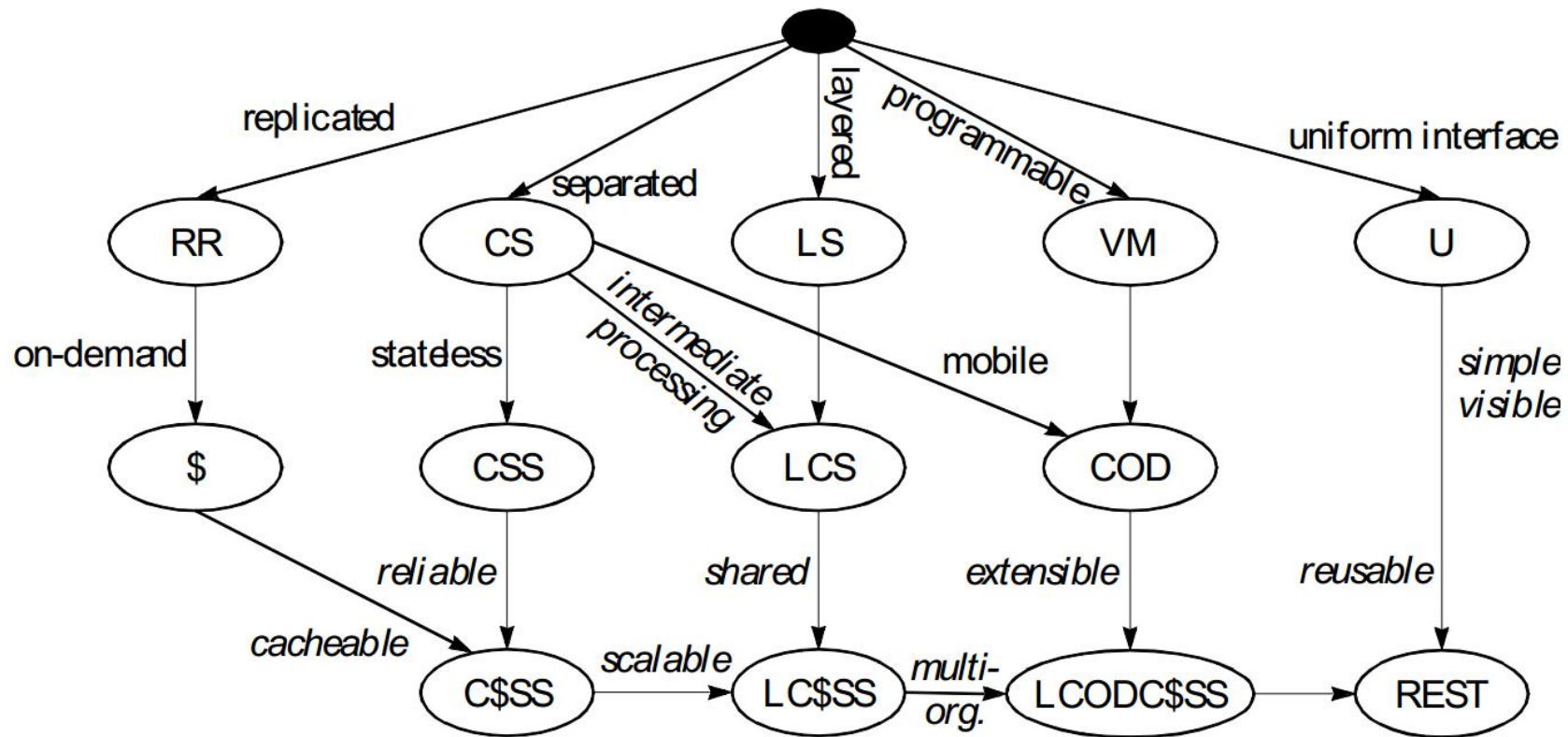
- Brokered Distributed Objects , BDO

- 引入名字解析组件来简化 DO, 例如 CORBA

点对点风格 Peer-to-Peer Styles

Style	Derivation	Net Perform.	UP Perform.	Efficiency	Scalability	Simplicity	Evolvability	Extensibility	Customiz.	Configur.	Reusability	Visibility	Portability	Reliability
EBI				+	- -	±	+	+		+	+	-		-
C2	EBI+LCS		-	+		+	++	+		+	++	±	+	±
DO	CS+CS	-		+			+	+		+	+	-		-
BDO	DO+LCS	-	-				++	+		+	++	-	+	

风格演化



第 6 课

Chrome 抓包：快速定位 HTTP 协议问题

- <https://developers.google.com/web/tools/chrome-devtools/network/>
- 快捷键：Control+Shift+I (*Windows*) or Command+Option+I (*Mac*)

Chrome 抓包：Network 面板

控制器

过滤器

概览

请求列表

概要

The screenshot shows the Chrome DevTools Network panel. At the top, there's a toolbar with icons for recording, pausing, and filtering. Below the toolbar is a filter bar with tabs for 'All', 'XHR', 'JS', 'CSS', 'Img', 'Media', 'Font', 'Doc', 'WS', 'Manifest', and 'Other'. The 'All' tab is selected. Below the filter bar is a timeline view showing a sequence of requests over time. The main part of the panel is a table of requests. The table has columns for Name, Status, Protocol, Type, Initiator, Size, Time, Priority, and Waterfall. The first request is 'www.taohui.pub' with a status of 200, protocol of h2, type of document, and a size of 13.7 KB. The second request is 'default.css?ver=0.9.2' with a status of 200, protocol of h2, type of stylesheet, and a size of 1.2 KB. The third request is 'pure-highlight.css?ver=0.1.0' with a status of 200, protocol of h2, type of stylesheet, and a size of 288 B. The fourth request is 'bootstrap.css?ver=4.9.9' with a status of 200, protocol of h2, type of stylesheet, and a size of 144 KB. The fifth request is 'font-awesome.min.css?ver=4.9.9' with a status of 200, protocol of h2, type of stylesheet, and a size of 23.4 KB. The sixth request is 'style.css?ver=4.9.9' with a status of 200, protocol of h2, type of stylesheet, and a size of 34.1 KB. The seventh request is 'css?family=Lato:400,300italic,700|Source+Sans+Pr...' with a status of 200, protocol of h2, type of stylesheet, and a size of 1.0 KB. The eighth request is 'social.css?ver=1.2.6' with a status of 200, protocol of h2, type of stylesheet, and a size of 9.7 KB. The ninth request is '%E5%B8%83%E9%9A%86%E8%BF%87%E6%BB%A...' with a status of 200, protocol of h2, type of jpeg, and a size of 60.7 KB. At the bottom of the panel, there's a summary bar showing '59 requests | 2.4 MB transferred | Finish: 43.89 s | DOMContentLoaded: 4.59 s | Load: 10.11 s'.

Name	Status	Protocol	Type	Initiator	Size	Time	Priority	Waterfall
www.taohui.pub	200	h2	document	Other	13.7 KB	285 ms	Highest	
default.css?ver=0.9.2	200	h2	stylesheet	(index)	1.2 KB	284 ms	Highest	
pure-highlight.css?ver=0.1.0	200	h2	stylesheet	(index)	288 B	284 ms	Highest	
bootstrap.css?ver=4.9.9	200	h2	stylesheet	(index)	144 KB	659 ms	Highest	
font-awesome.min.css?ver=4.9.9	200	h2	stylesheet	(index)	23.4 KB	673 ms	Highest	
style.css?ver=4.9.9	200	h2	stylesheet	(index)	34.1 KB	719 ms	Highest	
css?family=Lato:400,300italic,700 Source+Sans+Pr...	200	h2	stylesheet	(index)	1.0 KB	1.30 s	Highest	
social.css?ver=1.2.6	200	h2	stylesheet	(index)	9.7 KB	1.13 s	Highest	
%E5%B8%83%E9%9A%86%E8%BF%87%E6%BB%A...	200	h2	jpeg	(index)	60.7 KB	2.32 s	High	

59 requests | 2.4 MB transferred | Finish: 43.89 s | DOMContentLoaded: 4.59 s | Load: 10.11 s

Network 面板

- 控制器：控制面板的外观与功能
- 过滤器：过滤请求列表中显示的资源
 - 按住 Command (Mac) 或 Ctrl (Window / Linux) ，然后点击过滤器可以同时选择多个过滤器
- 概览：显示 HTTP 请求、响应的时间轴
- 请求列表：默认时间排序，可选择显示列
- 概要：请求总数、总数据量、总花费时间等

控制器

- 抓包 
- 停止抓包
- 清除请求 
- 要跨页面加载保存请求: Preserve log
- 屏幕截图: Capture screenshots 
- 重新执行 XHR 请求: 右键点击请求选择 Replay XHR
- 停用浏览器缓存
- 手动清除浏览器缓存: 右键点击请求选择 Clear Browser Cache
- 离线模拟: Offline
- 模拟慢速网络连接: Network Throttling, 可自定义网速
- 手动清除浏览器 Cookie: 右键点击请求选择 Clear Browser Cookies
- 隐藏 Filters 窗格 
- 隐藏 Overview 窗格 

过滤器：按类型

- XHR、JS、CSS、Img、Media、Font、Doc、WS (WebSocket)、Manifest 或 Other (此处未列出的任何其他类型)
- 多类型，按住 Command (Mac) 或 Ctrl (Windows、Linux)
- 按时间过滤：概览面板，拖动滚动条
- 隐藏 Data URLs：CSS 图片等小文件以 BASE64 格式嵌入 HTML 中，以减少 HTTP 请求数

过滤器：属性过滤（一）

- **domain**: 仅显示来自指定域的资源。 您可以使用通配符字符 (*) 纳入多个域
- **has-response-header**: 显示包含指定 HTTP 响应标头的资源
- **is**: 使用 is:running 可以查找 WebSocket 资源, is:from-cache 可查找缓存读出的资源
- **larger-than**: 显示大于指定大小的资源（以字节为单位）。 将值设为 1000 等同于设置为 1k
- **method**: 显示通过指定 HTTP 方法类型检索的资源
- **mime-type**: 显示指定 MIME 类型的资源

多属性间通过空格实现 AND 操作

过滤器：属性过滤（二）

- **mixed-content**: 显示所有混合内容资源 (mixed-content:all), 或者仅显示当前显示的资源 (mixed-content:displayed)。
- **scheme**: 显示通过未保护 HTTP (scheme:http) 或受保护 HTTPS (scheme:https) 检索的资源。
- **set-cookie-domain**: 显示具有 Set-Cookie 标头并且 Domain 属性与指定值匹配的资源。
- **set-cookie-name**: 显示具有 Set-Cookie 标头并且名称与指定值匹配的资源。
- **set-cookie-value**: 显示具有 Set-Cookie 标头并且值与指定值匹配的资源。
- **status-code**: 仅显示 HTTP 状态代码与指定代码匹配的资源。

多属性间通过空格实现 AND 操作

请求列表的排序

- 时间排序，默认
- 按列排序
- 按活动时间排序
 - Start Time: 发出的第一个请求位于顶部
 - Response Time: 开始下载的第一个请求位于顶部
 - End Time: 完成的第一个请求位于顶部
 - Total Duration: 连接设置时间和请求/响应时间最短的请求位于顶部
 - Latency: 等待最短响应时间的请求位于顶部

请求列表（一）

- **Name** : 资源的名称
- **Status** : HTTP 状态代码
- **Type** : 请求的资源的 MIME 类型

请求列表（二）

Initiator：发起请求的对象或进程。它可能有以下几种值：

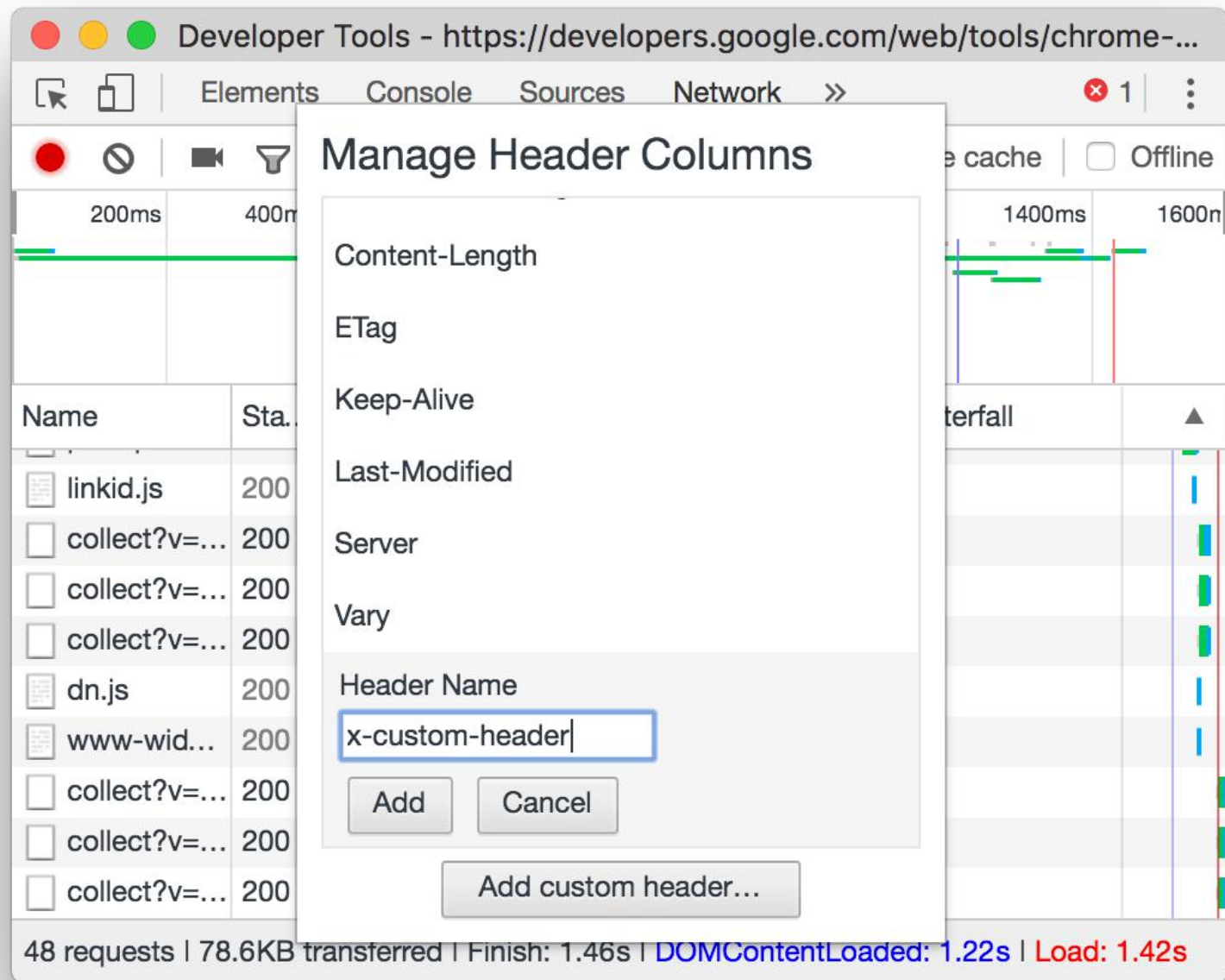
- **Parser（解析器）**：Chrome 的 HTML 解析器发起了请求
 - 鼠标悬停显示 JS 脚本
- **Redirect（重定向）**：HTTP 重定向启动了请求
- **Script（脚本）**：脚本启动了请求
- **Other（其他）**：一些其他进程或动作发起请求，例如用户点击链接跳转到页面或在地址栏中输入网址

请求列表（三）

- **Size**：服务器返回的响应大小（包括头部和包体），可显示解压后大小
- **Time**：总持续时间，从请求的开始到接收响应中的最后一个字节
- **Waterfall**：各请求相关活动的直观分析图

请求列表

- 添加其他列
 - 添加响应列
- 添加自定义列



预览请求内容（一）

- 查看头部
- 查看 cookie
- 预览响应正文：查看图像用
- 查看响应正文
- 时间详细分布
- 导出数据为 HAR 格式
- 查看未压缩的资源大小：Use Large Request Rows

预览请求内容（二）

- 浏览器加载时间（概览、概要、请求列表）
 - DOMContentLoaded 事件的颜色设置为蓝色，而 load 事件设置为红色
- 将请求数据复制到剪贴版
 - Copy Link Address: 将请求的网址复制到剪贴板
 - Copy Response: 将响应包体复制到剪贴板
 - Copy as cURL: 以 cURL 命令形式复制请求
 - Copy All as cURL: 以一系列 cURL 命令形式复制所有请求
 - Copy All as HAR: 以 HAR 数据形式复制所有请求
- 查看请求上下游：按住 shift 键悬停请求上，绿色是上游，红色是下游

浏览器加载时间

- 触发流程：
 - 解析 HTML 结构
 - 加载外部脚本和样式表文件
 - 解析并执行脚本代码 // 部分脚本会阻塞页面的加载
 - DOM 树构建完成 // DOMContentLoaded 事件
 - 加载图片等外部文件
 - 页面加载完毕 // load 事件

请求时间详细分布（一）

- **Queueing:** 浏览器在以下情况下对请求排队
 - 存在更高优先级的请求
 - 此源已打开六个 TCP 连接，达到限值，仅适用于 HTTP/1.0 和 HTTP/1.1
 - 浏览器正在短暂分配磁盘缓存中的空间
- **Stalled:** 请求可能会因 Queueing 中描述的任何原因而停止
- **DNS Lookup:** 浏览器正在解析请求的 IP 地址
- **Proxy Negotiation:** 浏览器正在与代理服务器协商请求

请求时间详细分布（二）

- **Request sent:** 正在发送请求
- **ServiceWorker Preparation:** 浏览器正在启动 Service Worker
- **Request to ServiceWorker:** 正在将请求发送到 Service Worker
- **Waiting (TTFB):** 浏览器正在等待响应的第一个字节。TTFB 表示 Time To First Byte（至第一字节的时间）。此时间包括 1 次往返延迟时间及服务器准备响应所用的时间
- **Content Download:** 浏览器正在接收响应
- **Receiving Push:** 浏览器正在通过 HTTP/2 服务器推送接收此响应的数据
- **Reading Push:** 浏览器正在读取之前收到的本地数据

第 6 课

当没有 URI 时

- 站长 A 欲分享一部电影 Forrest Gump 给 B, 需要告诉:
 - 请使用 FTP 协议访问 mysite.net, 端口是 8502
 - 登录用户名是user, 密码pass
 - 进入到 /shared/movie 目录下
 - 转换为二进制模式
 - 下载名为 Forrest Gump.mkv 格式的文件
- 有了 URI: ftp://user:pass@mysite.net:8502/shared/movie/Forrest Gump.mkv

什么是 URI

- **URL**: RFC1738 (1994.12) , Uniform Resource Locator, 表示资源的位置, 期望提供查找资源的方法
- **URN**: RFC2141 (1997.5) , Uniform Resource Name, 期望为资源提供持久的、位置无关的标识方式, 并允许简单地将多个命名空间映射到单个URN命名空间
 - 例如磁力链接 `magnet:?xt=urn:sha1:YNCKHTQC5C`
- **URI**: RFC1630 (1994.6) 、 RFC3986 (2005.1, 取代 RFC2396 和 RFC2732) , Uniform Resource Identifier, 用以区分资源, 是 URL 和 URN 的超集, 用以取代 URL 和 URN 概念

Uniform Resource Identifier 统一资源标识符

- Resource 资源

- 可以是图片、文档、今天杭州的温度等，也可以是不能通过互联网访问的实体，例如人、公司、实体书，也可以是抽象的概念，例如亲属关系或者数字符号
- 一个资源可以有多个 URI

- Identifier 标识符

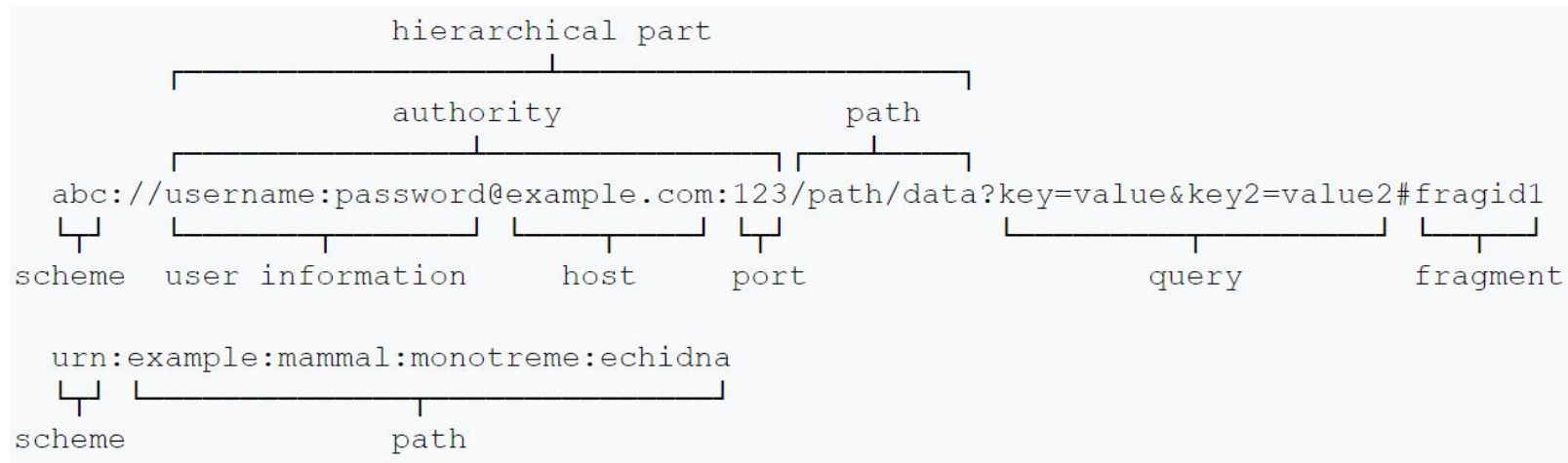
- 将当前资源与其他资源区分开的名称

- Uniform 统一

- 允许不同种类的资源在同一上下文中出现
- 对不同种类的资源标识符可以使用同一种语义进行解读
- 引入新标识符时，不会对已有标识符产生影响
- 允许同一资源标识符在不同的、internet 规模下的上下文中出现

URI 的组成

- 组成: schema, user information, host, port, path, query, fragment



<https://tools.ietf.org/html/rfc7231?test=1#page-7>

合法的 URI

- <ftp://ftp.is.co.za/rfc/rfc1808.txt>
- <http://www.ietf.org/rfc/rfc2396.txt>
- ldap://[2001:db8::7]/c=GB?objectClass?one
- mailto:John.Doe@example.com
- news:comp.infosystems.www.servers.unix tel:+1-816-555-1212
- telnet://192.0.2.16:80/
- urn:oasis:names:specification:docbook:dtd:xml:4.1.2

URI 格式 (一)

- URI = **scheme** ":" **hier-part** ["?" **query**] ["#" **fragment**]
- **scheme** = ALPHA *(ALPHA / DIGIT / "+" / "-" / ".")
 - 例如: http, https, ftp,mailto,rtsp,file,telnet
- **query** = *(pchar / "/" / "?")
- **fragment** = *(pchar / "/" / "?")

示例: <https://tools.ietf.org/html/rfc7231?test=1#page-7>

URI 格式 (二)

hier-part = "://" **authority** path-abempty / path-absolute / path-rootless / path-empty

- **authority** = [userinfo "@"] host [":" port]
 - **userinfo** = *(unreserved / pct-encoded / sub-delims / ":")
 - **host** = IP-literal / IPv4address / reg-name
 - **port** = *DIGIT

示例: `https://tom:pass@localhost:8080/index.html`

URI格式 (三) : hier-part

- **path = path-abempty/ path-absolute/ path-noscheme / path-rootless / path-empty**
 - path-abempty = `*( "/" segment )`
 - 以/开头的路径或者空路径
 - path-absolute = `"/" [ segment-nz *( "/" segment ) ]`
 - 以/开头的路径, 但不能以//开头
 - path-noscheme = `segment-nz-nc *( "/" segment )`
 - 以非:号开头的路径
 - path-rootless = `segment-nz *( "/" segment )`
 - 相对path-noscheme, 增加允许以:号开头的路径
 - path-empty = `0<pchar>`
 - 空路径

相对 URI

URI-reference = **URI**/**relative-ref**

- **relative-ref** = **relative-part** ["?" query] ["#" fragment]
 - **relative-part** = "//" authority path-abempty / path-absolute / path-noscheme / path-empty

https://tools.ietf.org/html/rfc7231?test=1#page-7

/html/rfc7231?test=1#page-7

第7课

为什么要进行 URI 编码

- 传递数据中，如果存在用作分隔符的保留字符怎么办？
- 对可能产生歧义性的数据编码
 - 不在 ASCII 码范围内的字符
 - ASCII 码中不可显示的字符
 - URI 中规定的保留字符
 - 不安全字符（传输环节中可能会被不正确处理），如空格、引号、尖括号等

示例：

<https://www.baidu.com/s?wd=?#!>

<https://www.baidu.com/s?wd=极客 时间>

<https://www.baidu.com/s?wd=极客 '>时 间>

保留字符与非保留字符

- 保留字符

- reserved = gen-delims / sub-delims

- gen-delims = ":" / "/" / "?" / "#" / "[" / "]" / "@"

- sub-delims = "!" / "\$" / "&" / "'" / "(" / ")" / "*" / "+" / "," / ";" / "="

- 非保留字符

- unreserved = ALPHA / DIGIT / "-" / "." / "_" / "~"

- ALPHA: %41-%5A and %61-%7A

- DIGIT: %30-%39

- : %2D .: %2E _: %5F

- ~: %7E, 某些实现将其认为保留字符

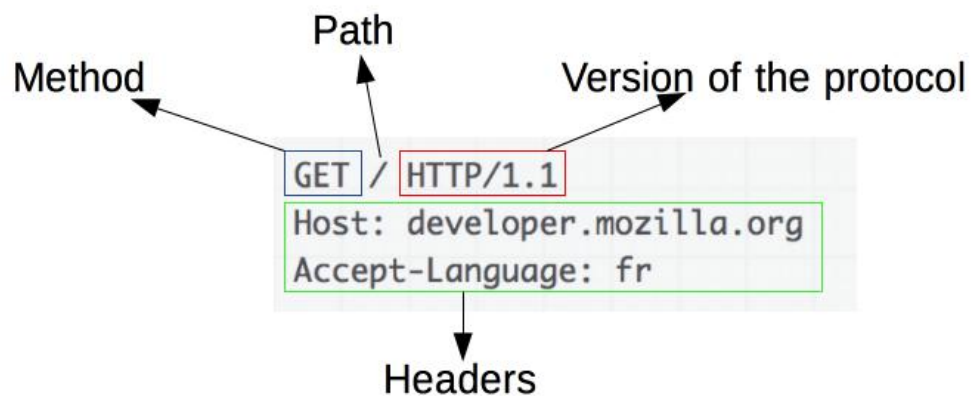
URI 百分号编码

- 百分号编码的方式
 - pct-encoded = "%" HEXDIG HEXDIG
 - US-ASCII: 128 个字符 (95 个可显示字符, 33 个不可显示字符)
 - 参见: <https://zh.wikipedia.org/wiki/ASCII>
 - 对于 HEXDIG 十六进制中的字母, 大小写等价
- 非 ASCII 码字符 (例如中文): 建议先 UTF8 编码, 再 US-ASCII 编码
- 对 URI 合法字符, 编码与不编码是等价的
 - 例如, "URI 转换"既可以"URI%e8%bd%ac%e6%8d%a", 也可以"%55%52%49%e8%bd%ac%e6%8d%a2"
 - <https://www.baidu.com/s?wd=URI%20%e8%bd%ac%e6%8d%a2>
 - <https://www.baidu.com/s?wd=%55%52%49%20%e8%bd%ac%e6%8d%a2>

第 7 课

请求行（一）

request-line = method SP request-target SP HTTP-version CRLF



- method 方法：指明操作目的，动词
- request-target = origin-form / absolute-form / authority-form / asterisk-form

请求行 (二)

request-target = origin-form / absolute-form / authority-form / asterisk-form

- origin-form = absolute-path ["?" query]
 - 向 origin server 发起的请求, path 为空时必须传递 /
- absolute-form = absolute-URI
 - 仅用于向正向代理 proxy 发起请求时, 详见正向代理与隧道
- authority-form = authority
 - 仅用于 CONNECT 方法, 例如 CONNECT www.example.com:80 HTTP/1.1
- asterisk-form = "*"
 - 仅用于 OPTIONS 方法

请求行 (三)

HTTP-version 版本号发展历史: <https://www.w3.org/Protocols/History.html>

- HTTP/0.9: 只支持 GET 方法, 过时
- HTTP/ 1.0: RFC1945, 1996, 常见使用于代理服务器 (例如 Nginx 默认配置)
- HTTP/ 1.1: RFC2616, 1999
- HTTP/ 2.0: 2015.5 正式发布

常见方法 (RFC7231)

- GET: 主要的获取信息方法, 大量的性能优化都针对该方法, 幂等方法
- HEAD: 类似 GET 方法, 但服务器不发送 BODY, 用以获取 HEAD 元数据, 幂等方法
- POST: 常用于提交 HTML FORM 表单、新增资源等
- PUT: 更新资源, 带条件时是幂等方法
- DELETE: 删除资源, 幂等方法
- CONNECT: 建立 tunnel 隧道
- OPTIONS: 显示服务器对访问资源支持的方法, 幂等方法
- TRACE: 回显服务器收到的请求, 用于定位问题。有安全风险

Changes with nginx 0.5.17

02 Apr 2007

*) Change: now nginx always returns the 405 status for the TRACE method.

用于文档管理的 WEBDAV 方法(RFC2518)

- **PROPFIND**: 从 Web 资源中检索以 XML 格式存储的属性。它也被重载, 以允许一个检索远程系统的集合结构 (也叫目录层次结构)
- **PROPPATCH**: 在单个原子性动作中更改和删除资源的多个属性
- **MKCOL**: 创建集合或者目录
- **COPY**: 将资源从一个 URI 复制到另一个 URI
- **MOVE**: 将资源从一个 URI 移动到另一个 URI
- **LOCK**: 锁定一个资源。WebDAV 支持共享锁和互斥锁。
- **UNLOCK**: 解除资源的锁定

WEBDAV 验证环境

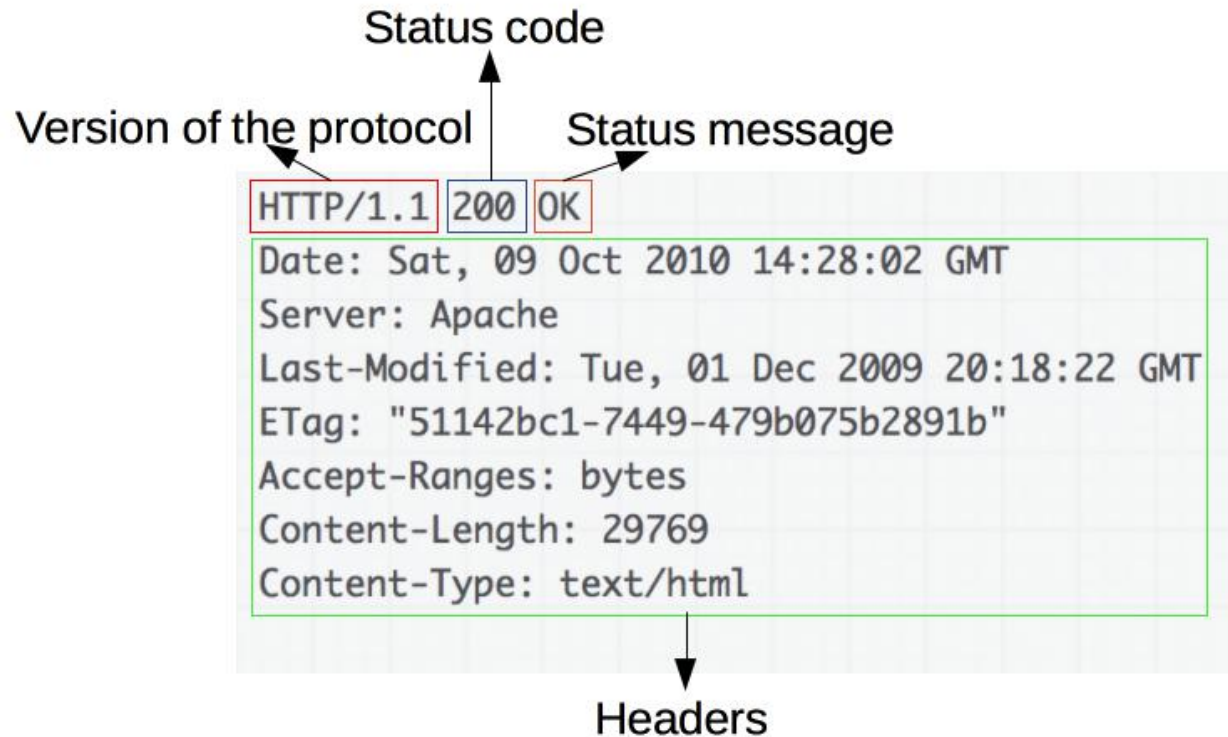
- 服务器
 - Nginx
 - http_dav_module 模块
 - nginx-dav-ext-module 模块
- 客户端
 - winscp

第 8 课

HTTP 响应行

status-line = HTTP-version SP status-code SP reason-phrase CRLF

- status-code = 3DIGIT
- reason-phrase = *(HTAB / SP / VCHAR / obs-text)



响应码分类：1xx

- 响应码规范：RFC6585 (2012.4) 、 RFC7231 (2014.6)
- 1xx：请求已接收到，需要进一步处理才能完成，HTTP1.0 不支持
 - 100 Continue：上传大文件前使用
 - 由客户端发起请求中携带 Expect: 100-continue 头部触发
 - 101 Switch Protocols：协议升级使用
 - 由客户端发起请求中携带 Upgrade: 头部触发，如升级 websocket 或者 http/2.0
 - 102 Processing：WebDAV 请求可能包含许多涉及文件操作的子请求，需要很长时间才能完成请求。该代码表示服务器已经收到并正在处理请求，但无响应可用。这样可以防止客户端超时，并假设请求丢失

响应码分类： 2xx（一）

- 2xx：成功处理请求
 - 200 OK: 成功返回响应。
 - 201 Created: 有新资源在服务器端被成功创建。
 - 202 Accepted: 服务器接收并开始处理请求，但请求未处理完成。这样一个模糊的概念是有意如此设计，可以覆盖更多的场景。例如异步、需要长时间处理的任务。
 - 203 Non-Authoritative Information: 当代理服务器修改了 origin server 的原始响应包体时（例如更换了HTML中的元素值），代理服务器可以通过修改200为203的方式告知客户端这一事实，方便客户端为这一行为作出相应的处理。203响应可以被缓存。

响应码分类： 2xx (二)

- **204 No Content**: 成功执行了请求且不携带响应包体，并暗示客户端无需更新当前的页面视图。
- **205 Reset Content**: 成功执行了请求且不携带响应包体，同时指明客户端需要更新当前页面视图。
- **206 Partial Content**: 使用 range 协议时返回部分响应内容时的响应码
- **207 Multi-Status**: RFC4918，在 WEBDAV 协议中以 XML 返回多个资源的状态。
- **208 Already Reported**: RFC5842，为避免相同集合下资源在207响应码下重复上报，使用 208 可以使用父集合的响应码。

响应码分类： 3xx (一)

- **3xx**: 重定向使用 Location 指向的资源或者缓存中的资源。在 RFC2068 中规定客户端重定向次数不应超过 5 次，以防止死循环。
 - **300 Multiple Choices**: 资源有多种表述，通过 300 返回给客户端后由其自行选择访问哪一种表述。由于缺乏明确的细节，300 很少使用。
 - **301 Moved Permanently**: 资源永久性的重定向到另一个 URI 中。
 - **302 Found**: 资源临时的重定向到另一个 URI 中。

响应码分类： 3xx (二)

- **303 See Other**: 重定向到其他资源，常用于 POST/PUT 等方法的响应中。
- **304 Not Modified**: 当客户端拥有可能过期的缓存时，会携带缓存的标识 etag、时间等信息询问服务器缓存是否仍可复用，而304是告诉客户端可以复用缓存。
- **307 Temporary Redirect**: 类似302，但明确重定向后请求方法必须与原请求方法相同，不得改变。
- **308 Permanent Redirect**: 类似301，但明确重定向后请求方法必须与原请求方法相同，不得改变。

第 9 课

响应码分类： 4xx（一）

- 4xx：客户端出现错误
 - 400 Bad Request：服务器认为客户端出现了错误，但不能明确判断为以下哪种错误时使用此错误码。例如HTTP请求格式错误。
 - 401 Unauthorized：用户认证信息缺失或者不正确，导致服务器无法处理请求。
 - 407 Proxy Authentication Required：对需要经由代理的请求，认证信息未通过代理服务器的验证
 - 403 Forbidden：服务器理解请求的含义，但没有权限执行此请求
 - 404 Not Found：服务器没有找到对应的资源
 - 410 Gone：服务器没有找到对应的资源，且明确的知道该位置永久性找不到该资源

响应码分类： 4xx (二)

- **405 Method Not Allowed**: 服务器不支持请求行中的 method 方法
- **406 Not Acceptable**: 对客户端指定的资源表述不存在（例如对语言或者编码有要求），服务器返回表述列表供客户端选择。
- **408 Request Timeout**: 服务器接收请求超时
- **409 Conflict**: 资源冲突，例如上传文件时目标位置已经存在版本更新的资源
- **411 Length Required**: 如果请求含有包体且未携带 Content-Length 头部，且不属于 chunk 类请求时，返回 411

响应码分类： 4xx (三)

- **412 Precondition Failed**: 复用缓存时传递的 If-Unmodified-Since 或 If-None-Match 头部不被满足
- **413 Payload Too Large/Request Entity Too Large**: 请求的包体超出服务器能处理的最大长度
- **414 URI Too Long**: 请求的 URI 超出服务器能接受的最大长度
- **415 Unsupported Media Type**: 上传的文件类型不被服务器支持
- **416 Range Not Satisfiable**: 无法提供 Range 请求中指定的那段包体
- **417 Expectation Failed**: 对于 Expect 请求头部期待的情况无法满足时的响应码

响应码分类： 4xx（四）

- 421 Misdirected Request: 服务器认为这个请求不该发给它，因为它没有能力处理。
- 426 Upgrade Required: 服务器拒绝基于当前 HTTP 协议提供服务，通过 Upgrade 头部告知客户端必须升级协议才能继续处理。
- 428 Precondition Required: 用户请求中缺失了条件类头部，例如 If-Match
- 429 Too Many Requests: 客户端发送请求的速率过快
- 431 Request Header Fields Too Large: 请求的 HEADER 头部大小超过限制
- 451 Unavailable For Legal Reasons: RFC7725，由于法律原因资源不可访问

响应码分类： 5xx (一)

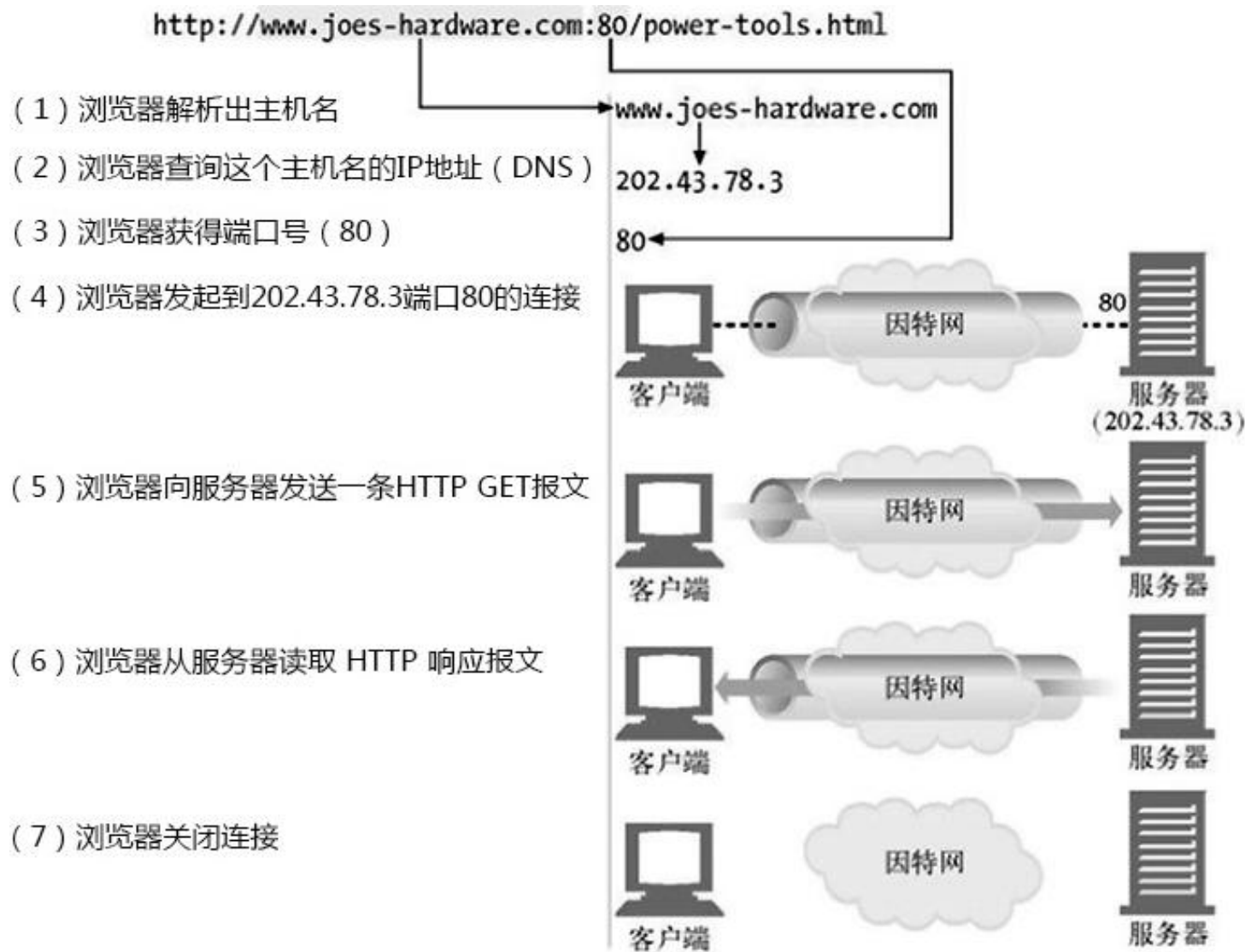
- 5xx: 服务器端出现错误
 - 500 Internal Server Error: 服务器内部错误, 且不属于以下错误类型
 - 501 Not Implemented: 服务器不支持实现请求所需要的功能
 - 502 Bad Gateway: 代理服务器无法获取到合法响应
 - 503 Service Unavailable: 服务器资源尚未准备好处理当前请求

响应码分类： 5xx (二)

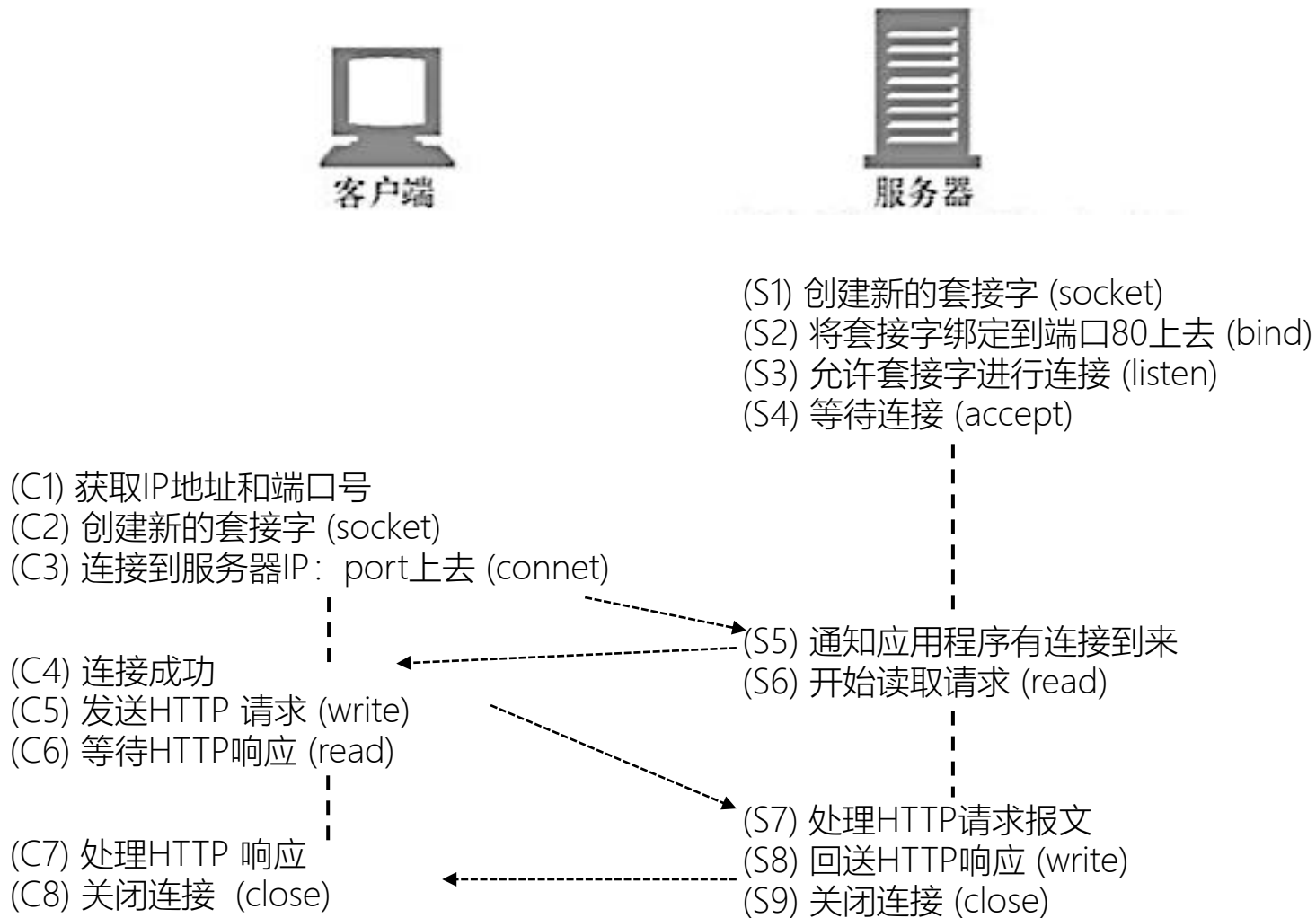
- 504 Gateway Timeout: 代理服务器无法及时的从上游获得响应
- 505 HTTP Version Not Supported: 请求使用的 HTTP 协议版本不支持
- 507 Insufficient Storage: 服务器没有足够的空间处理请求
- 508 Loop Detected: 访问资源时检测到循环
- 511 Network Authentication Required: 代理服务器发现客户端需要进行身份验证才能获得网络访问权限

第 10 课

HTTP 连接的常见流程

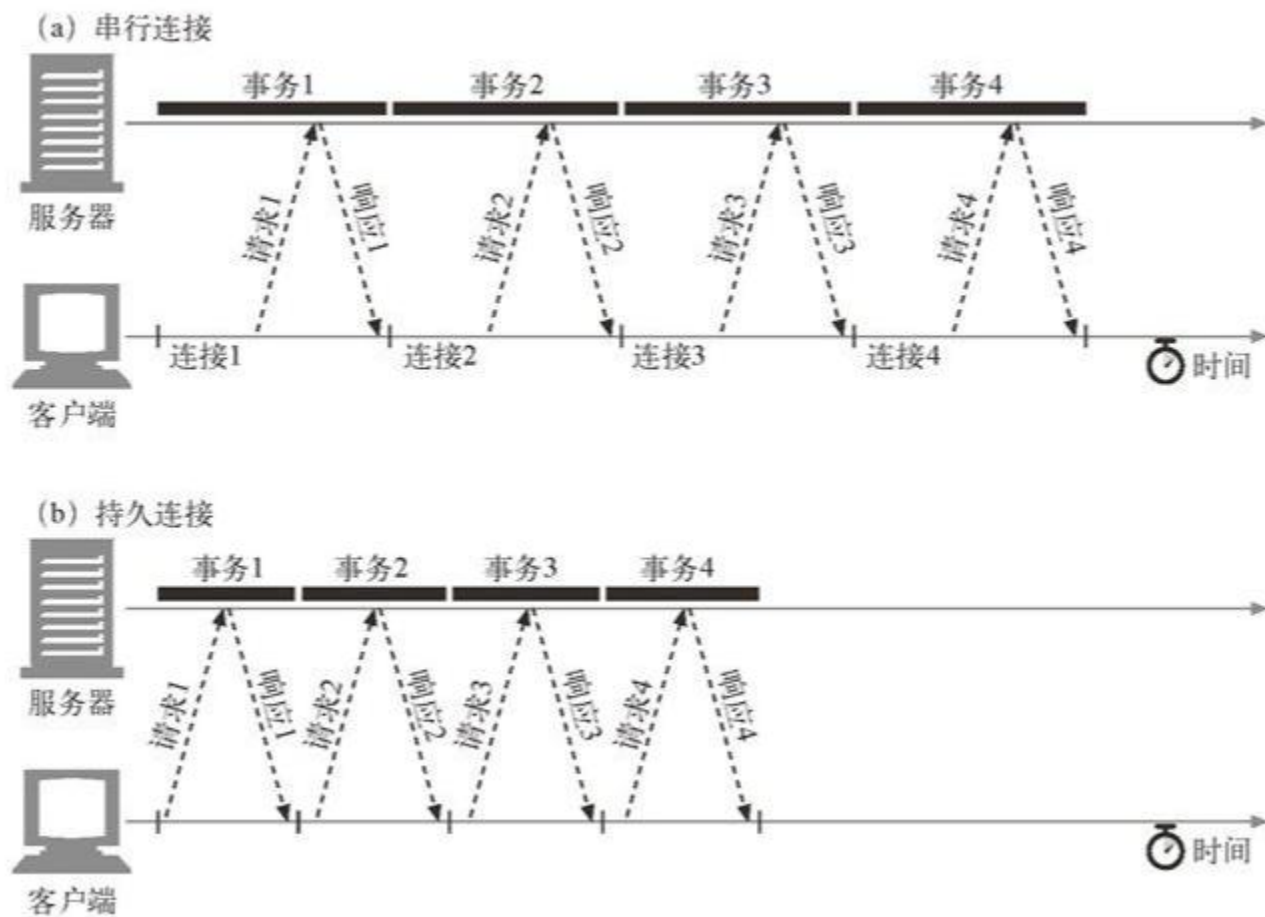


从 TCP 编程上看 HTTP 请求处理



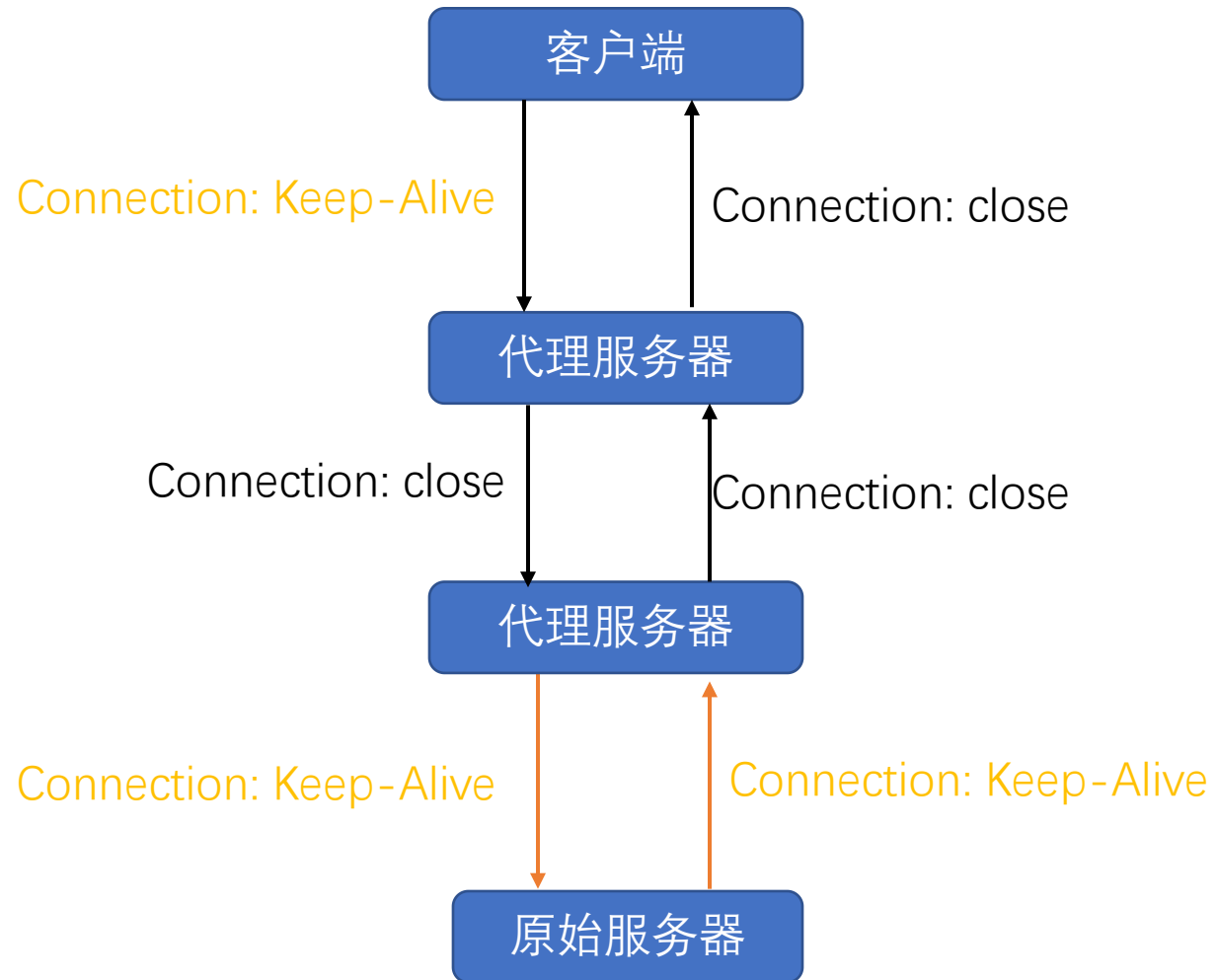
短连接与长连接

- Connection 头部
 - Keep-Alive: 长连接
 - 客户端请求长连接
 - Connection: Keep-Alive
 - 服务器表示支持长连接
 - Connection: Keep-Alive
 - 客户端复用连接
 - HTTP/1.1 默认支持长连接
 - Connection: Keep-Alive 无意义
 - Close: 短连接
 - 对代理服务器的要求
 - 不转发 Connection 列出头部, 该头部仅与当前连接相关



Connection 仅针对当前连接有效

user agent 与 origin server
间有层层 proxy 代理



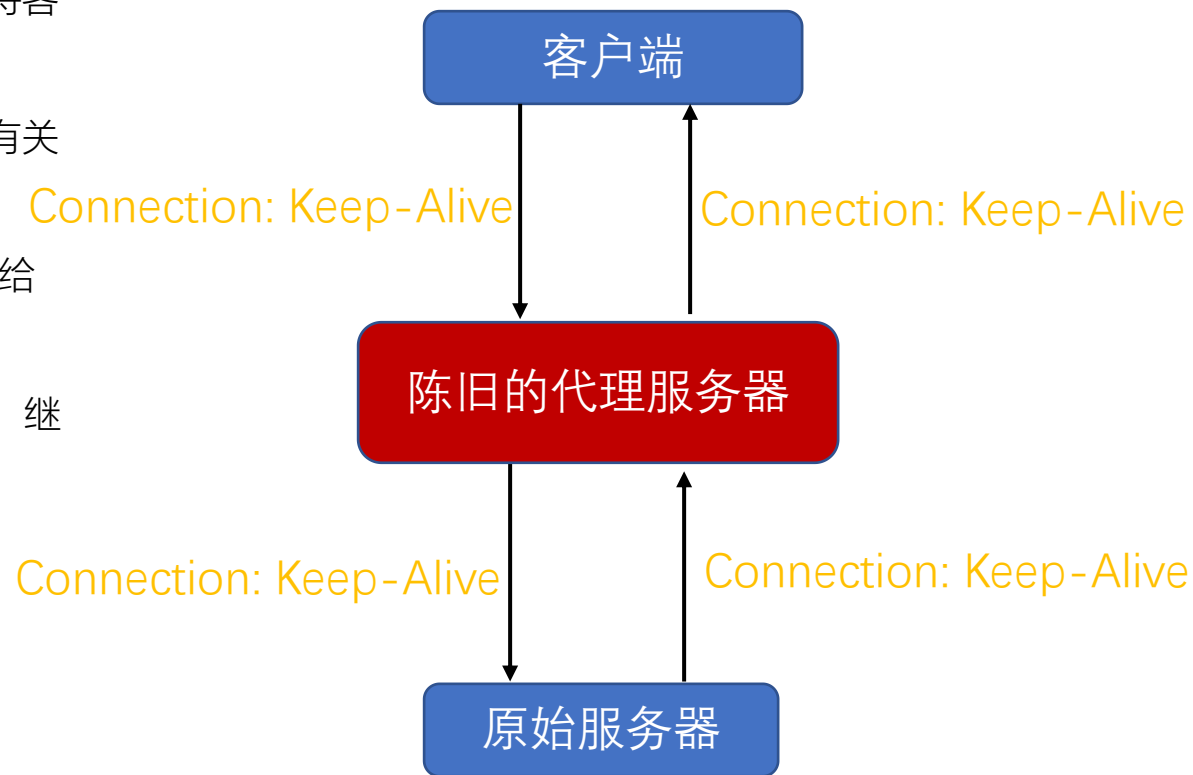
代理服务器对长连接的支持

- 问题：各方错误使用了长连接

- 客户端发起长连接
- 代理服务器陈旧，不能正确的处理请求的 Connection 头部，将客户端请求中的 Connection: Keep-Alive 原样转发给上游服务器
- 上游服务器正确的处理了 Connection 头部，在发送响应后没有关闭连接，而试图保持、复用与不认长连接的代理服务器的连接
- 代理服务器收到响应中 Connection: Keep-Alive 后不认，转发给客户端，同时等待服务器关闭短连接
- 客户端收到了 Connection: Keep-Alive，认为可以复用长连接，继续在该连接上发起请求
- 代理服务器出错，因为短连接上不能发起两次请求

- Proxy-Connection

- 陈旧的代理服务器不识别该头部：退化为短连接
- 新版本的代理服务器理解该头部
 - 与客户端建立长连接
 - 与服务器使用 Connection 替代 Proxy-Connect 头部



第 15课

Host 头部

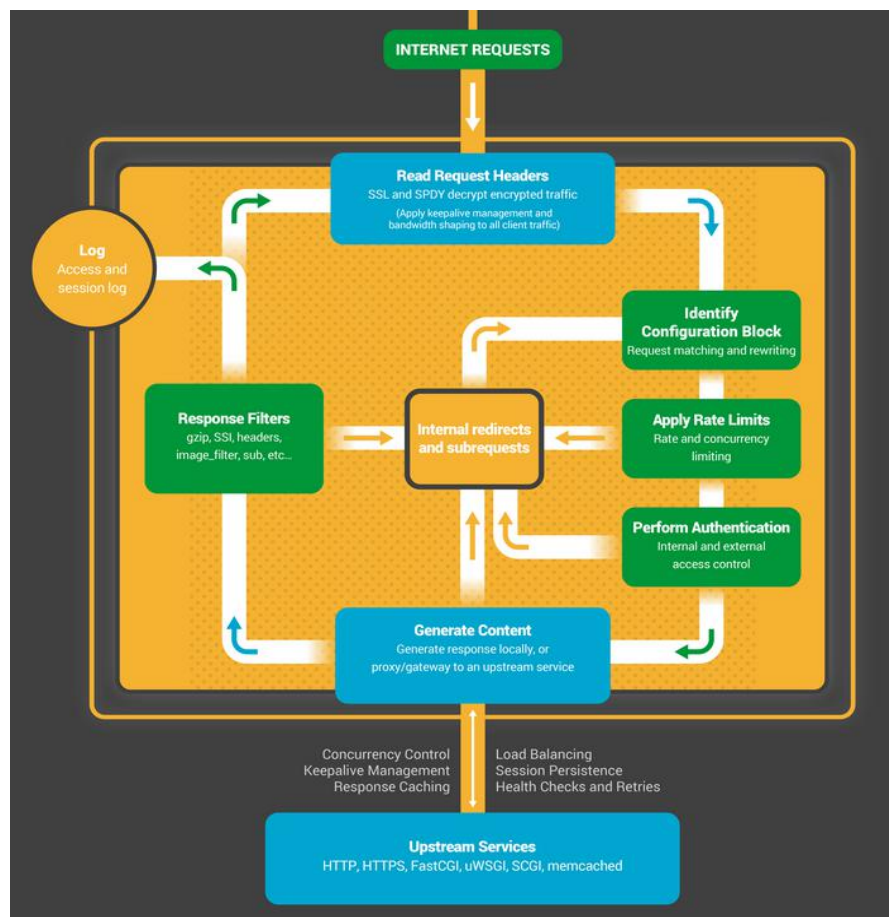
- Host = uri-host [":" port]
 - HTTP/1.1 规范要求，不传递 Host 头部则返回 400 错误响应码
 - 为防止陈旧的代理服务器，发向正向代理的请求 `request-target` 必须以 `absolute-form` 形式出现
 - request-line = method SP `request-target` SP HTTP-version CRLF
 - `absolute-form` = absolute-URI
 - absolute-URI = scheme ":" hier-part ["?" query]

规范与实现间是有差距的

- 关于 Host 头部: <https://tools.ietf.org/html/rfc7230#section-5.4>
 - A client MUST send a Host header field in all HTTP/1.1 request messages.
 - A server MUST respond with a 400 (Bad Request) status code to any HTTP/1.1 request message that **lacks** a Host header field and to any request message that contains **more than one** Host header field or a Host header field with an **invalid field-value**.

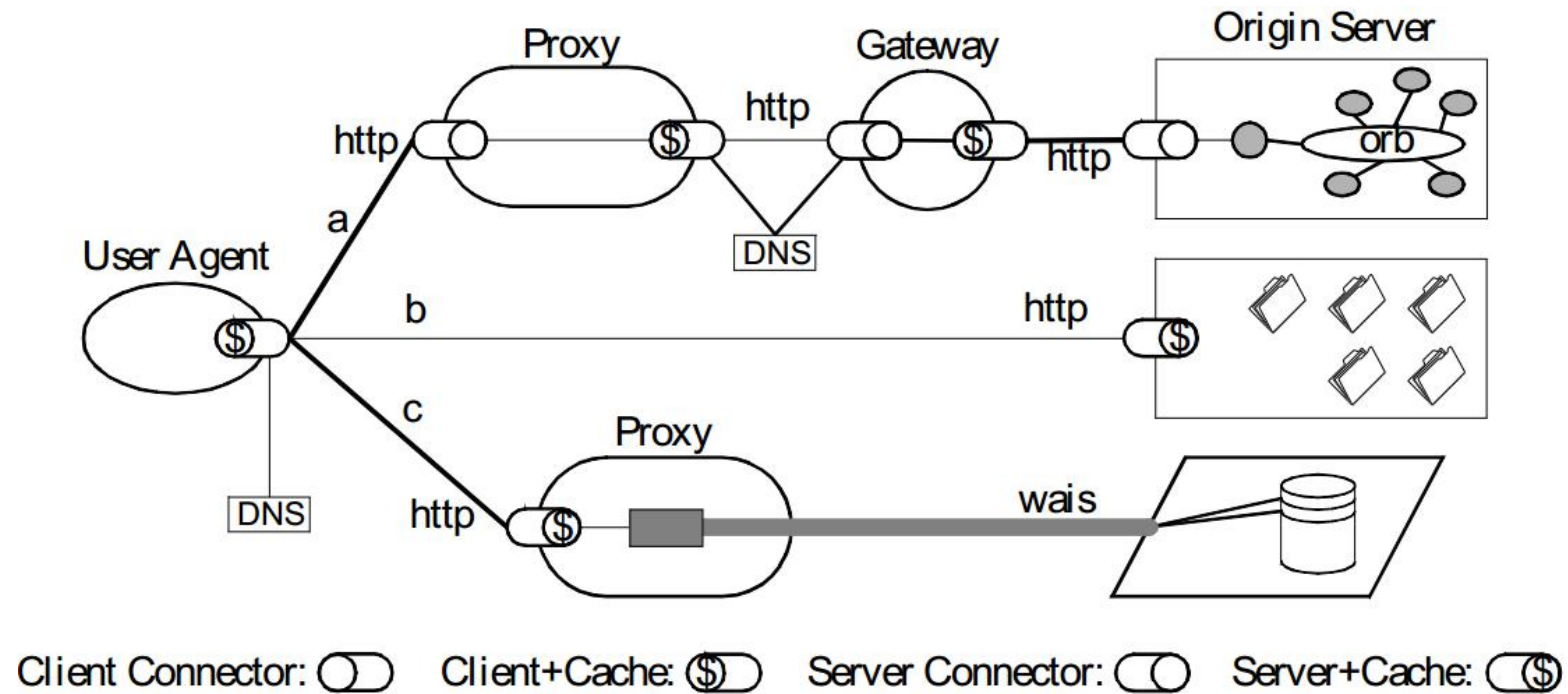
Host 头部与消息的路由

1. 建立 TCP 连接
 - 确定服务器的 IP 地址
2. 接收请求
3. 寻找虚拟主机
 - 匹配 Host 头部与域名
4. 寻找 URI 的处理代码
 - 匹配 URI
5. 执行处理请求的代码
 - 访问资源
6. 生成 HTTP 响应
 - 各中间件基于 PF 架构串行修改响应
7. 发送 HTTP 响应
8. 记录访问日志



第 12 课

客户端与源服务器间存在多个代理



消息的转发

- **Max-Forwards 头部**

- 限制 Proxy 代理服务器的最大转发次数，仅对 TRACE/OPTIONS 方法有效
- $\text{Max-Forwards} = 1 * \text{DIGIT}$

- **Via 头部**

- 指明经过的代理服务器名称及版本
- $\text{Via} = 1 \# (\text{received-protocol RWS received-by [RWS comment] })$
 - $\text{received-protocol} = [\text{protocol-name "/" }] \text{ protocol-version}$
 - $\text{received-by} = (\text{uri-host [":" port] }) / \text{pseudonym}$
 - $\text{pseudonym} = \text{token}$

- **Cache-Control:no-transform**

- 禁止代理服务器修改响应包体

问题：如何传递 IP 地址？

01 TCP 连接四元组 (src ip,src port, dst ip,dst port)

02 HTTP 头部 X-Forwarded-For 用于传递 IP

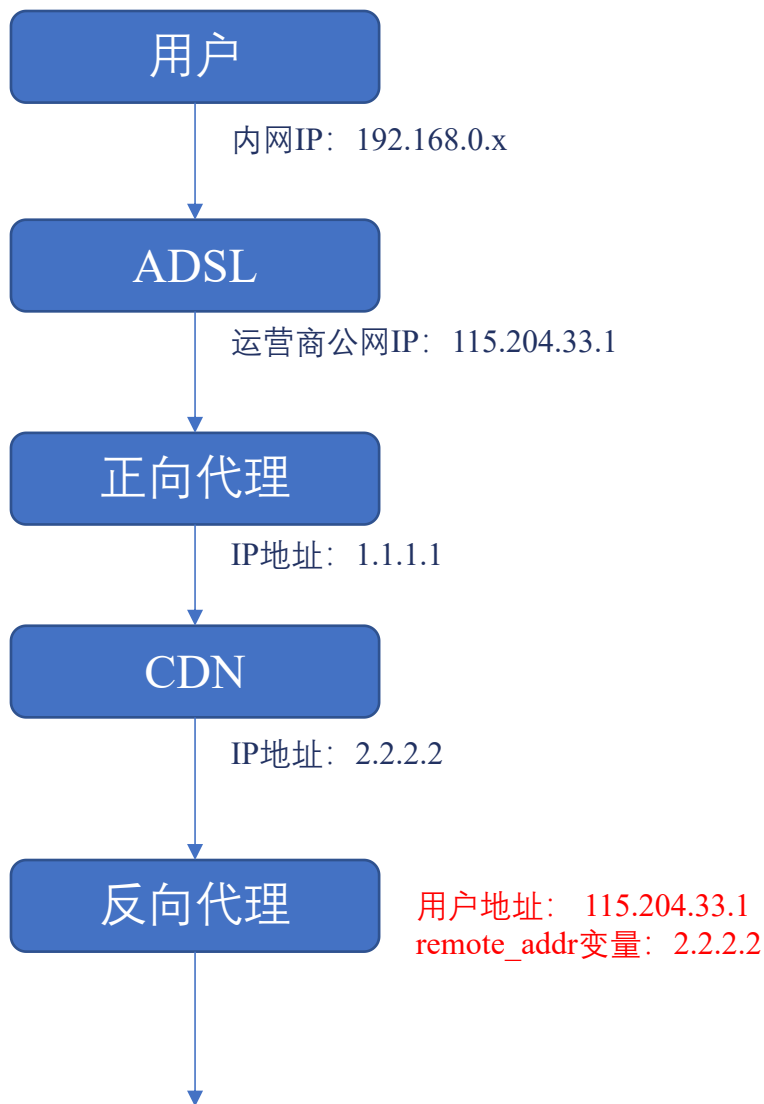
03 HTTP 头部 X-Real-IP 用于传递用户 IP

04 网络中存在许多反向代理

前提

X-Forwarded-For: 115.204.33.1
X-Real-IP: 115.204.33.1

X-Forwarded-For: 115.204.33.1, 1.1.1.1
X-Real-IP: 115.204.33.1



第 13 课

请求的上下文: User-Agent

指明客户端的类型信息，服务器可以据此对资源的表述做抉择

- User-Agent = product *(RWS (product / comment))
 - product = token ["/" product-version]
 - RWS = 1*(SP / HTAB)
- 例如:
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:66.0) Gecko/20100101 Firefox/66.0
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/73.0.3683.86 Safari/537.36

请求的上下文: Referer

浏览器对来自某一页面的请求自动添加的头部

- Referer = absolute-URI / partial-URI
- 例如:
 - Referer: <https://developer.mozilla.org/zh-CN/docs/Web/HTTP/Headers/User-Agent>
- Referer 不会被添加的场景
 - 来源页面采用的协议为表示本地文件的 "file" 或者 "data" URI
 - 当前请求页面采用的是 http 协议，而来源页面采用的是 https 协议
- 服务器端常用于统计分析、缓存优化、防盗链等功能

请求的上下文: From

- 主要用于网络爬虫，告诉服务器如何通过邮件联系到爬虫的负责人
- From = mailbox
 - 例如: From: webmaster@example.org

响应的上下文: Server

- 指明服务器上所用软件的信息, 用于帮助客户端定位问题或者统计数据
- $\text{Server} = \text{product} * (\text{RWS} (\text{product} / \text{comment}))$
 - $\text{product} = \text{token} ["/" \text{ product-version}]$
- 例如:
 - Server: nginx
 - Server: openresty/1.13.6.2

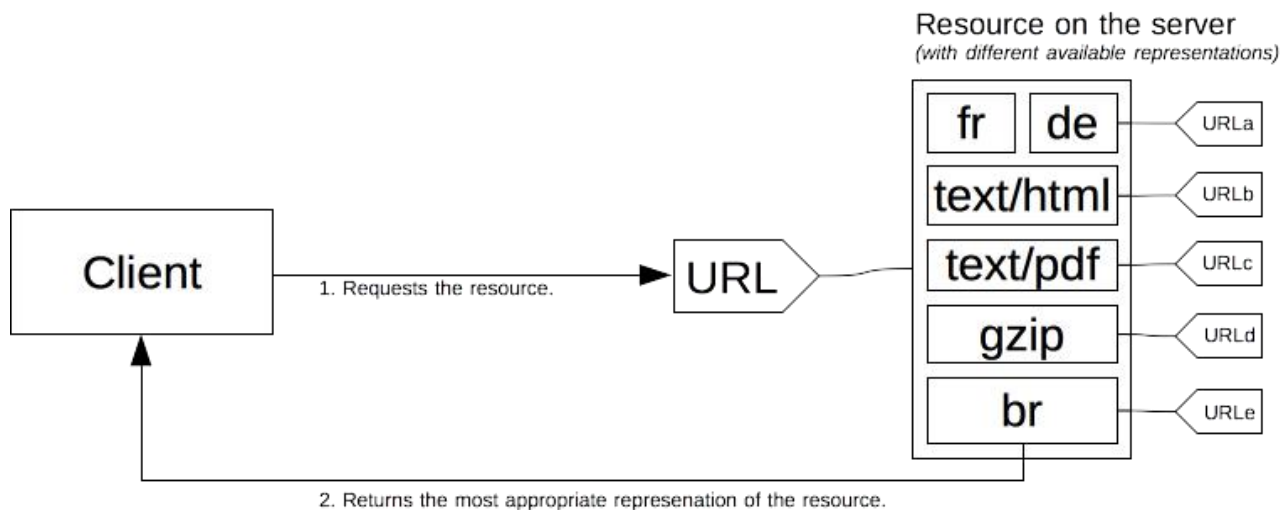
响应的上下文: Allow 与 Accept-Ranges

- Allow: 告诉客户端, 服务器上该 URI 对应的资源允许哪些方法的执行
 - Allow = #method
 - 例如:
 - Allow: GET, HEAD, PUT
- Accept-Ranges: 告诉客户端服务器上该资源是否允许 range 请求
 - Accept-Ranges = acceptable-ranges
 - 例如:
 - Accept-Ranges: bytes
 - Accept-Ranges: none

第 14 课

内容协商

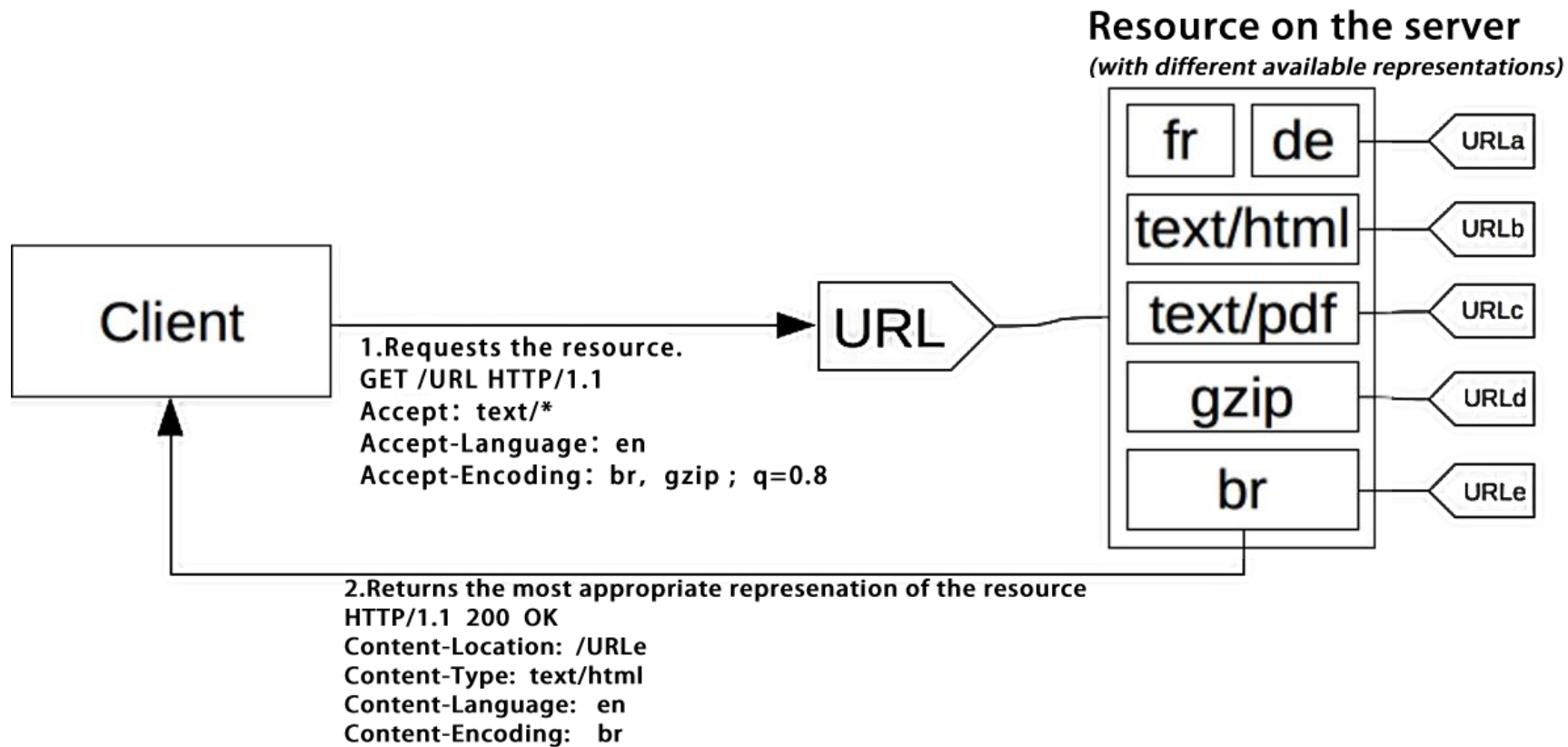
每个 URI 指向的资源可以是任何事物，可以有多种不同的表述，例如一份文档可以有不同语言的翻译、不同的媒体格式、可以针对不同的浏览器提供不同的压缩编码等。



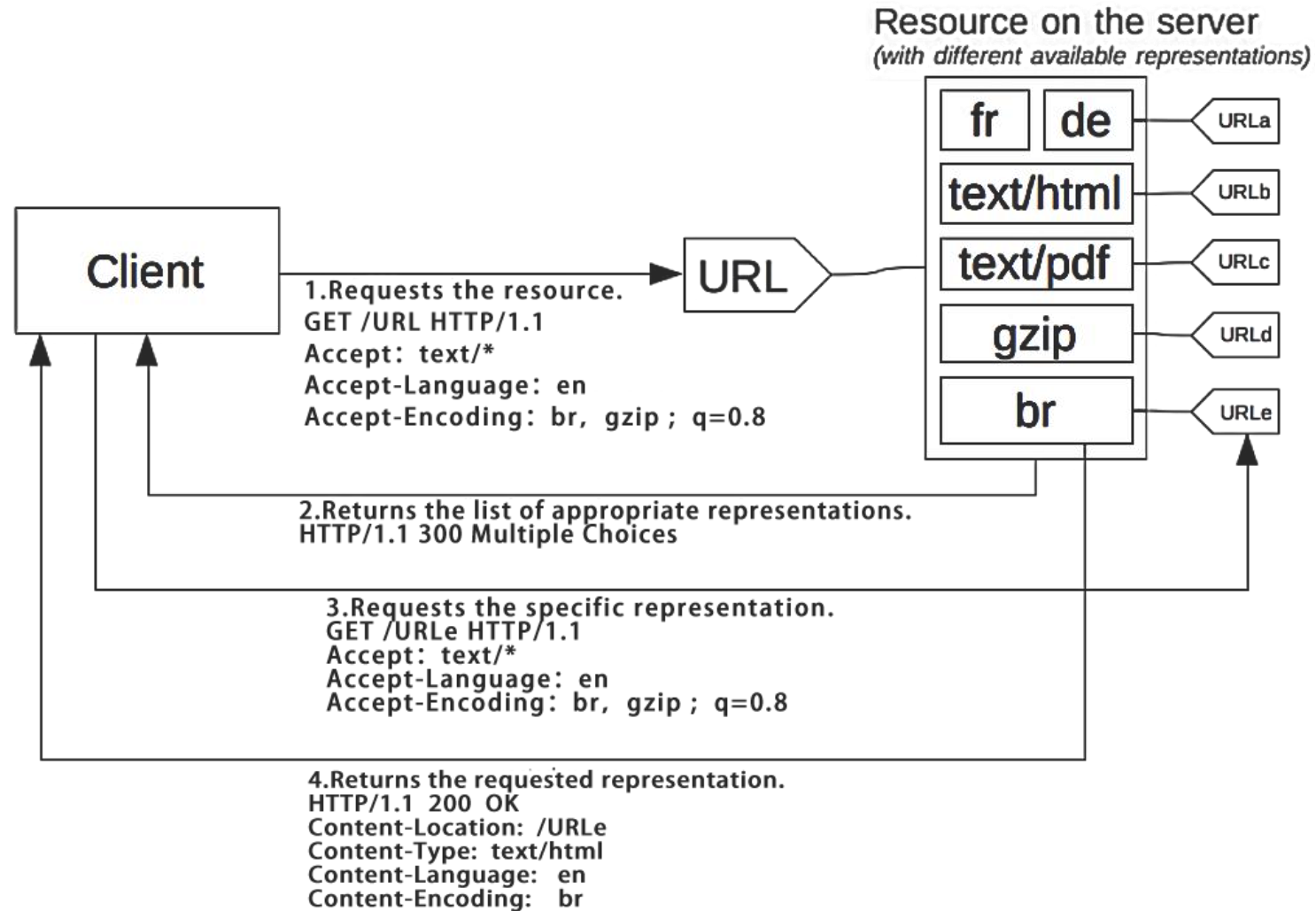
内容协商的两种方式

- Proactive 主动式内容协商：
 - 指由客户端先在请求头部中提出需要的表述形式，而服务器根据这些请求头部提供特定的 representation 表述
- Reactive 响应式内容协商：
 - 指服务器返回 300 Multiple Choices 或者 406 Not Acceptable，由客户端选择一种表述 URI 使用

Proactive 主动式内容协商



Reactive 响应式内容协商



常见的协商要素（一）

- 质量因子 q：内容的质量、可接受类型的优先级
- 媒体资源的 MIME 类型及质量因子
 - Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
 - Accept:text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3

常见的协商要素（二）

- 字符编码：由于 UTF-8 格式广为使用，Accept-Charset 已被废弃
 - Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
- 内容编码：主要指压缩算法
 - Accept-Encoding: gzip, deflate, br
- 表述语言
 - Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7
 - Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2

国际化与本地化

- internationalization (i18n, i 和 n 间有 18 个字符)
 - 指设计软件时, 在不同的国家、地区可以不做逻辑实现层面的修改便能够以不同的语言显示
- localization (l10n, l 和 n 间有 10 个字符)
 - 指内容协商时, 根据请求中的语言及区域信息, 选择特定的语言作为资源表述

资源表述的元数据头部

- 媒体类型、编码
 - content-type: text/html; charset=utf-8
- 内容编码
 - content-encoding: gzip
- 语言
 - Content-Language: de-DE, en-CA

第 15 课

HTTP 包体：承载的消息内容

- 请求或者响应都可以携带包体
 - HTTP-message = start-line *(header-field CRLF) CRLF [message-body]
 - message-body = *OCTET：二进制字节流
- 以下消息不能含有包体
 - HEAD 方法请求对应的响应
 - 1xx、204、304 对应的响应
 - CONNECT 方法对应的 2xx 响应

两种传输 HTTP 包体的方式（一）

- 发送 HTTP 消息时已能够确定包体的全部长度
 - 使用 Content-Length 头部明确指明包体长度
 - $\text{Content-Length} = 1 * \text{DIGIT}$
 - 用 10 进制（不是 16 进制）表示包体中的字节个数，且必须与实际传输的包体长度一致
- 优点：接收端处理更简单

两种传输 HTTP 包体的方式（二）

- 发送 HTTP 消息时不能确定包体的全部长度
 - 使用 Transfer-Encoding 头部指明使用 Chunk 传输方式
 - 含 Transfer-Encoding 头部后 Content-Length 头部应被忽略
- 优点
 - 基于长连接持续推送动态内容
 - 压缩体积较大的包体时，不必完全压缩完（计算出头部）再发送，可以边发送边压缩
 - 传递必须在包体传输完才能计算出的 Trailer 头部

不定长包体的 chunk 传输方式

- Transfer-Encoding 头部

- transfer-coding = "chunked" / "compress" / "deflate" / "gzip" / transfer-extension

- Chunked transfer encoding 分块传输编码: Transfer-Encoding: chunked

- chunked-body = *chunk

- last-chunk

- trailer-part

- CRLF

- chunk = chunk-size [chunk-ext] CRLF chunk-data CRLF

- chunk-size = 1*HEXDIG: 注意这里是 16 进制而不是10进制

- chunk-data = 1*OCTET

- last-chunk = 1*("0") [chunk-ext] CRLF

- trailer-part = *(header-field CRLF)

Trailer 头部的传输

- TE 头部：客户端在请求中声明是否接收 Trailer 头部
 - TE: trailers
- Trailer 头部：服务器告知接下来 chunk 包体后会传输哪些 Trailer 头部
 - Trailer: Date
- 以下头部不允许出现在 Trailer 的值中：
 - 用于信息分帧的首部 (例如 Transfer-Encoding 和 Content-Length)
 - 用于路由用途的首部 (例如 Host)
 - 请求修饰首部 (例如控制类和条件类的，如 Cache-Control, Max-Forwards, 或者 TE)
 - 身份验证首部 (例如 Authorization 或者 Set-Cookie)
 - Content-Encoding, Content-Type, Content-Range, 以及 Trailer 自身

MIME

- MIME (Multipurpose Internet Mail Extensions)
- content := "Content-Type" ":" type "/" subtype *("; " parameter)
 - type := discrete-type / composite-type
 - discrete-type := "text" / "image" / "audio" / "video" / "application" / extension-token
 - composite-type := "message" / "multipart" / extension-token
 - extension-token := ietf-token / x-token
 - subtype := extension-token / iana-token
 - parameter := attribute "=" value
- 大小写不敏感，但通常是小写
- 例如： Content-type: text/plain; charset="us-ascii "
- <https://www.iana.org/assignments/media-types/media-types.xhtml>

Content-Disposition 头部(RFC6266)

- **disposition-type = "inline" | "attachment" | disp-ext-type**
 - inline: 指定包体是以 inline 内联的方式, 作为页面的一部分展示
 - attachment: 指定浏览器将包体以附件的方式下载
 - 例如: Content-Disposition: attachment
 - 例如: Content-Disposition: attachment; filename="filename.jpg"
- 在 multipart/form-data 类型应答中, 可以用于子消息体部分
 - 如 Content-Disposition: form-data; name="fieldName";
filename="filename.jpg"

第 16 课

HTML FORM 表单

- HTML: HyperText Markup Language, 结构化的标记语言 (非编程语言)
 - 浏览器可以将 HTML 文件渲染为可视化网页
- FORM 表单: HTML 中的元素, 提供了交互控制元件用来向服务器通过 HTTP 协议提交信息, 常见控件有:
 - Text Input Controls: 文本输入控件
 - Checkboxes Controls: 复选框控件
 - Radio Box Controls : 单选按钮控件
 - Select Box Controls: 下拉列表控件
 - File Select boxes: 选取文件控件
 - Clickable Buttons: 可点击的按钮控件
 - **Submit** and Reset Button: 提交或者重置按钮控件

HTML FORM 表单提交请求时的关键属性

- action: 提交时发起 HTTP 请求的 URI
- method: 提交时发起 HTTP 请求的 http 方法
 - GET: 通过 URI, 将表单数据以 **URI 参数**的方式提交
 - POST: 将表单数据放在请求包体中提交
- enctype: 在 POST 方法下, 对表单内容在请求包体中的编码方式
 - **application/x-www-form-urlencoded**
 - 数据被编码成以 '&' 分隔的键-值对, 同时以 '=' 分隔键和值, 字符以 URL 编码方式编码
 - multipart/form-data
 - boundary 分隔符
 - 每部分表述皆有HTTP头部描述子包体, 例如 Content-Type
 - last boundary 结尾

multipart(RFC1521): 一个包体中多个资源表述

- Content-type 头部指明这是一个多表述包体

- Content-type: multipart/form-data;

boundary=----WebKitFormBoundaryRRJKeWfHPGrS4LKe

- Boundary 分隔符的格式

- boundary := 0*69<bchars> bcharsnospace

- bchars := bcharsnospace / " "

- bcharsnospace := DIGIT / ALPHA / "'" / "(" / ")" / "+" / "_" / "," / "-" / "." / "/" / ":" / "=" / "?"

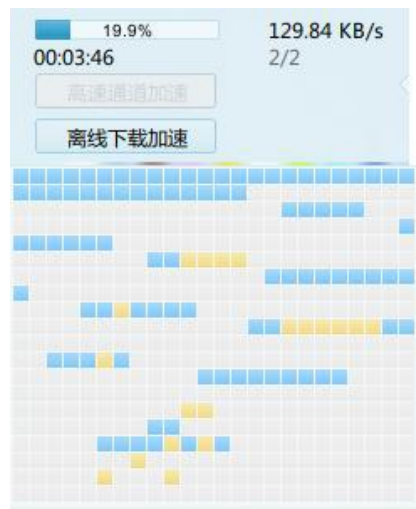
Multipart 包体格式(RFC822)

- multipart-body = preamble 1*encapsulation close-delimiter epilogue
 - *preamble := discard-text*
 - *epilogue := discard-text*
 - *discard-text := *(*text CRLF)*
- 每部分包体格式: encapsulation = delimiter body-part CRLF
 - delimiter = "--" **boundary** CRLF
 - body-part = fields *(CRLF ***text**)
 - field = field-name ":" [field-value] CRLF
 - content-disposition: form-data; **name**="xxxxx"
 - content-type 头部指明该部分包体的类型
- close-delimiter = "--" **boundary** "--" CRLF

第 17 课

多线程、断点续传、随机点播等场景的步骤

1. 客户端明确任务：从哪开始下载
 - 本地是否已有部分文件
 - 文件已下载部分在服务器端发生改变？
 - 使用几个线程并发下载
2. 下载文件的指定部分内容
3. 下载完毕后拼装成统一的文件



HTTP Range规范(RFC7233)

- 允许服务器基于客户端的请求只发送响应包体的一部分给到客户端，而客户端自动将多个片断的包体组合成完整的体积更大的包体
 - 支持断点续传
 - 支持多线程下载
 - 支持视频播放器实时拖动
- 服务器通过 Accept-Range 头部表示是否支持 Range 请求
 - Accept-Ranges = acceptable-ranges
 - 例如：
 - Accept-Ranges: bytes: 支持
 - Accept-Ranges: none: 不支持

Range 请求范围的单位

基于字节，设包体总长度为 10000

- 第 1 个 500 字节: bytes=0-499
- 第 2 个 500 字节
 - bytes=500-999
 - bytes=500-600,601-999
 - bytes=500-700,601-999
- 最后 1 个 500 字节:
 - bytes=-500
 - bytes=9500-
- 仅要第 1 个和最后 1 个字节: bytes=0-0,-1

通过Range头部传递请求范围，如：Range: bytes=0-499

Range 条件请求

- 如果客户端已经得到了 Range 响应的一部分，并想在这部分响应未过期的情况下，获取其他部分的响应
 - 常与 If-Unmodified-Since 或者 If-Match 头部共同使用
- If-Range = entity-tag / HTTP-date
 - 可以使用 Etag 或者 Last-Modified

服务器响应 (一)

206 Partial Content

- Content-Range 头部: 显示当前片断包体在完整包体中的位置
- Content-Range = byte-content-range / other-content-range
 - byte-content-range = bytes-unit SP (byte-range-resp / unsatisfied-range)
 - byte-range-resp = `byte-range` "/" (`complete-length` / "*")
 - `complete-length` = 1*DIGIT
 - 完整资源的大小, 如果未知则用 * 号替代
 - `byte-range` = first-byte-pos "-" last-byte-pos
- 例如:
 - Content-Range: bytes `42-1233/1234`
 - Content-Range: bytes `42-1233/*`

服务器响应 (二)

416 Range Not Satisfiable

- 请求范围不满足实际资源的大小，其中 Content-Range 中的 complete-length 显示完整响应的长度，例如：
 - Content-Range: bytes */1234

200 OK

- 服务器不支持 Range 请求时，则以 200 返回完整的响应包体

多重范围与 multipart

- 请求：
 - Range: bytes=0-50, 100-150
- 响应：
 - Content-Type: multipart/byteranges; boundary=...

第 18 课

Cookie 是什么?

RFC6265, HTTP State Management Mechanism

保存在客户端、由浏览器维护、表示应用状态的 HTTP 头部

- 存放在内存或者磁盘中
- 服务器端生成 Cookie 在响应中通过 Set-Cookie 头部告知客户端（允许多个 Set-Cookie 头部传递多个值）
- 客户端得到 Cookie 后，后续请求都会自动将 Cookie 头部携带至请求中



Cookie 与 Set-Cookie头部的定义

- Cookie 头部中可以存放多个 name/value 名值对
 - cookie-header = "Cookie:" OWS cookie-string OWS
 - cookie-string = **cookie-pair** *(";" SP **cookie-pair**)
 - cookie-pair = **cookie-name** "=" **cookie-value**
- Set-Cookie 头部一次只能传递 1 个 name/value 名值对, 响应中可以含多个头部
 - set-cookie-header = "Set-Cookie:" SP set-cookie-string
 - set-cookie-string = **cookie-pair** *(";" SP **cookie-av**)
 - cookie-pair = **cookie-name** "=" **cookie-value**
 - **cookie-av**: 描述 **cookie-pair** 的可选属性

Set-Cookie 中描述 cookie-pair 的属性

cookie-av = expires-av / max-age-av / domain-av / path-av / secure-av / httponly-av / extension-av

- expires-av = "Expires=" sane-cookie-date
 - cookie 到日期 sane-cookie-date 后失效
- max-age-av = "Max-Age=" non-zero-digit *DIGIT
 - cookie 经过 *DIGIT 秒后失效。max-age 优先级高于 expires
- domain-av = "Domain=" domain-value
 - 指定 cookie 可用于哪些域名，默认可以访问当前域名
- path-av = "Path=" path-value
 - 指定 Path 路径下才能使用 cookie
- secure-av = "Secure"
 - 只有使用 TLS/SSL 协议 (https) 时才能使用 cookie
- httponly-av = "HttpOnly"
 - 不能使用 JavaScript (Document.cookie、XMLHttpRequest、Request APIs) 访问到 cookie

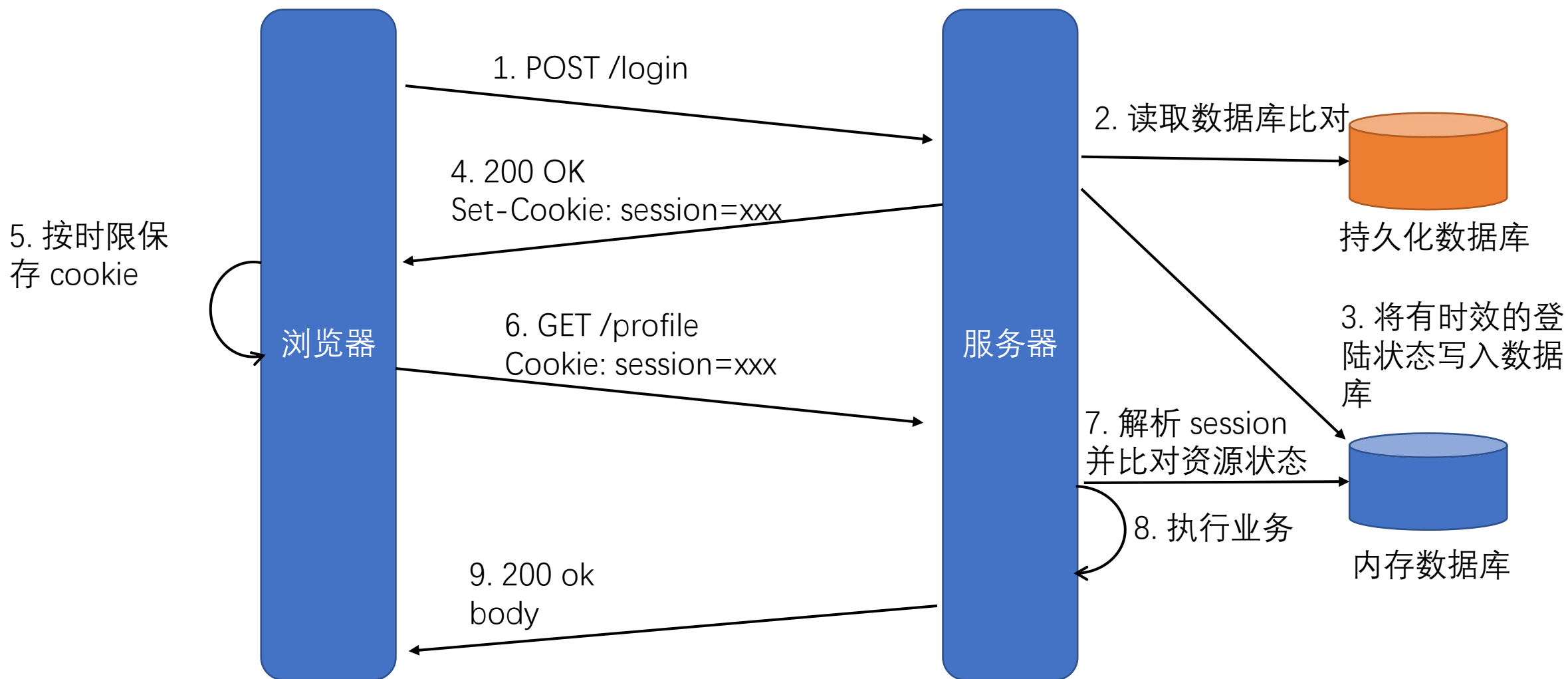
Cookie 使用的限制

- RFC 规范对浏览器使用 Cookie 的要求
 - 每条 Cookie 的长度（包括 name、value 以及描述的属性等总长度）至于要达到 4KB
 - 每个域名下至少支持 50 个 Cookie
 - 至少要支持 3000 个 Cookie
- 代理服务器传递 Cookie 时会有限制

Cookie 在协议设计上的问题

- Cookie 会被附加在每个 HTTP 请求中，所以无形中增加了流量
- 由于在 HTTP 请求中的 Cookie 是明文传递的，所以安全性成问题（除非用 HTTPS）
- Cookie 的大小不应超过 4KB，故对于复杂的存储需求来说是不够用的

登录场景下 Cookie 与 Session 的常见用法



无状态的 REST 架构 VS 状态管理

- **应用状态与资源状态**

- 应用状态：应由客户端管理，不应由服务器管理
 - 如浏览器目前在哪一页
 - REST 架构要求服务器不应保存应用状态
- 资源状态：应由服务器管理，不应由客户端管理
 - 如数据库中存放的数据状态，例如用户的登陆信息

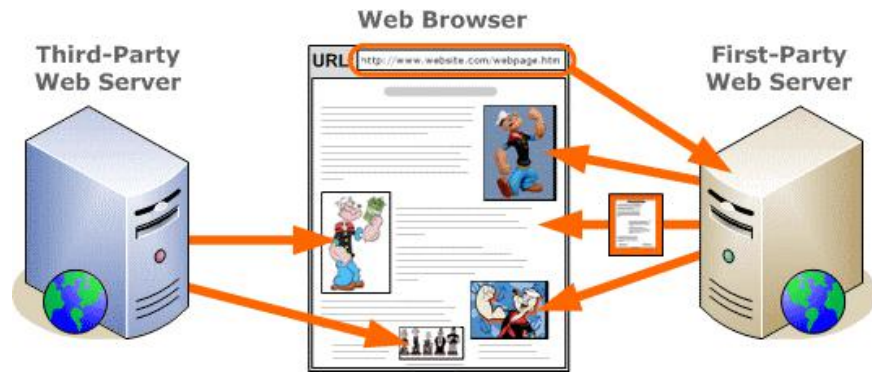
- **HTTP 请求的状态**

- 有状态的请求：服务器端保存请求的相关信息，每个请求可以使用以前保留的请求相关信息
 - 服务器 session 机制使服务器保存请求的相关信息
 - cookie 使请求可以携带查询信息，与 session 配合完成有状态的请求
- 无状态的请求：服务器能够处理的所有信息都来自当前请求所携带的信息
 - 服务器不会保存 session 信息
 - 请求可以通过 cookie 携带

第三方 Cookie

浏览器允许对于不安全域下的资源（如广告图片）响应中的 Set-Cookie 保存，并在后续访问该域时自动使用 Cookie

- 用户踪迹信息的搜集

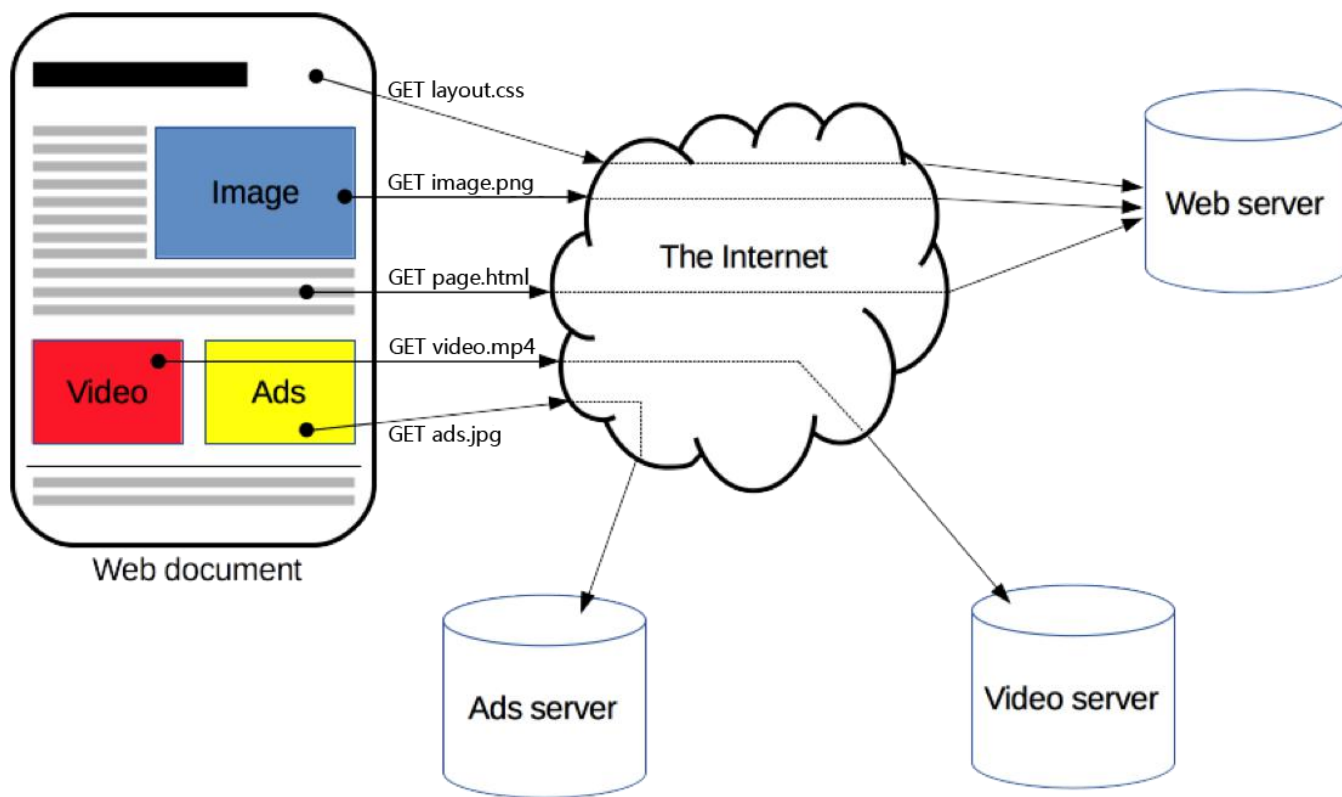


第 19 课

为什么需要同源策略？

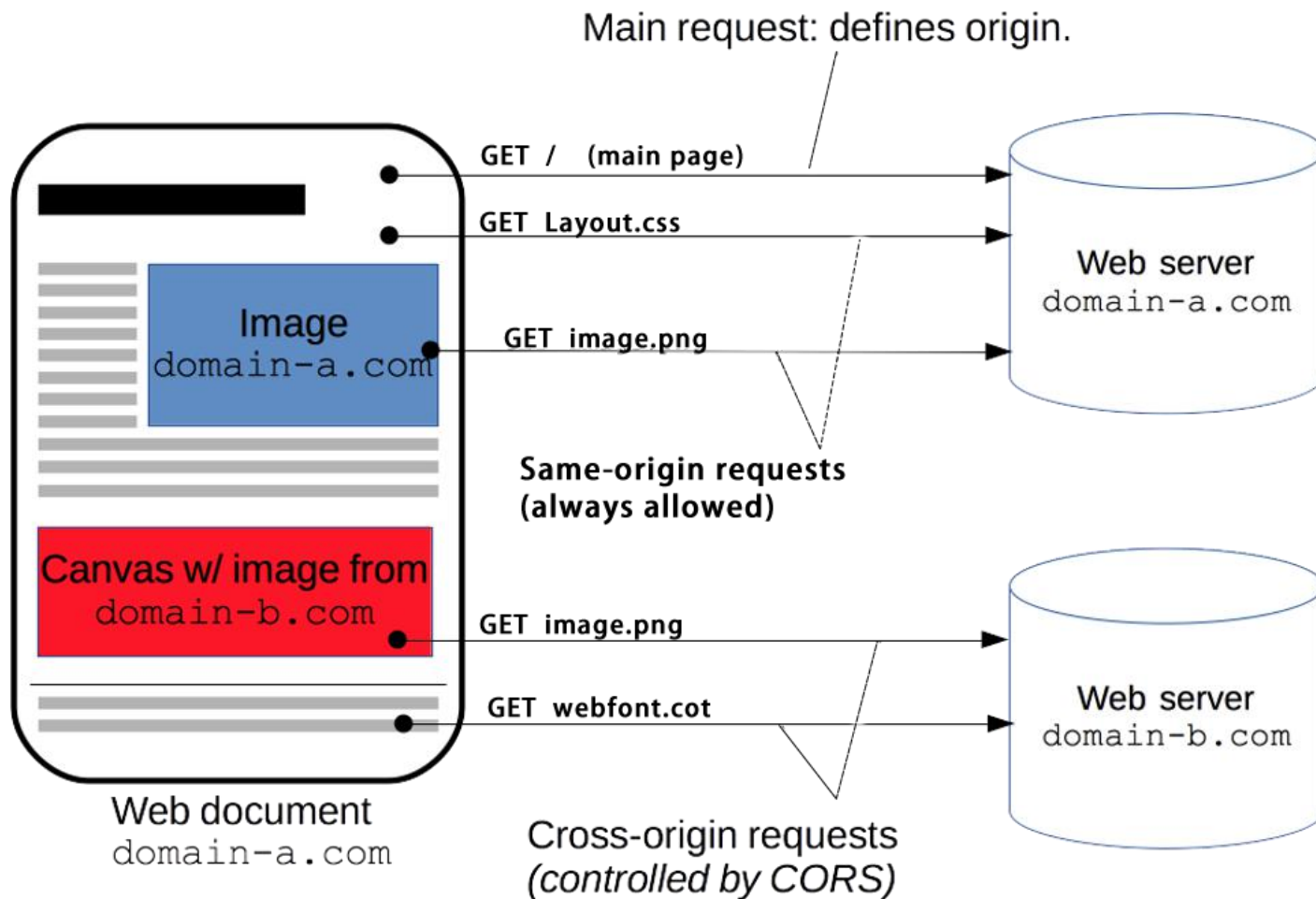
同一个浏览器发出的请求，未必都是用户自愿发出的请求

- 只有 page.html 是用户发出的，其他请求是浏览器自动发出的



为什么需要同源策略？

站点 domain-b.com 收到的来自同一浏览器的请求，可能来自于站点domain-a.com



没有同源策略下的 Cookie

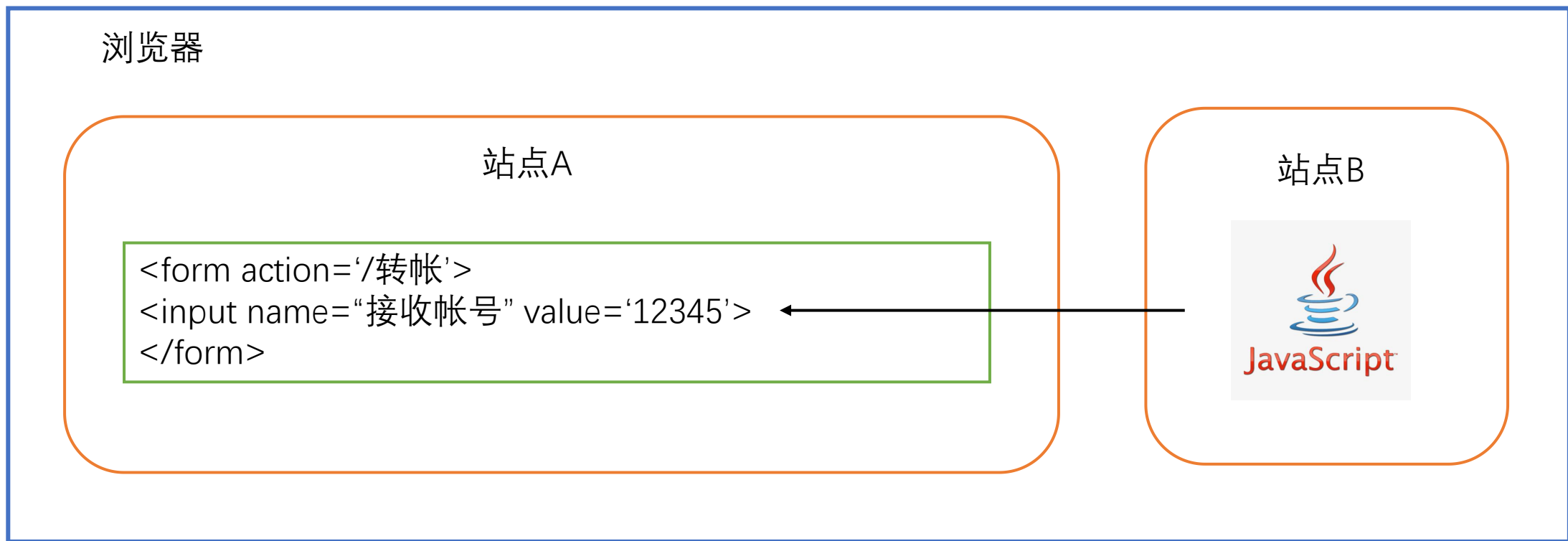
只能保证用户请求来自于同一浏览器，不能确保是用户自愿发出的

- 访问站点 A 后，站点 A 通过 Set-Cookie: Cookie 头部将 Cookie 返回给浏览器
- 浏览器会保存 Cookie，留待下次访问
- **站点 B 的脚本**访问站点 A 时，浏览器会自动将 Cookie: cookie 添加到请求的头部访问站点 A，提升用户体验
- 站点 A 的鉴权策略：取出 Cookie 值与数据库或者缓存中的 token 验证，通过后将数据赋予请求继续处理。



如果没有同源策略

站点 B 的脚本就可以随意修改站点 A 的 DOM 结构



浏览器的同源策略

限制了从同一个源加载的文档或脚本如何与来自另一个源的资源进行交互

- 何谓同源？协议、主机、端口必须完全相同
- 例如：<http://store.company.com/dir/page.html> 检测以下是否同源

URL

`http://store.company.com/dir2/other.html`

`http://store.company.com/dir/inner/another.html`

`https://store.company.com/secure.html`

`http://store.company.com:81/dir/etc.html`

`http://news.company.com/dir/other.html`

结果 原因

成功 只有路径不同

成功 只有路径不同

失败 不同协议 (https 和 http)

失败 不同端口 (http:// 80是默认的)

失败 不同域名 (news 和 store)

安全性与可用性需要一个平衡点

可用性：HTML 的创作者决定跨域请求是否对本站点安全

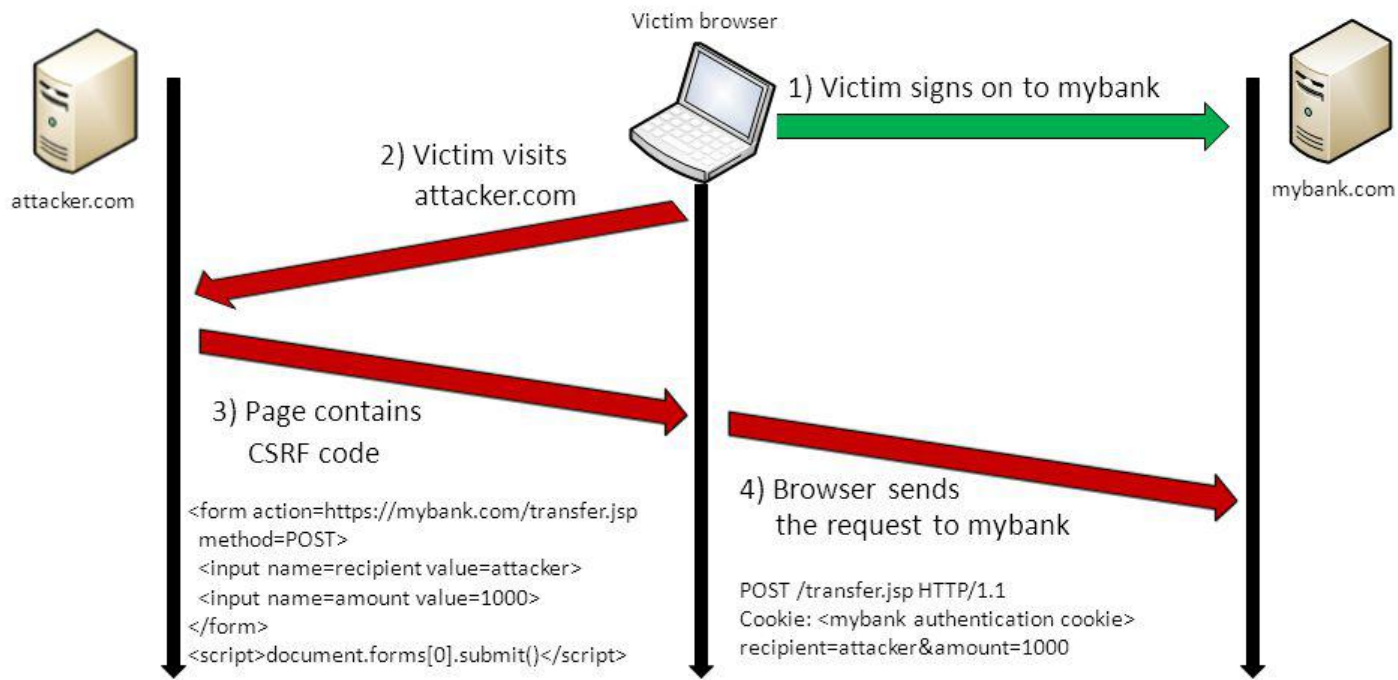
- `<script>` `<img>` `<iframe>` `<link>` `<video>` `<audio>` 带有 `src` 属性可以跨域访问
- 允许跨域写操作：例如表单提交或者重定向请求
 - CSRF 安全性问题

安全性：浏览器需要防止站点 A 的脚本向站点 B 发起危险动作

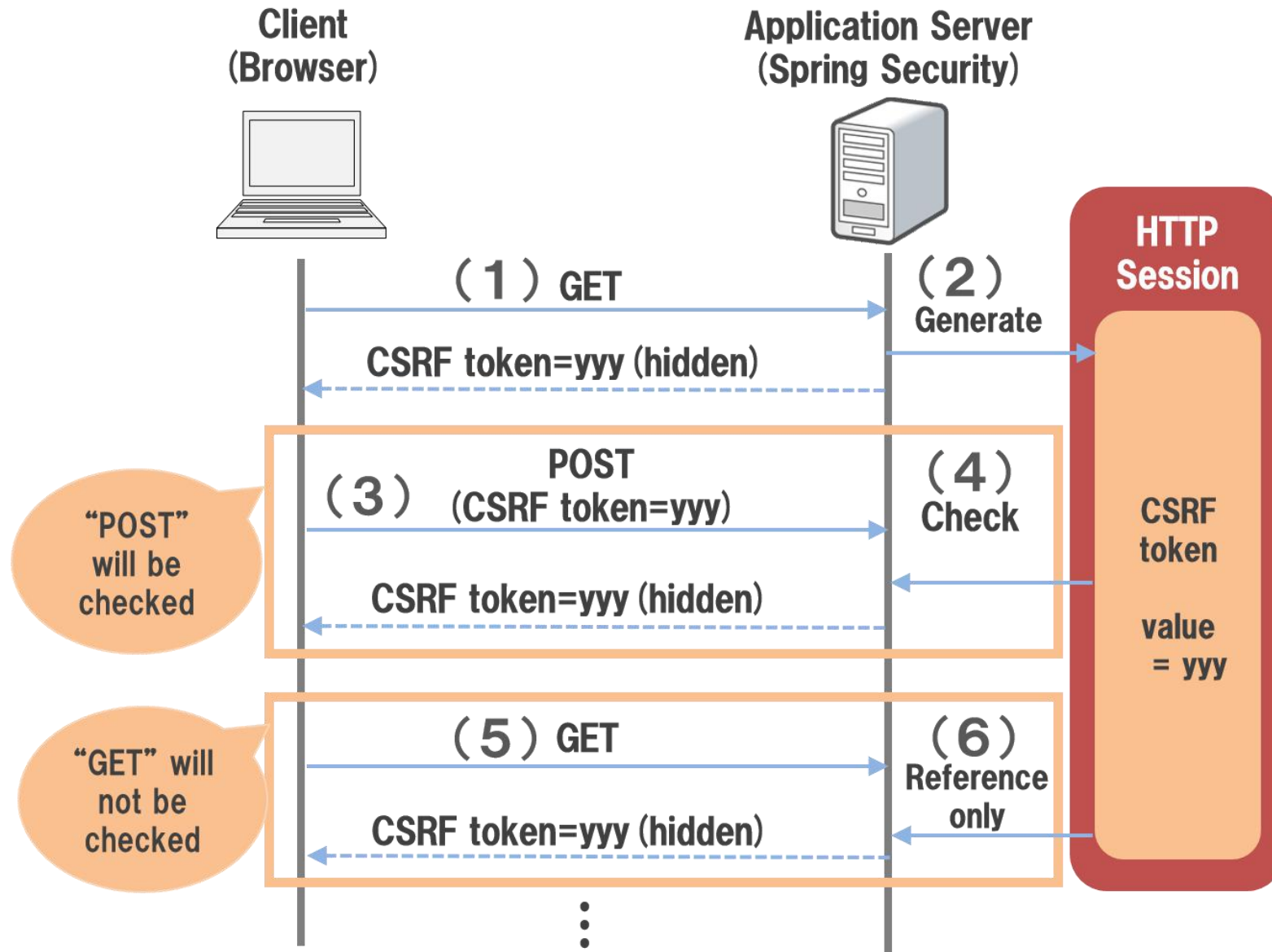
- Cookie、LocalStorage 和 IndexedDB 无法读取
- DOM 无法获得（防止跨域脚本篡改 DOM 结构）
- AJAX 请求不能发送

跨站请求伪造攻击

Cross-Site Request Forgery (CSRF)



CSRF 的一种防攻击方式

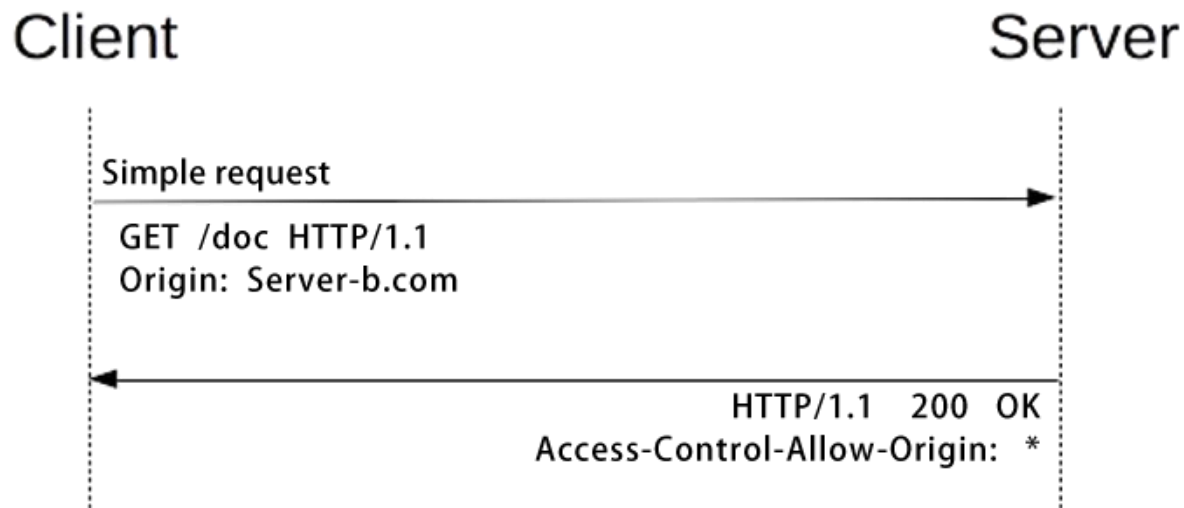


CORS: Cross-Origin Resource Sharing

- 浏览器同源策略下的跨域访问解决方案：
 - 如果站点 A 允许站点 B 的脚本访问其资源，必须在 HTTP 响应中显式的告知浏览器：站点 B 是被允许的
 - 访问站点 A 的请求，浏览器应告知该请求来自站点 B
 - 站点 A 的响应中，应明确哪些跨域请求是被允许的
- 策略 1：何为简单请求？
 - GET/HEAD/POST 方法之一
 - 仅能使用 CORS 安全的头部：Accept、Accept-Language、Content-Language、Content-Type
 - Content-Type 值只能是：text/plain、multipart/form-data、application/x-www-form-urlencoded 三者其中之一
- 策略 2：简单请求以外的其他请求
 - 访问资源前，需要先发起 preflight 预检请求（方法为 OPTIONS）询问何种请求是被允许的

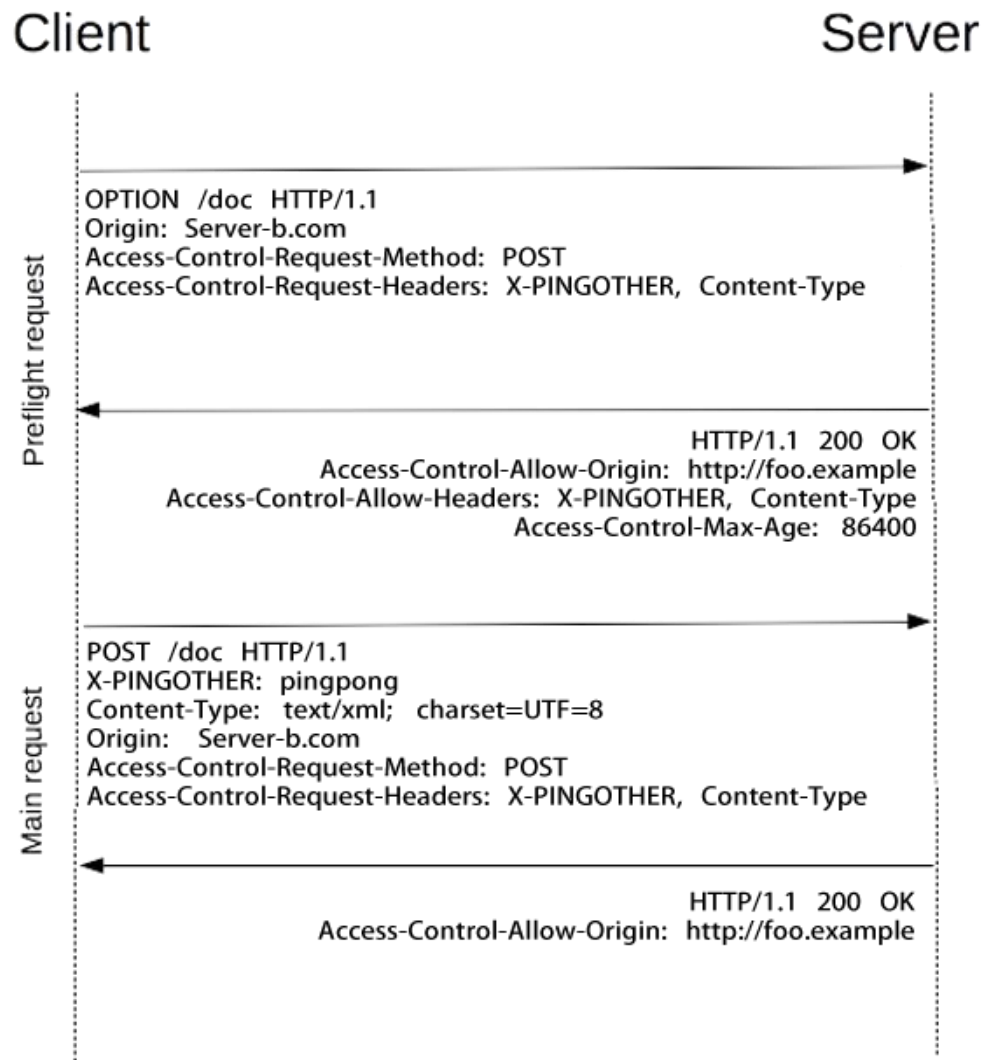
简单请求的跨域访问

- 请求中携带 Origin 头部告知来自哪个域
- 响应中携带 Access-Control-Allow-Origin 头部表示允许哪些域
- 浏览器放行



预检请求

- 预检请求头部
 - Access-Control-Request-Method
 - Access-Control-Request-Headers
- 预检请求响应
 - Access-Control-Allow-Methods
 - Access-Control-Allow-Headers
 - Access-Control-Max-Age



跨域访问资源：请求头部

请求头部

- **Origin (RFC6454)**：一个页面的资源可能来自于多个域名，在 AJAX 等子请求中标明来源于某个域名下的脚本，以通过服务器的安全校验
 - `origin` = "Origin:" OWS `origin-list-or-null` OWS
 - `origin-list-or-null` = `%x6E %x75 %x6C %x6C / origin-list`
 - `origin-list` = `serialized-origin *( SP serialized-origin )`
 - `serialized-origin` = `scheme "://" host [ ":" port ]`
- **Access-Control-Request-Method**
 - 在 preflight 预检请求 (OPTIONS) 中，告知服务器接下来的请求会使用哪些方法
- **Access-Control-Request-Headers**
 - 在 preflight 预检请求 (OPTIONS) 中，告知服务器接下来的请求会传递哪些头部

跨域访问资源：响应头部

响应头部

- Access-Control-Allow-Methods
 - 在 preflight 预检请求的响应中，告知客户端后续请求允许使用的方法
- Access-Control-Allow-Headers
 - 在 preflight 预检请求的响应中，告知客户端后续请求允许携带的头部
- Access-Control-Max-Age
 - 在 preflight 预检请求的响应中，告知客户端该响应的信息可以缓存多久
- Access-Control-Expose-Headers
 - 告知浏览器哪些响应头部可以供客户端使用，默认情况下只有 Cache-Control、Content-Language、Content-Type、Expires、Last-Modified、Pragma 可供使用
- Access-Control-Allow-Origin
 - 告知浏览器允许哪些域访问当前资源，*表示允许所有域。为避免缓存错乱，响应中需要携带 Vary: Origin
- Access-Control-Allow-Credentials
 - 告知浏览器是否可以将 Credentials 暴露给客户端使用，Credentials 包含 cookie、authorization 类头部、TLS证书等。

第 20 课

资源 URI 与资源表述 Representation

- 资源 R 可被定义为随时间变化的函数 $M_R(t)$
 - 静态资源：创建后任何时刻值都不变，例如指定版本号的库文件
 - 动态资源：其值随时间而频繁地变化，例如某新闻站点首页
- 优点
 - 提供了无需人为设定类型或者实现方式的情况下，同一资源多种不同来源的信息
 - 基于请求特性进行内容协商，使资源的渲染延迟绑定
 - 允许表述概念而不是具体的 Representation，故资源变化时不用修改所有链接

Precondition 条件请求

目的

- 由客户端携带条件判断信息，而服务器预执行条件验证过程成功后，再返回资源的表述

常见应用场景

- 使缓存的更新更有效率（如 304 响应码使服务器不用传递包体）
- 断点续传时对之前内容的验证
- 当多个客户端并行修改同一资源时，防止某一客户端的更新被错误丢弃

强验证器与弱验证器的概念

- 验证器 **validator**: 根据客户端请求中携带的相关头部, 以及服务器资源的信息, 执行两端的资源验证
 - 强验证器: 服务器上的资源表述只要有变动 (例如版本更新或者元数据更新), 那么以旧的验证头部访问一定会导致验证不过
 - 弱验证器: 服务器上资源变动时, 允许一定程度上仍然可以验证通过 (例如一小段时间内仍然允许缓存有效)

验证器响应头部（一）

- Etag 响应头部

- 定义:

- Etag = entity-tag

- entity-tag = [weak] opaque-tag

- weak = %x57.2F

- opaque-tag = DQUOTE *etagc DQUOTE

- etagc = %x21 / %x23-7E / obs-text

- 给出当前资源表述的标签

- 例如:

- 强验证器 ETag: "xyzzy"

- 弱验证器 ETag: W/"xyzzy"

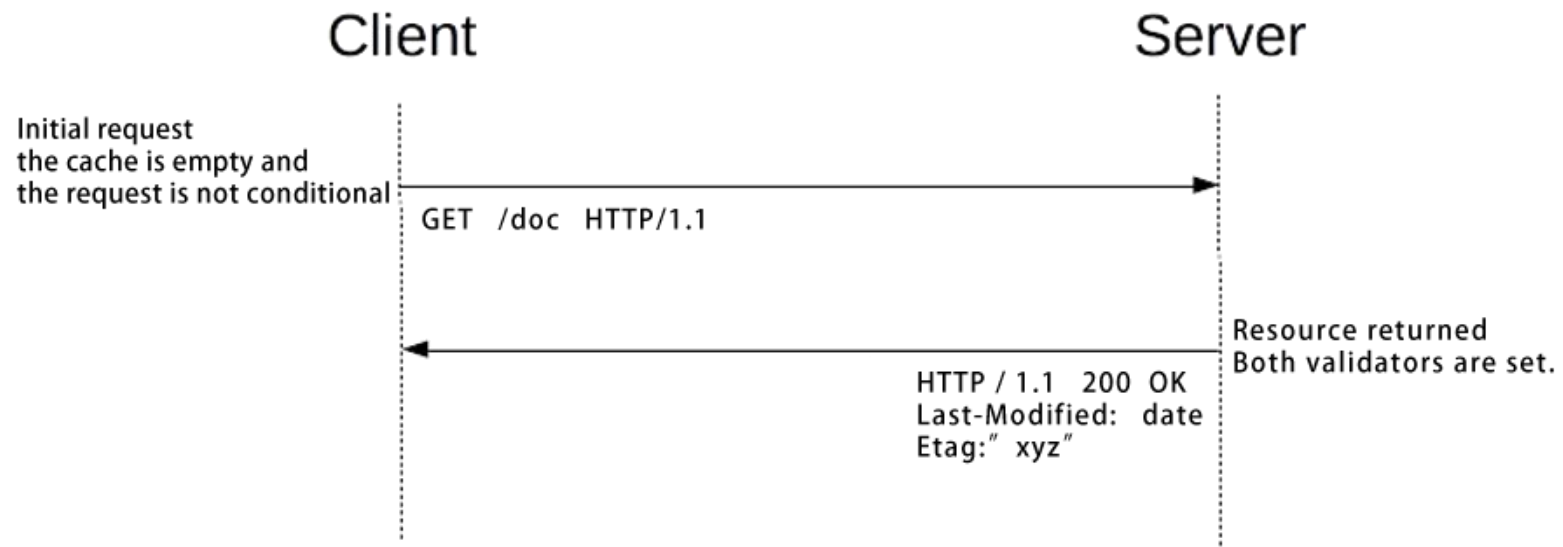
验证器响应头部（二）

- Last-Modified 响应头部
 - 定义：Last-Modified = HTTP-date
 - 表示对应资源表述的上次修改时间
 - 对比 Date 头部：Date = HTTP-date
 - 表示响应包体生成的时间
 - Last-Modified 不能晚于 Date 的值

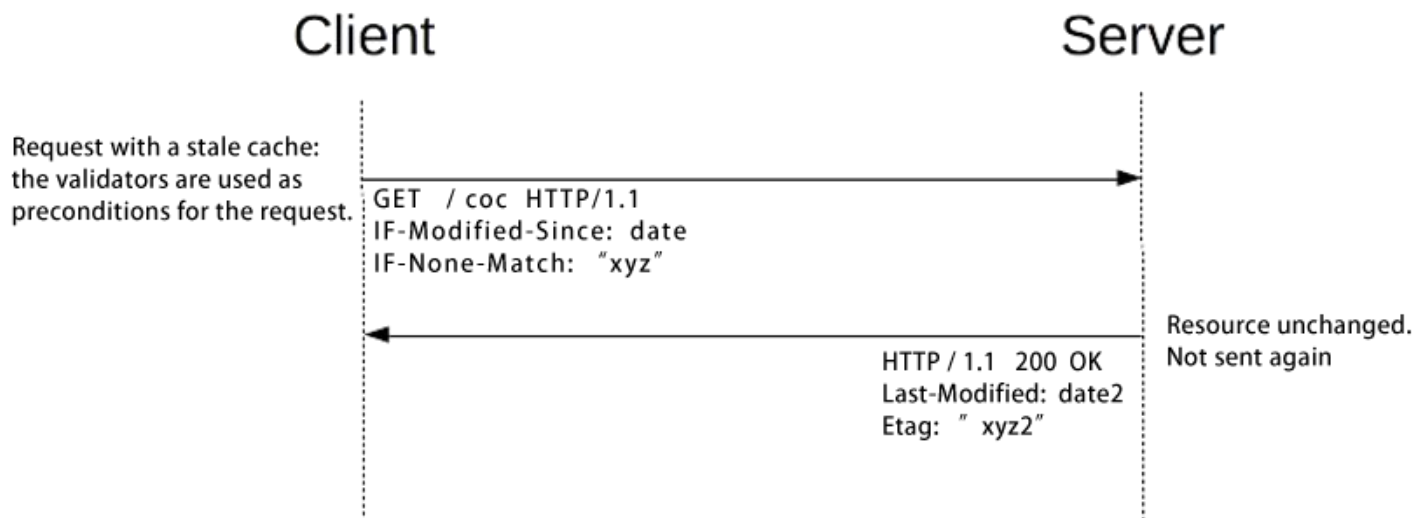
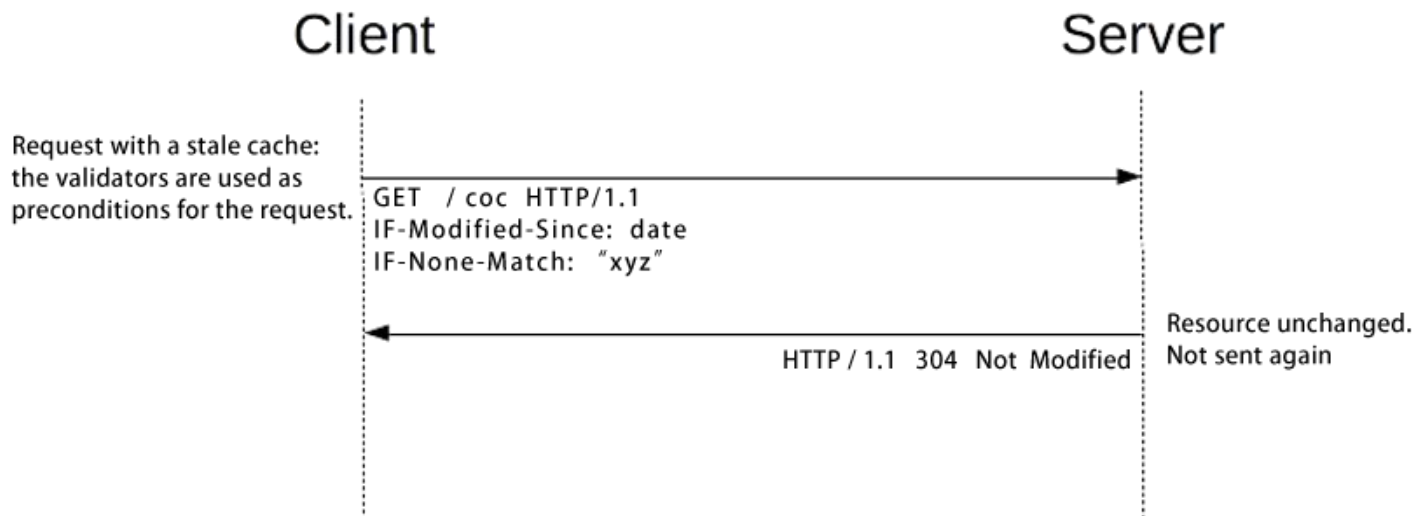
条件请求头部

- If-Match = "*" / 1#entity-tag
- If-None-Match = "*" / 1#entity-tag
- If-Modified-Since = HTTP-date
- If-Unmodified-Since = HTTP-date
- If-Range = entity-tag / HTTP-date

缓存更新（一）：首次缓存

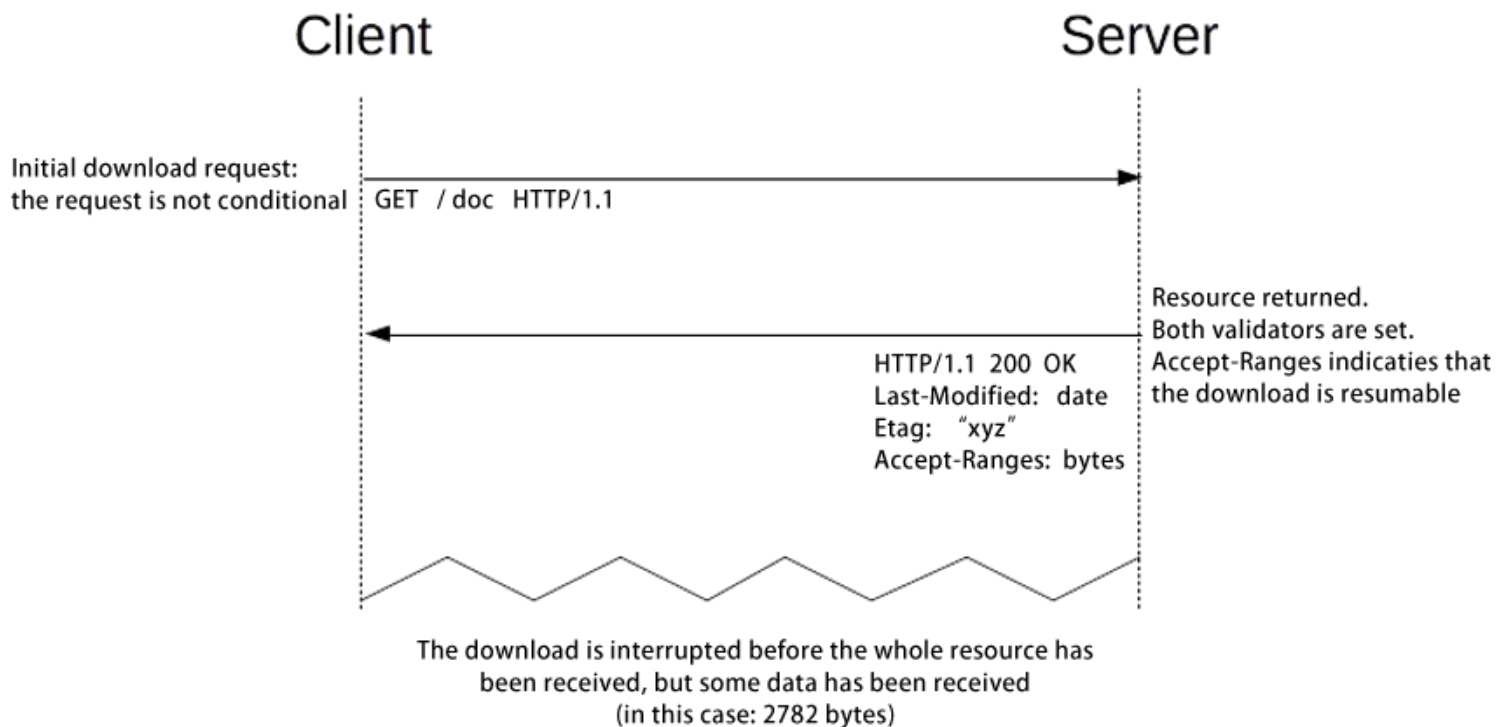


缓存更新（二）：基于过期缓存发起条件请求



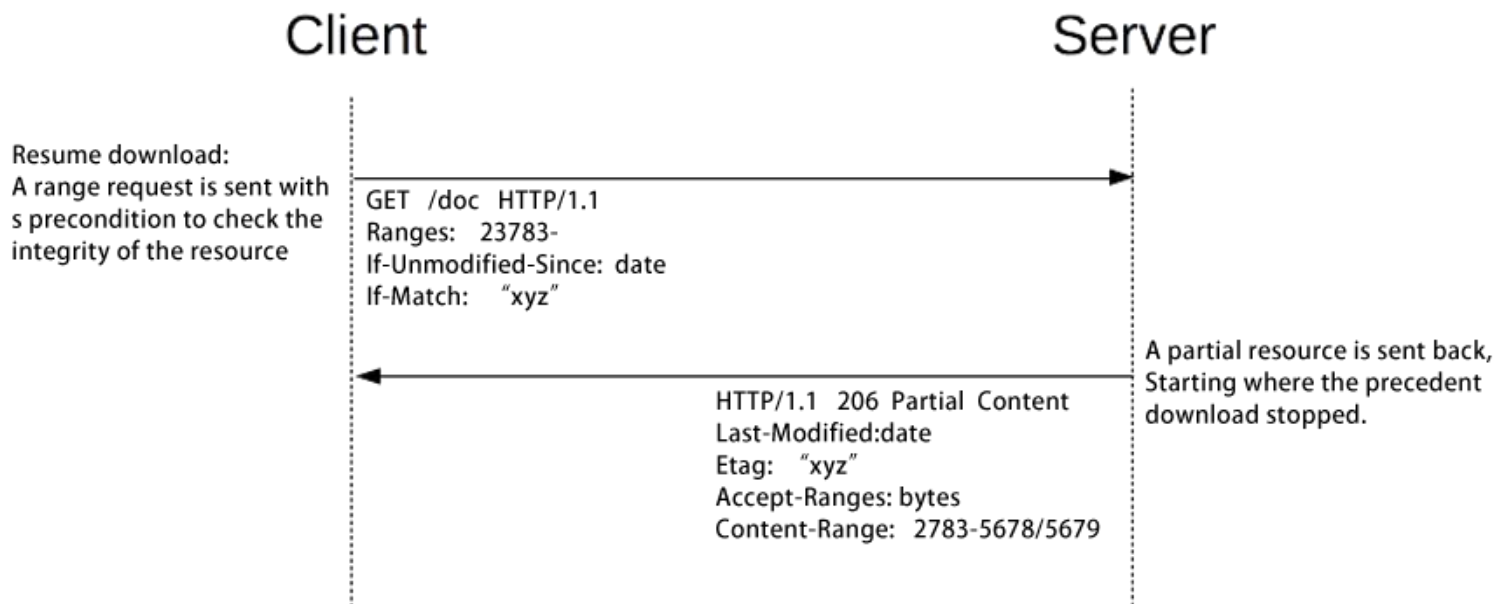
增量更新（一）

当服务器支持 Range 服务时，连接意外中断时已接收到部分数据



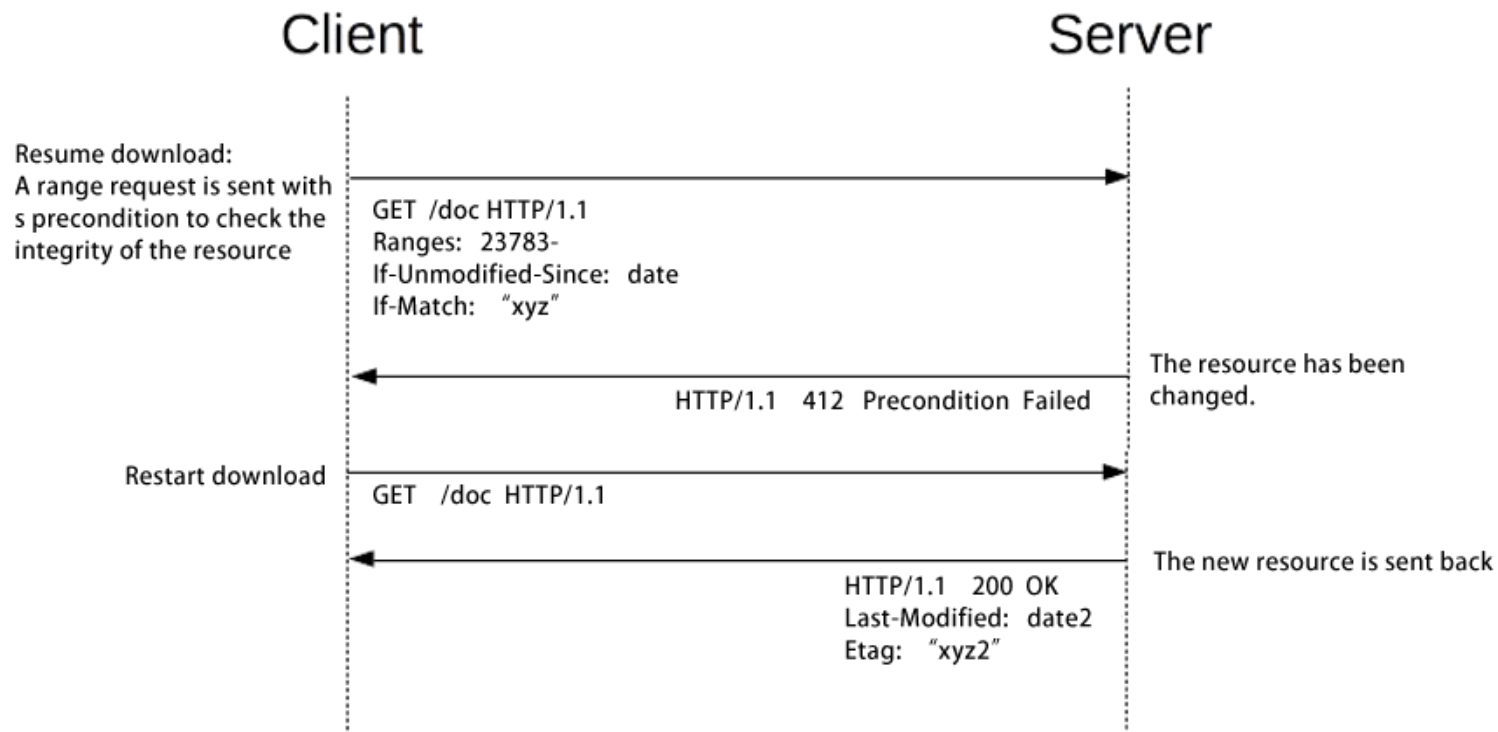
增量更新（二）

通过 Range 请求下载其他包体时，加入验证器防止两次下载间资源已发生了变更



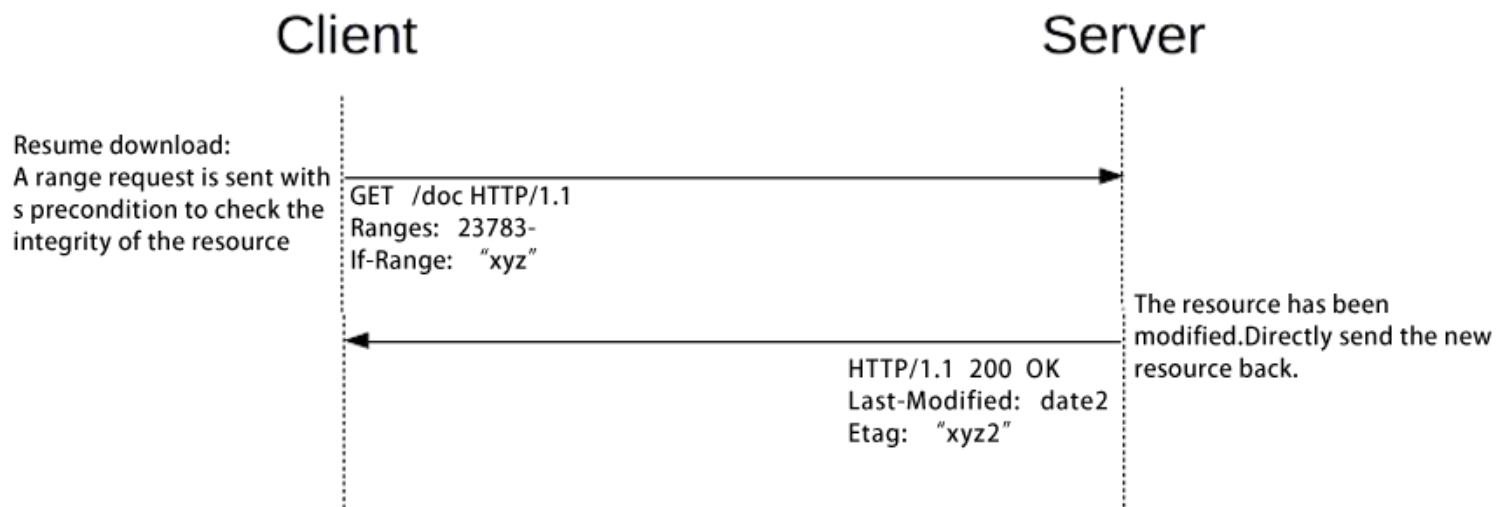
增量更新（三）

如果两次下载操作中，资源已经变量，则服务器用 412 通知客户端，而客户端重新下载完整包体



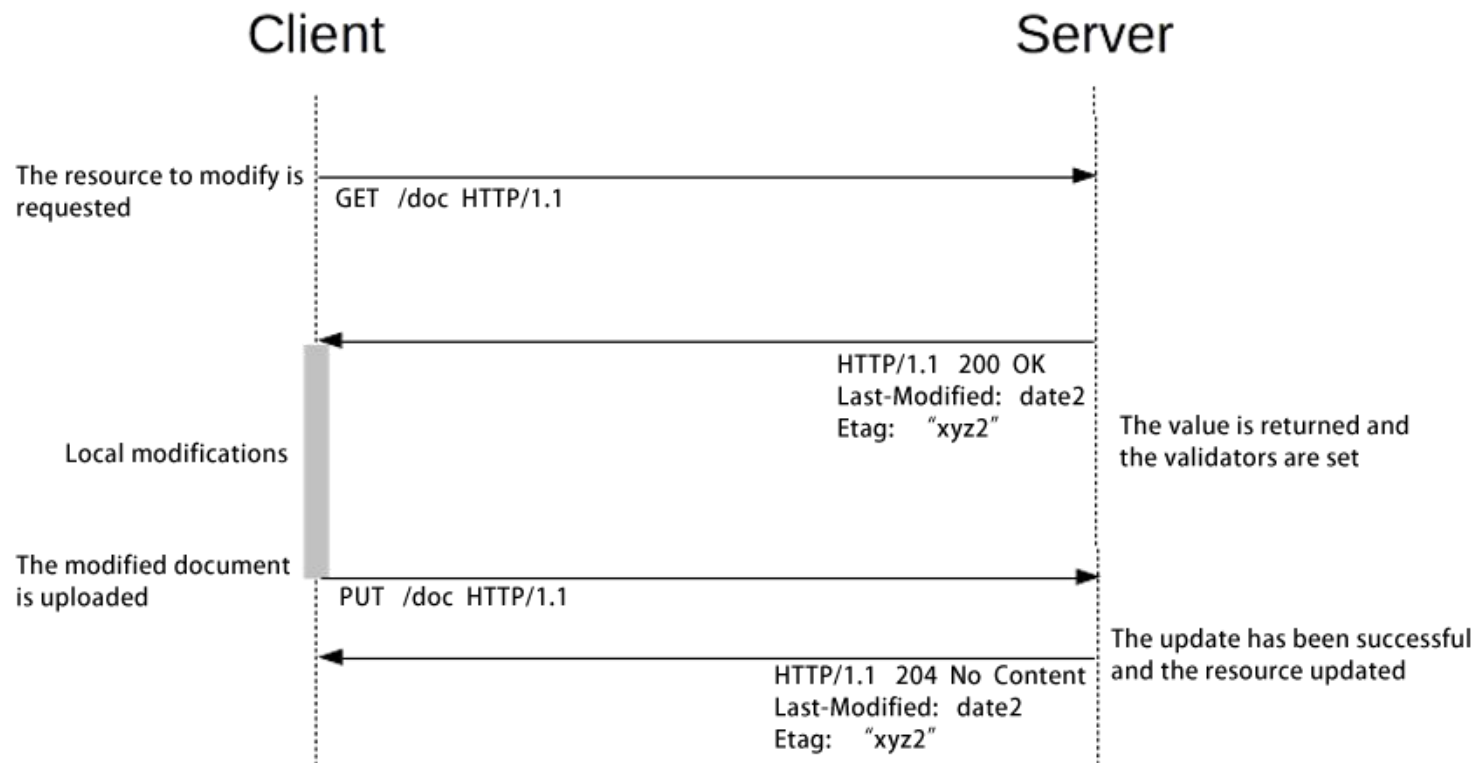
增量更新（四）

通过 If-Range 头部可以避免 2 次请求交互带来的损耗



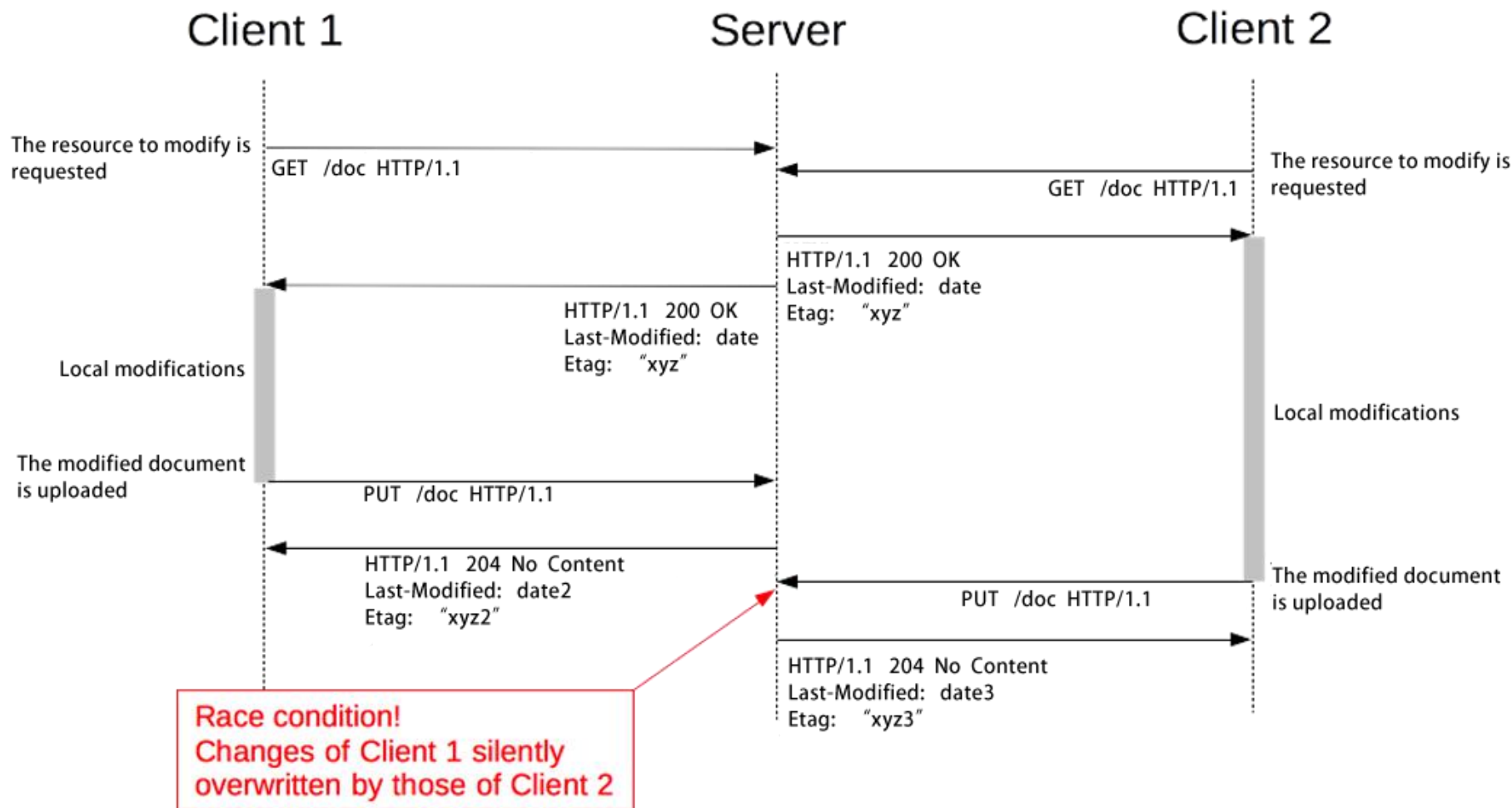
更新丢失问题（一）

更新资源意味着 2 步操作：先获取资源，再把本地修改后的资源提交



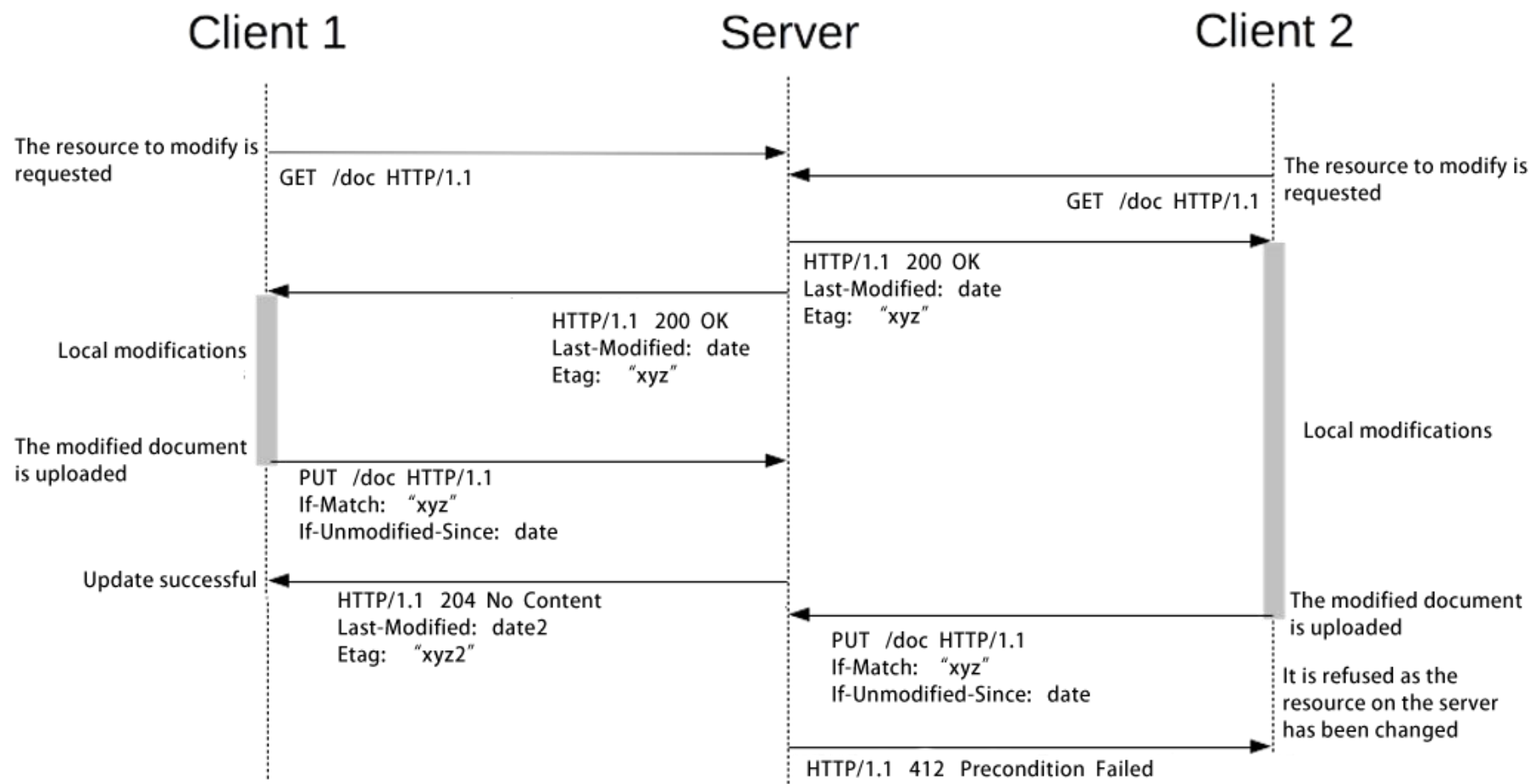
更新丢失问题（二）

2 个客户端并发修改同一资源会导致更新丢失

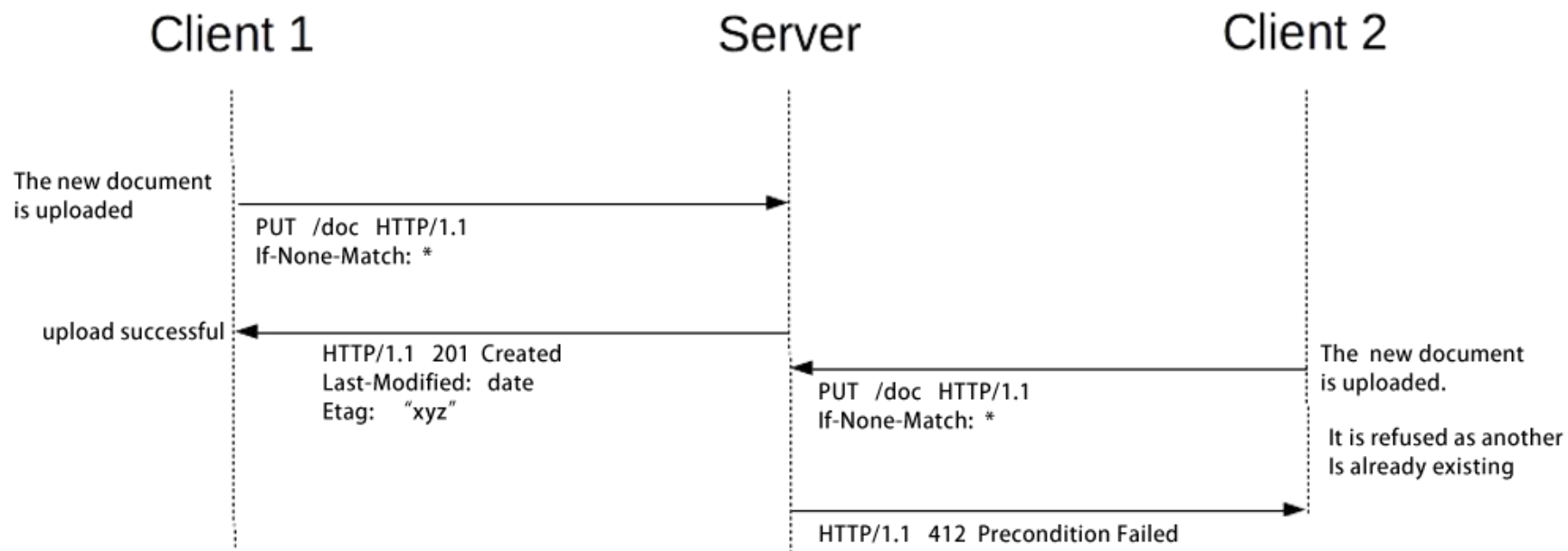


更新丢失问题（三）：乐观锁

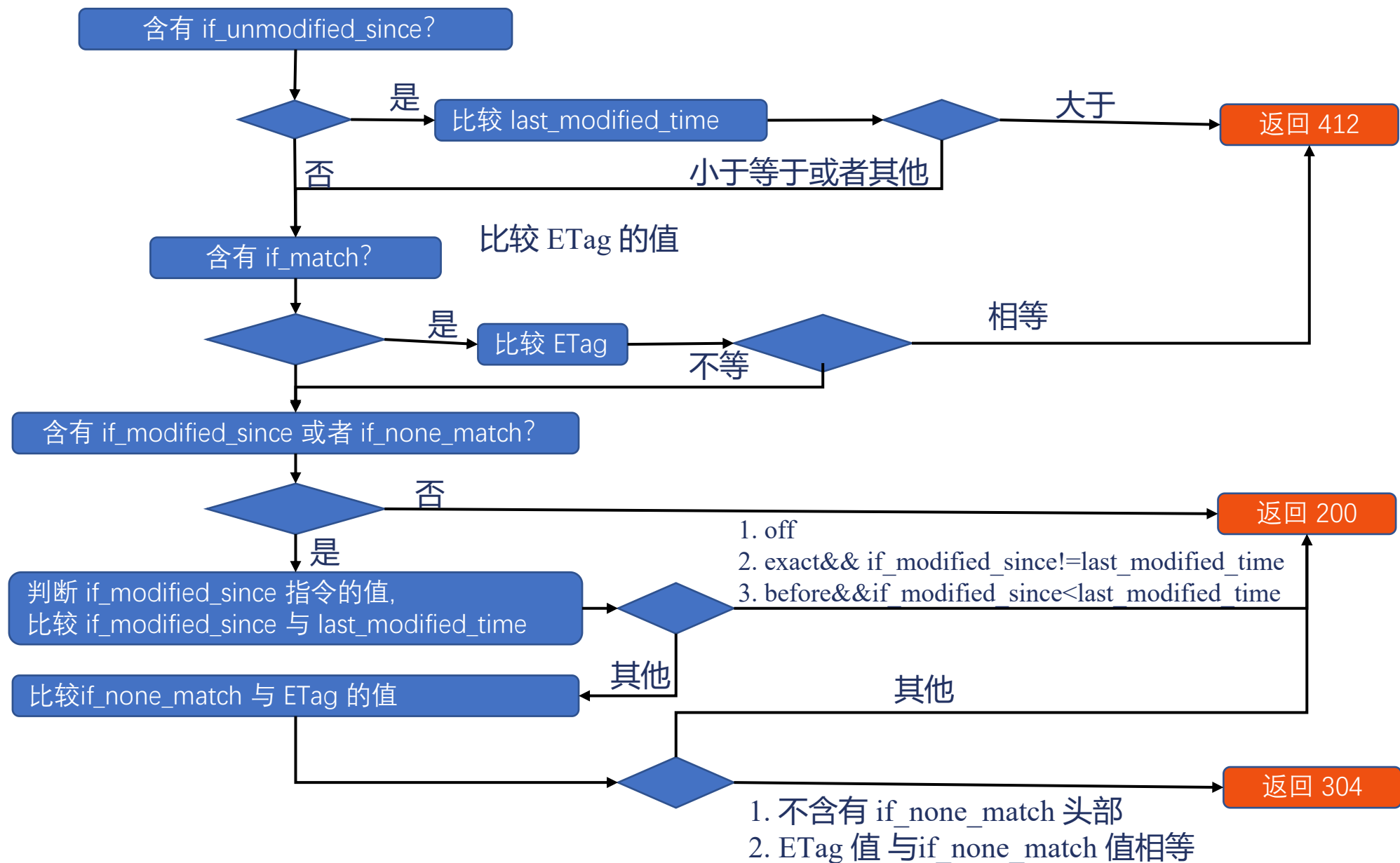
只允许第 1 个提交更新的客户端更新资源



更新丢失问题（四）：乐观锁解决首次上传

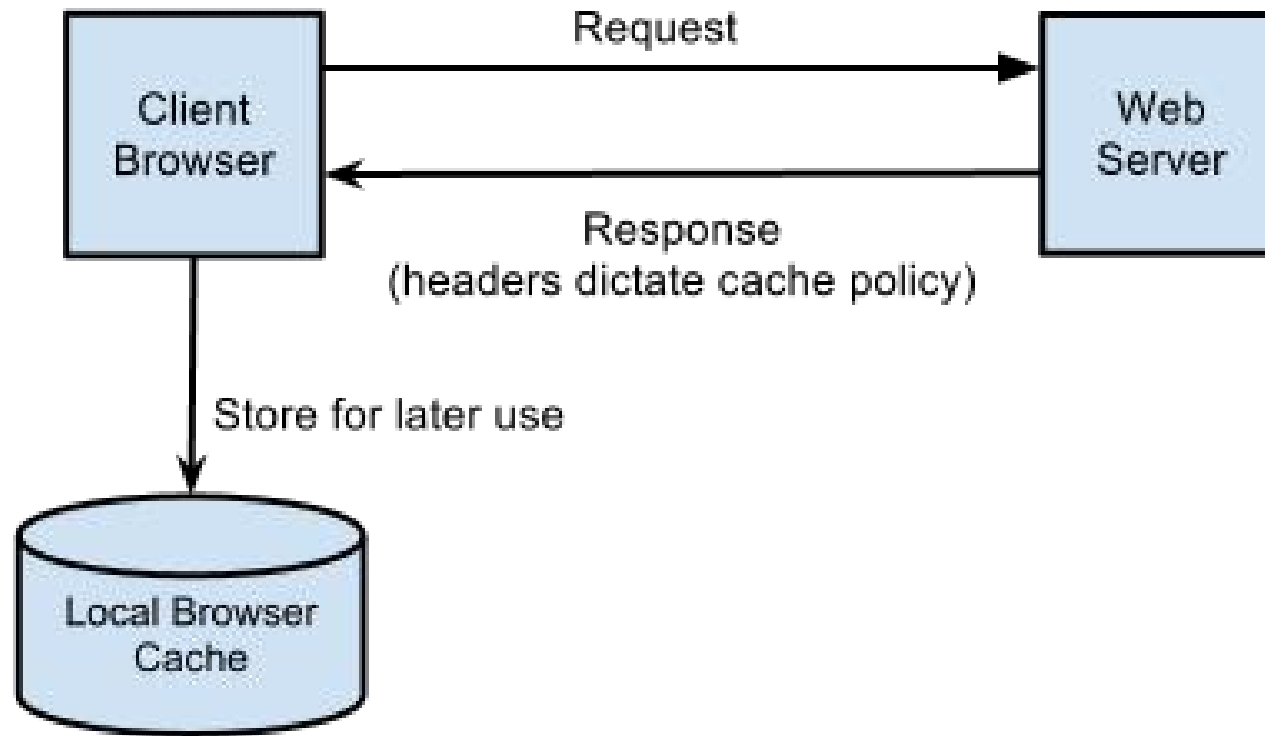


服务器处理条件请求的常见规则：Nginx



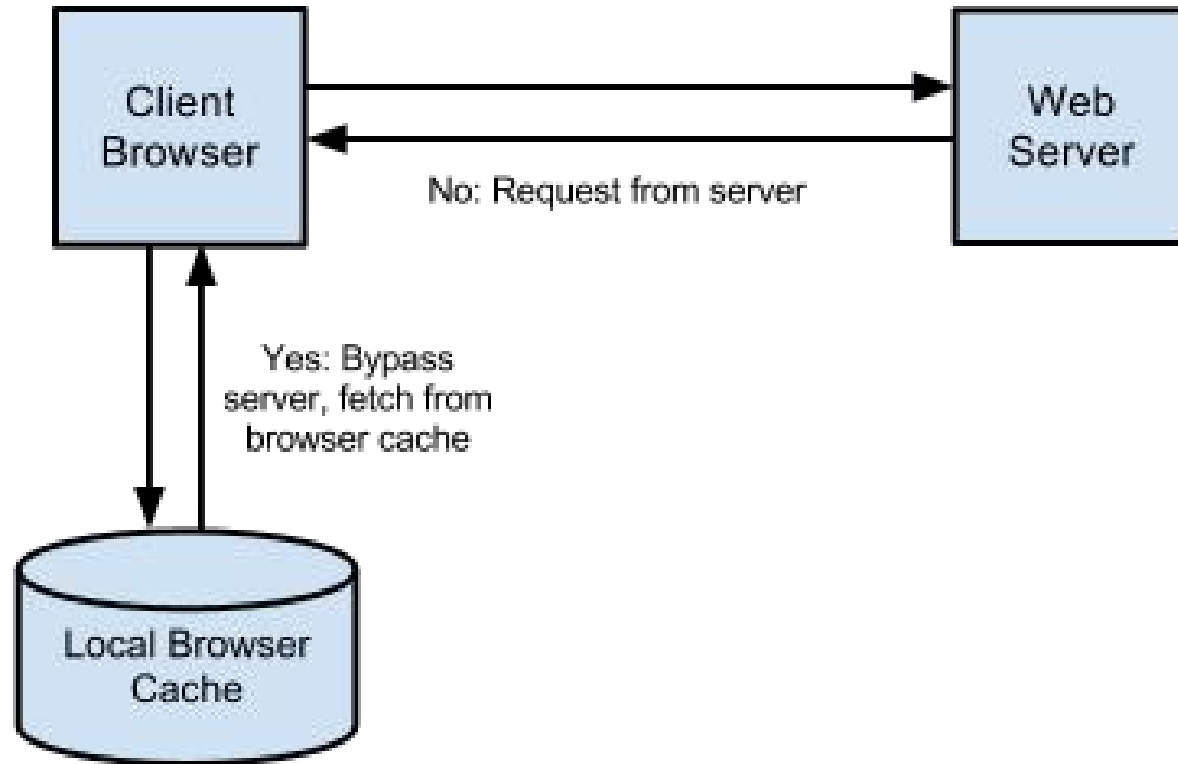
HTTP 缓存：为当前请求复用前请求的响应

- 目标：减少时延；降低带宽消耗
- 可选而又必要

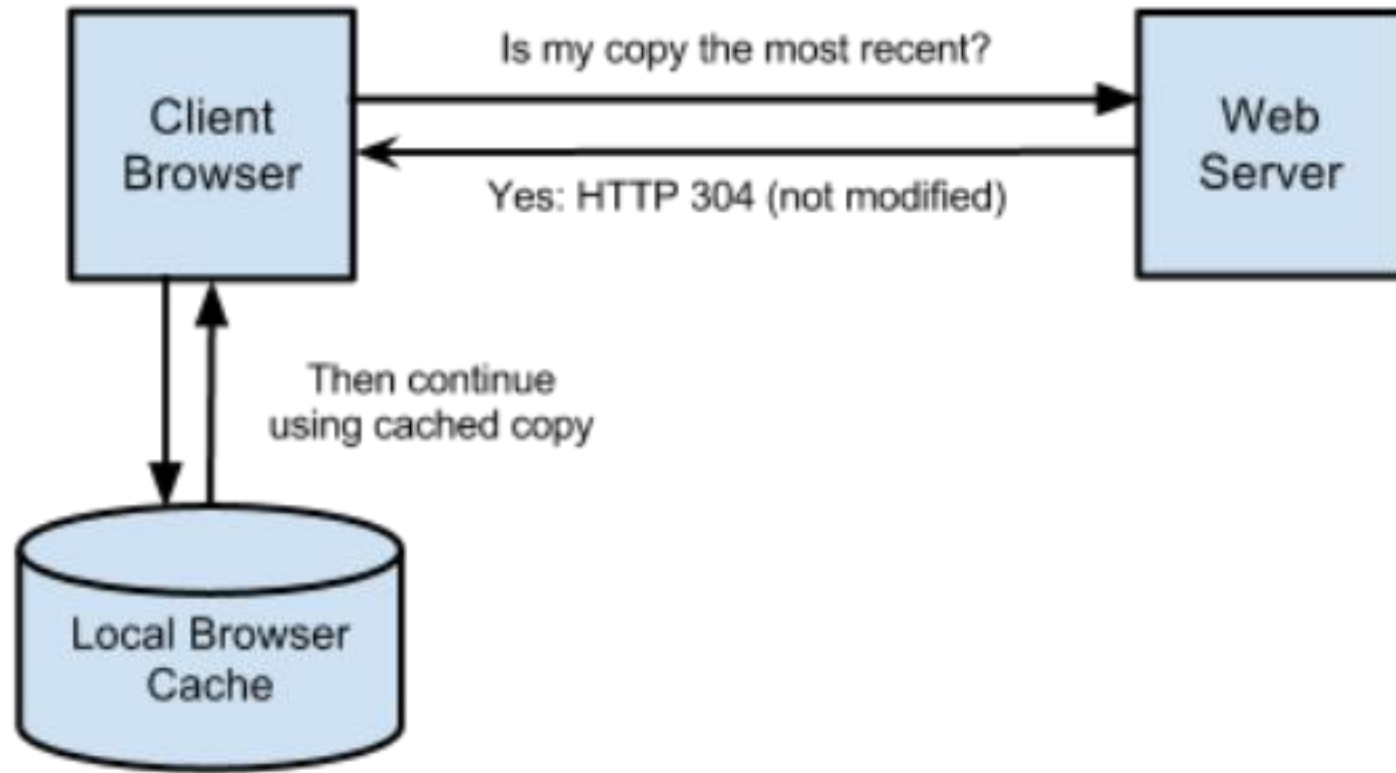


如果缓存没有过期

Is resource cached
and not stale?



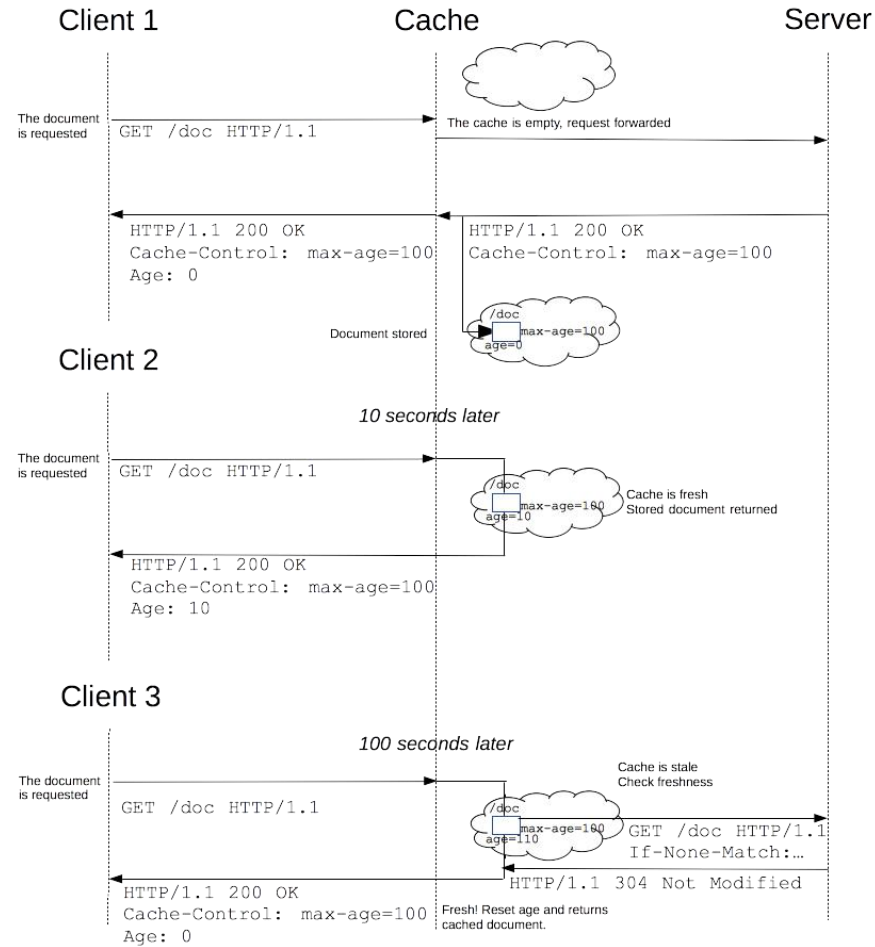
如果缓存过期，则继续从服务器验证



私有缓存与共享缓存

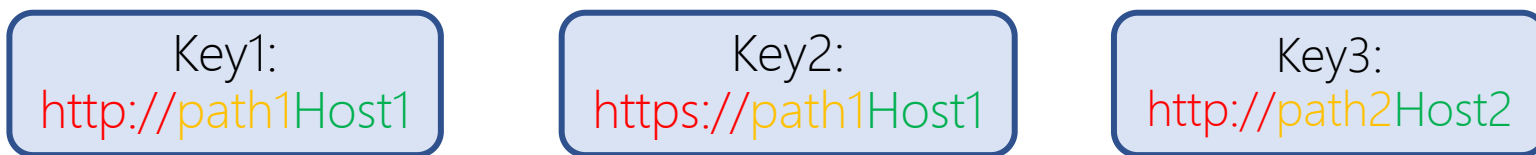
- 私有缓存：仅供一个用户使用的缓存，通常只存在于如浏览器这样的客户端上
- 共享缓存：可以供多个用户的缓存，存在于网络中负责转发消息的代理服务器（对热点资源常使用共享缓存，以减轻源服务器的压力，并提升网络效率）
 - Authentication 响应不可被代理服务器缓存
 - 正向代理
 - 反向代理

过期的共享缓存--代理服务器



缓存实现示意图

以 HTTP 请求中部分信息(如 schema、path、host)构成的字典



双向链
表头部



双向链
表尾部

HTTP响应消息构成的 LRU 链表

判断缓存是否过期

- `response_is_fresh = (freshness_lifetime > current_age)`

- `freshness_lifetime`: 按优先级, 取以下响应头部的值

- `s-maxage > max-age > Expires > 预估过期时间`

- 例如:

- Cache-Control: s-maxage=3600

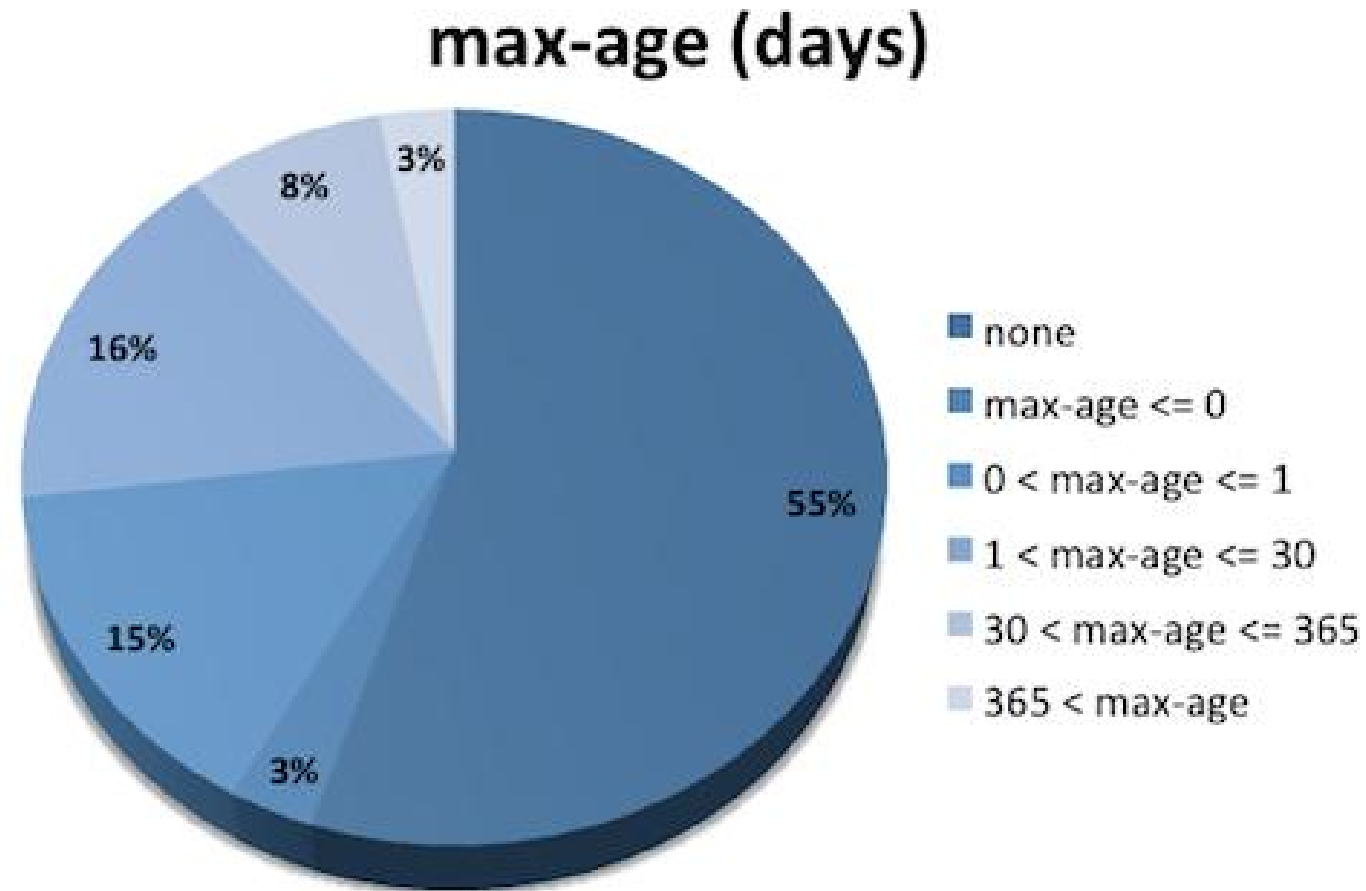
- Cache-Control: max-age=86400

- Expires: Fri, 03 May 2019 03:15:20 GMT

- Expires = HTTP-date, 指明缓存的绝对过期时间

```
04564:    p = ngx_strlcasestrn(start, last, (u_char *) "s-maxage=", 9 - 1);
04565:    offset = 9;
04566:
04567:    if (p == NULL) {
04568:        p = ngx_strlcasestrn(start, last, (u_char *) "max-age=", 8 - 1);
04569:        offset = 8;
04570:    }
```


为什么要预估缓存时间?



常见的预估时间

- RFC7234 推荐: $(\text{DownloadTime} - \text{LastModified}) * 10\%$

Chrome

```
1009 if ((response_code_ == 200 || response_code_ == 203 ||
1010      response_code_ == 206) &&
1011      !HasHeaderValue("cache-control", "must-revalidate")) {
1012     // TODO(darin): Implement a smarter heuristic.
1013     Time last_modified_value;
1014     if (GetLastModifiedValue(&last_modified_value)) {
1015         // The last-modified value can be a date in the future!
1016         if (last_modified_value <= date_value) {
1017             lifetimes.freshness = (date_value - last_modified_value) / 10;
1018             return lifetimes;
1019         }
1020     }
1021 }
```

Webkit

```
// Implicit Lifetime.
switch (response.httpStatusCode()) {
case 301: // Moved Permanently
case 410: // Gone
    // These are semantically permanent and so get long implicit lifetime.
    return 365 * 24h;
default:
    // Heuristic Freshness:
    // http://tools.ietf.org/html/rfc7234#section-4.2.2
    if (auto lastModified = response.lastModified())
        return duration_cast<microseconds>((effectiveDate - *lastModified) * 0.1);
    return 0s;
```

Firefox

```
743 // Fallback on heuristic using last modified header...
744 if (NS_SUCCEEDED(GetLastModifiedValue_locked(&date2))) {
745     LOG(("using last-modified to determine freshness-lifetime\n"));
746     LOG(("last-modified = %u, date = %u\n", date2, date));
747     if (date2 <= date) {
748         // this only makes sense if last-modified is actually in the past
749         *result = (date - date2) / 10;
750         const uint32_t kOneWeek = 60 * 60 * 24 * 7;
751         *result = std::min(kOneWeek, *result);
752         return NS_OK;
753     }
754 }
```

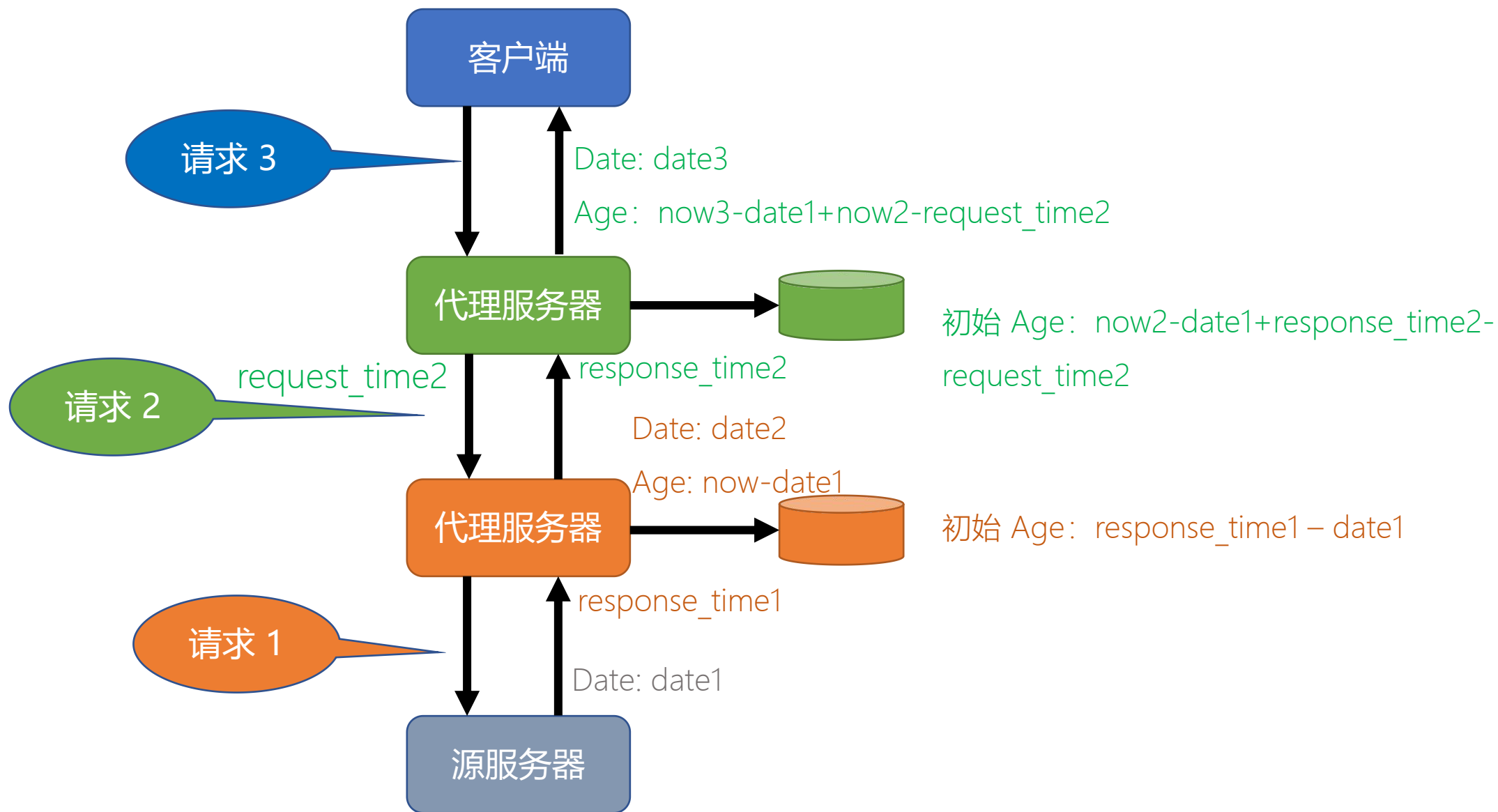
10% of time since Last-Modified

shorter of 10% since Last-Modified or one week.

Age 头部及 current_age 的计算

- Age 表示自源服务器发出响应（或者验证过期缓存），到使用缓存的响应发出时经过的秒数
 - 对于代理服务器管理的共享缓存，客户端可以根据 Age 头部判断缓存时间
 - $\text{Age} = \text{delta-seconds}$
- current_age 计算: $\text{current_age} = \text{corrected_initial_age} + \text{resident_time};$
 - $\text{resident_time} = \text{now} - \text{response_time}(\text{接收到响应的时间});$
 - $\text{corrected_initial_age} = \max(\text{apparent_age}, \text{corrected_age_value});$
 - $\text{corrected_age_value} = \text{age_value} + \text{response_delay};$
 - $\text{response_delay} = \text{response_time} - \text{request_time}(\text{发起请求的时间});$
 - $\text{apparent_age} = \max(0, \text{response_time} - \text{date_value});$

代理服务器缓存中的 Age 头部



Cache-Control 头部

- Cache-Control = 1#cache-directive
 - cache-directive = token ["=" (token / quoted-string)]
 - `delta-seconds = 1*DIGIT`
 - RFC 规范中的要求是, 至少能支持到 2147483648 (2^{31})
- 请求中的头部: `max-age`、`max-stale`、`min-fresh`、`no-cache`、`no-store`、`no-transform`、`only-if-cached`
- 响应中的头部: `max-age`、`s-maxage`、`must-revalidate`、`proxy-revalidate`、`no-cache`、`no-store`、`no-transform`、`public`、`private`

Cache-Control 头部在请求中的值

- **max-age**: 告诉服务器，客户端不会接受 Age 超出 max-age 秒的缓存
- **max-stale**: 告诉服务器，即使缓存不再新鲜，但陈旧秒数没有超出 max-stale 时，客户端仍打算使用。若 max-stale 后没有值，则表示无论过期多久客户端都可使用
- **min-fresh**: 告诉服务器，Age 至少经过 min-fresh 秒后缓存才可使用
- **no-cache**: 告诉服务器，不能直接使用已有缓存作为响应返回，除非带着缓存条件到上游服务端得到 304 验证返回码才可使用现有缓存
- **no-store**: 告诉各代理服务器不要对该请求的响应缓存（实际有不少不遵守该规定的代理服务器）
- **no-transform**: 告诉代理服务器不要修改消息包体的内容
- **only-if-cached**: 告诉服务器仅能返回缓存的响应，否则若没有缓存则返回 504 错误码

Cache-Control 头部在响应中的值

- `must-revalidate`: 告诉客户端一旦缓存过期，必须向服务器验证后才可使用
- `proxy-revalidate`: 与 `must-revalidate` 类似，但它仅对代理服务器的共享缓存有效
- `no-cache`: 告诉客户端不能直接使用缓存的响应，使用前必须在源服务器验证得到 304 返回码。如果 `no-cache` 后指定头部，则若客户端的后续请求及响应中不含有这些头则可直接使用缓存
- `max-age`: 告诉客户端缓存 Age 超出 `max-age` 秒后则缓存过期

Cache-Control 头部在响应中的值

- **s-maxage**: 与 max-age 相似，但仅针对共享缓存，且优先级高于 max-age 和 Expires
- public: 表示无论私有缓存或者共享缓存，皆可将该响应缓存
- **private**: 表示该响应不能被代理服务器作为共享缓存使用。若 private 后指定头部，则在告诉代理服务器不能缓存指定的头部，但可缓存其他部分
- no-store: 告诉所有下游节点不能对响应进行缓存
- no-transform: 告诉代理服务器不能修改消息包体的内容

什么样的 HTTP 响应会缓存? RFC7234

- 请求方法可以被缓存理解（不只于 GET 方法）
- 响应码可以被缓存理解（404、206 也可以被缓存）
- 响应与请求的头部没有指明 no-store
- 响应中至少应含有以下头部中的 1 个或者多个：
 - Expires、max-age、s-maxage、public
 - 当响应中没有明确指示过期时间的头部时，如果响应码非常明确，也可以缓存
- 如果缓存在代理服务器上
 - 不含有 private
 - 不含有 Authorization

其他响应头部

- Pragma = 1#pragma-directive
 - pragma-directive = "no-cache" / extension-pragma
 - extension-pragma = token ["=" (token / quoted-string)]
- Pragma: no-cache与Cache-Control: no-cache 意义相同

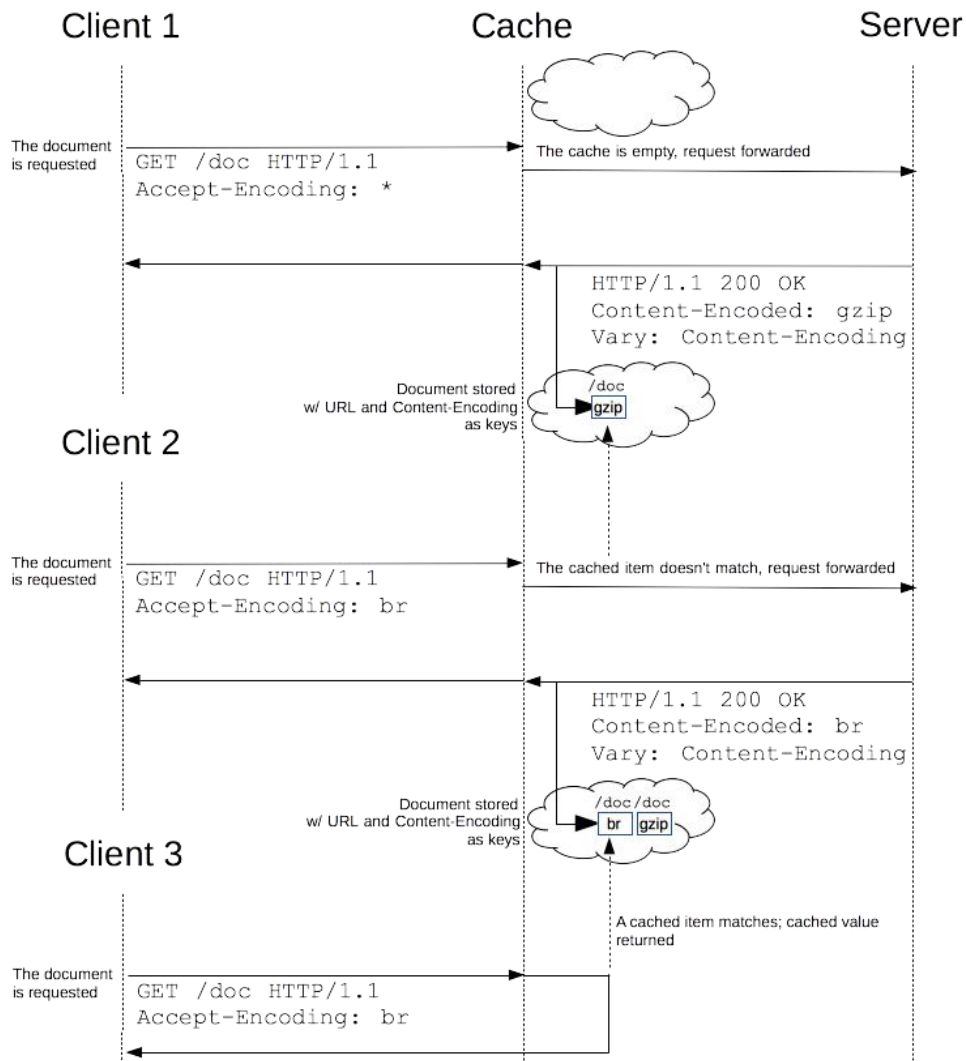
使用缓存作为当前请求响应的条件

- URI 是匹配的
 - URI 作为主要的缓存关键字，当一个 URI 同时对应多份缓存时，选择日期最近的缓存
 - 例如 Nginx 中默认的缓存关键字：`proxy_cache_key`
`$scheme$proxy_host$request_uri;`
- 缓存中的响应允许当前请求的方法使用缓存

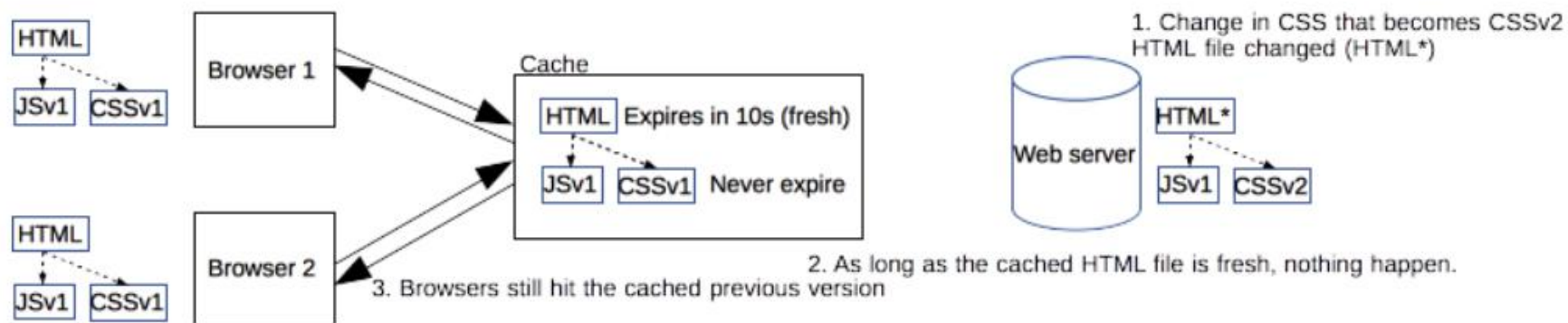
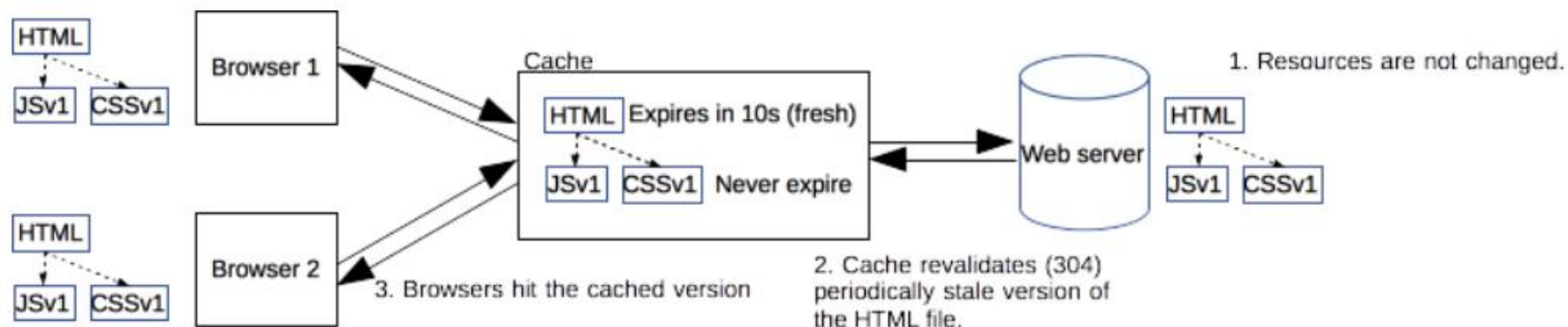
使用缓存作为当前请求响应的条件

- 缓存中的响应 Vary 头部指定的头部必须与请求中的头部相匹配：
 - `Vary = "*" / 1#field-name`
 - Vary: *意味着一定匹配失败
- 当前请求以及缓存中的响应都不包含 no-cache 头部 (Pragma: no-cache 或者 Cache-Control: no-cache)
- 缓存中的响应必须是以下三者之一：
 - 新鲜的 (时间上未过期)
 - 缓存中的响应头部明确告知可以使用过期的响应 (如 Cache-Control: max-stale=60)
 - 使用条件请求去服务器端验证请求是否过期, 得到 304 响应

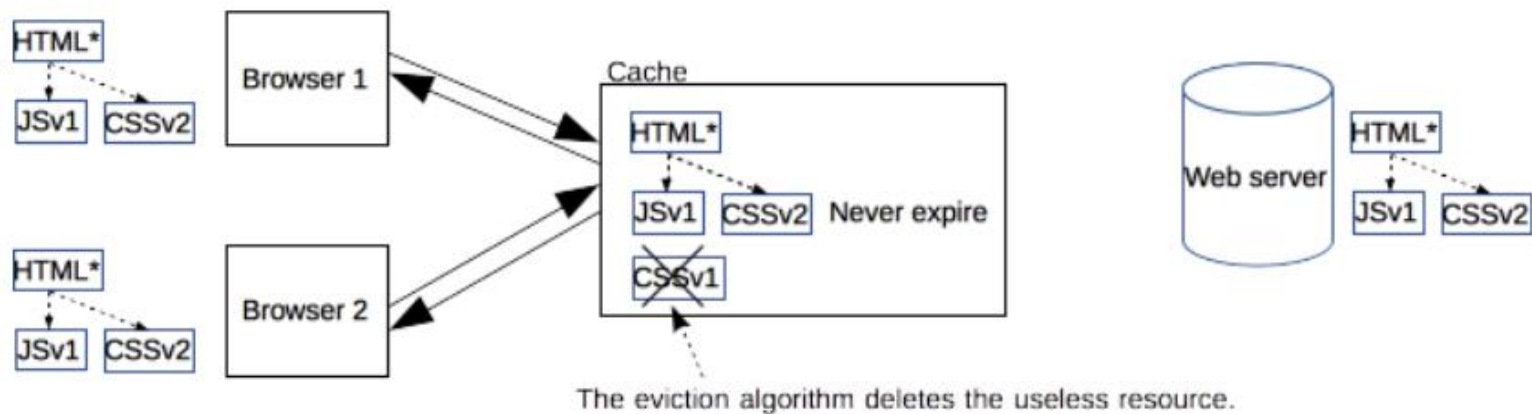
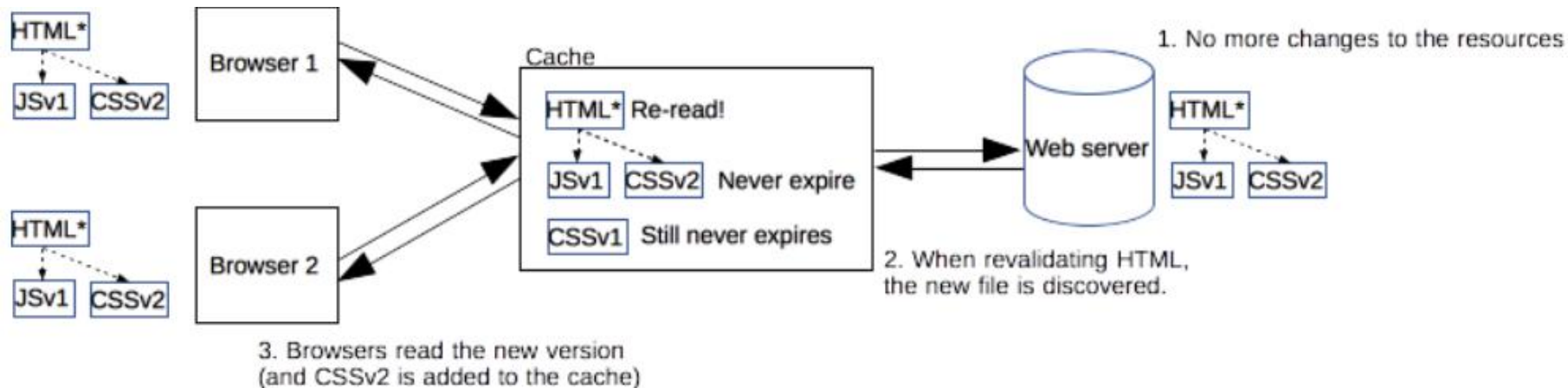
Vary 缓存



如何缓存更新频率不同的资源（一）



如何缓存更新频率不同的资源（二）



Warning 头部：对响应码进行补充（缓存或包体转换）

- Warning = 1#warning-value
 - warning-value = warn-code SP warn-agent SP warn-text [SP warn-date]
 - warn-code = 3DIGIT
 - warn-agent = (uri-host [":" port]) / pseudonym
 - warn-text = quoted-string
 - warn-date = DQUOTE HTTP-date DQUOTE
- 常见的 warn-code
 - Warning: 110 - "Response is Stale"
 - Warning: 111 - "Revalidation Failed"
 - Warning: 112 - "Disconnected Operation"
 - Warning: 113 - "Heuristic Expiration"
 - Warning: 199 - "Miscellaneous Warning"
 - Warning: 214 - "Transformation Applied"
 - Warning: 299 - "Miscellaneous Persistent Warning"

验证请求与响应

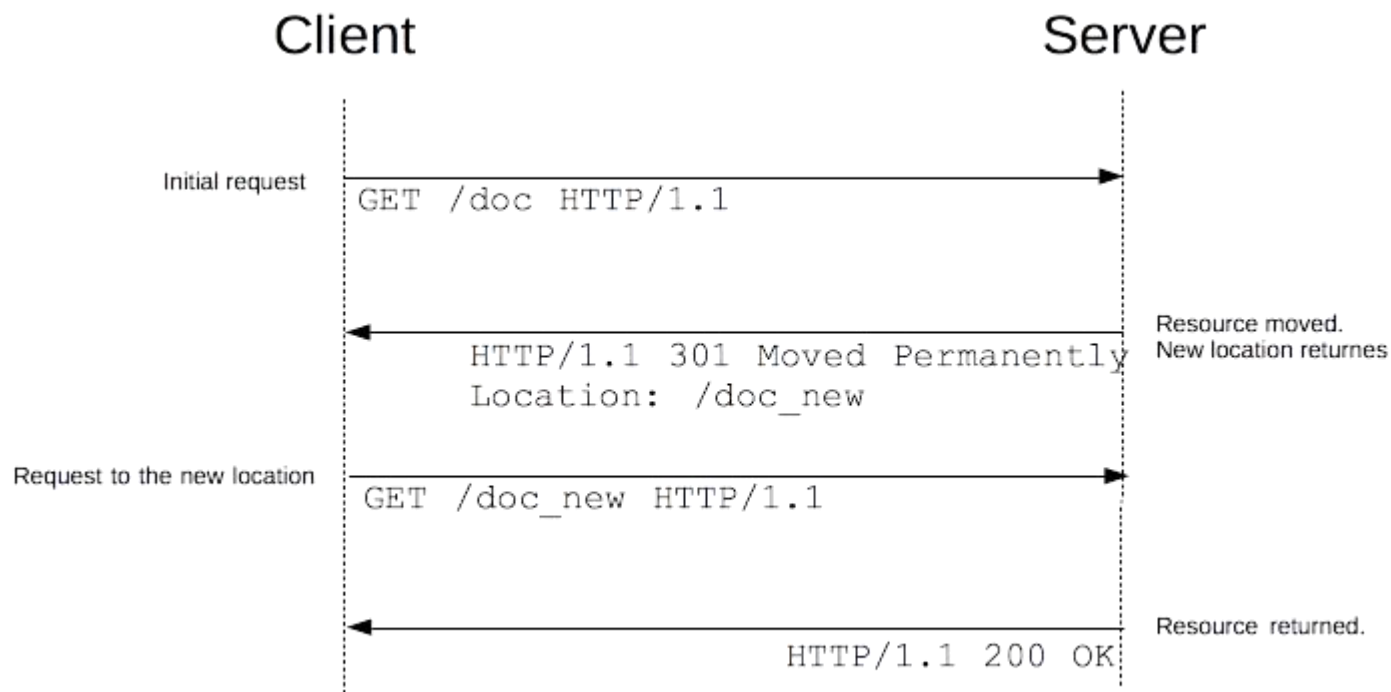
- 验证请求
 - 若缓存响应中含有 Last-Modified 头部
 - If-Unmodified-Since
 - If-Modified-Since
 - If-Range
 - 若缓存响应中含有 Etag 头部
 - If-None-Match
 - If-Match
 - If-Range

为什么需要 URI 重定向?

- 提交 FORM 表单成功后需要显示内容页，怎么办？
- 站点从 HTTP 迁移到 HTTPS，怎么办？
- 站点部分 URI 发生了变化，但搜索引擎或者流量入口站点只收录了老的 URI，怎么办？
- 站点正在维护中，需要给用户展示不一样的内容，怎么办？
- 站点更换了新域名，怎么办？

重定向的流程

- 当浏览器接收到重定向响应码时，需要读取响应头部 Location 头部的值，获取到新的 URI 再跳转访问该页面



Location 头部

- Location = URI-reference (对 201 响应码表示新创建的资源)

URI-reference = URI/relative-ref

- relative-ref = relative-part ["?" query] ["#" fragment]
 - relative-part = "//" authority path-abempty / path-absolute / path-noscheme / path-empty

重定向响应返回码

- 概念
 - 原请求：接收到重定向响应码的请求这里称为原请求
 - 重定向请求：浏览器接收到重定向响应码后，会发起新的重定向请求
- 永久重定向，表示资源永久性变更到新的 URI
 - 301 (HTTP/1.0)：重定向请求**通常**（由于历史原因一些浏览器会把 POST 改为 GET）会**使用 GET 方法**，而不管原请求究竟采用的是**什么方法**
 - 308 (HTTP/1.1)：重定向请求必须**使用原请求的方法和包体**发起访问

重定向响应返回码

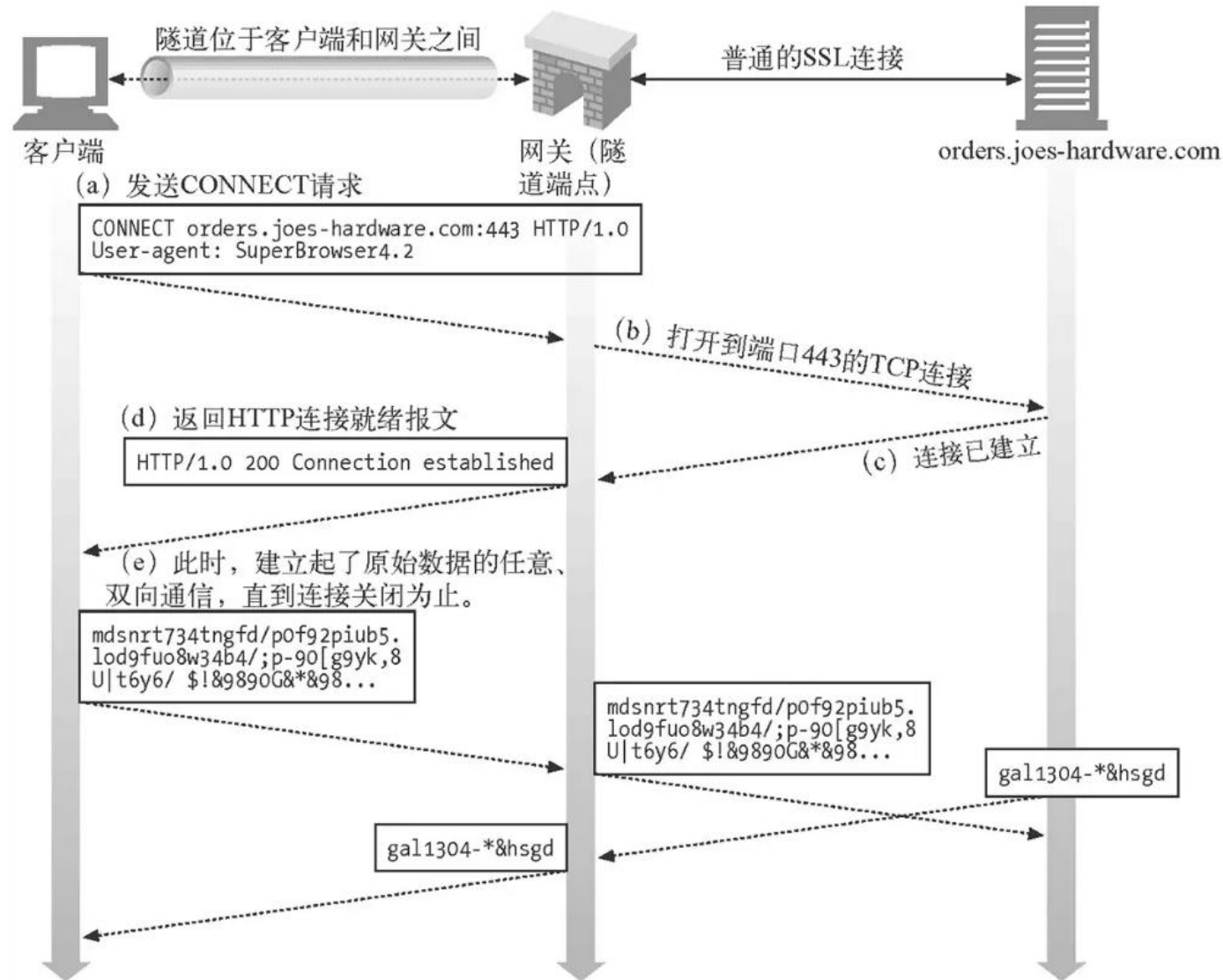
- **临时**重定向，表示资源只是临时的变更 URI
 - 302 (HTTP/1.0)：重定向请求**通常会使用 GET 方法**，而不管原请求究竟采用的是**什么方法**
 - 303 (HTTP/1.1)：它并不表示资源变迁，而是用新 URI 的响应表述而为原请求服务，重定向请求会**使用 GET 方法**
 - 例如表单提交后向用户返回新内容（亦可防止重复提交）
 - 307 (HTTP/1.1)：重定向请求必须**使用原请求的方法和包体**发起访问
- 特殊重定向
 - 300：响应式内容协商中，告知客户端有多种资源表述，要求客户端选择一种自认为合适的表述
 - 304：服务器端验证过期缓存有效后，要求客户端使用该缓存

重定向循环

- 服务器端在生成 Location 重定向 URI 时，在同一条路径上使用了之前的 URI，导致无限循环出现
- Chrome 浏览器会提示：ERR_TOO_MANY_REDIRECTS

Http Tunnel 隧道

- 用于通过 HTTP 连接传输非 HTTP 协议格式的消息，常用于穿越防火墙
- 建立隧道后，由于传输的并非 HTTP 消息，因此不再遵循请求/响应模式，已变为双向传输



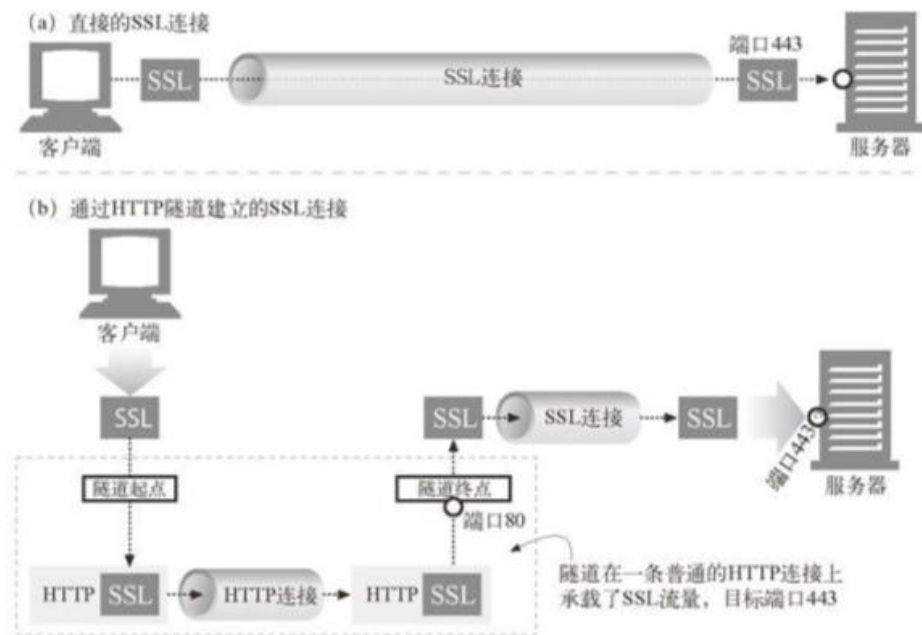
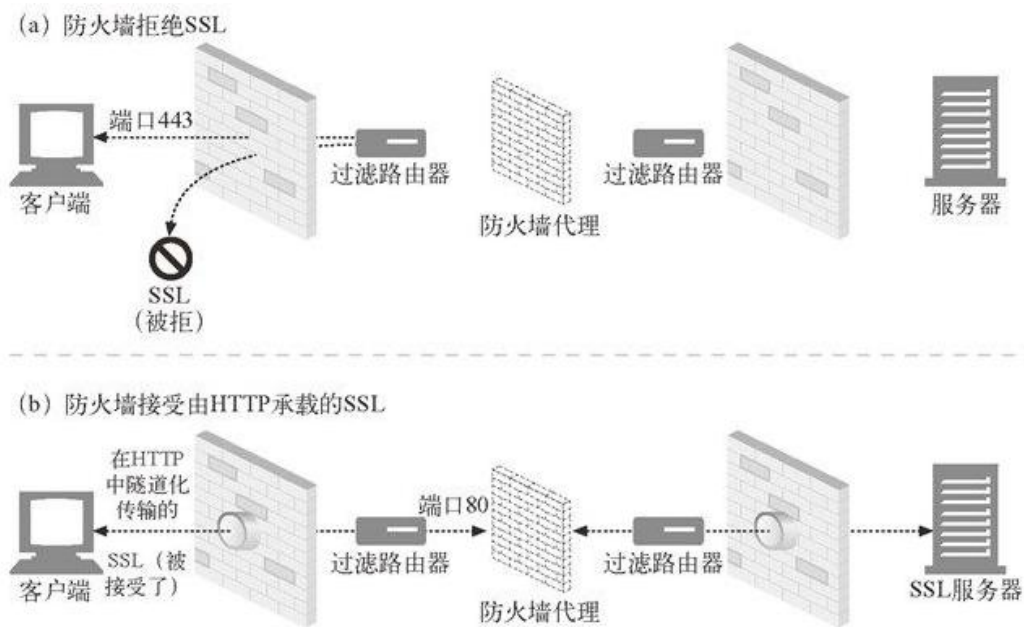
request-line = method SP request-target SP HTTP-version CRLF

request-target = origin-form / absolute-form / **authority-form** / asterisk-form

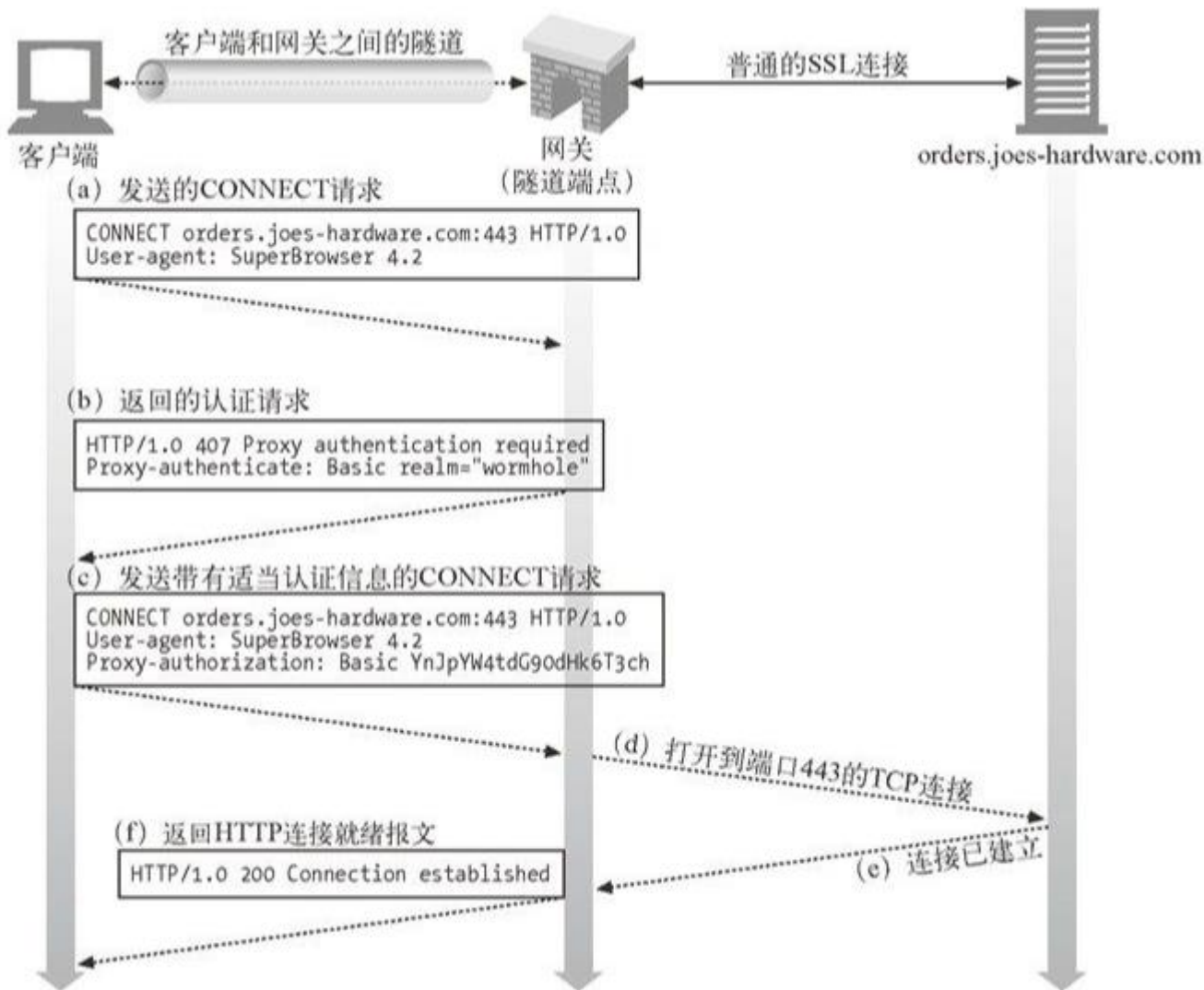
- **origin-form** = absolute-path ["?" query]
 - 向源服务器发起的请求, path 为空时必须传递 /
- **absolute-form** = absolute-URI
 - 仅用于向正向代理 proxy 发起请求时, 详见正向代理与隧道
- **authority-form** = **authority**
 - authority = [userinfo "@"] host [":" port], 指定源服务器
 - 仅用于 CONNECT 方法, 例如 CONNECT www.example.com:80 HTTP/1.1
- **asterisk-form** = "*" "
 - 仅用于 OPTIONS 方法

tunnel 隧道的常见用途：传递 SSL 消息

- 防火墙拒绝 SSL 流量怎么办？
- 代理服务器没有证书，如何转发 SSL 流量？

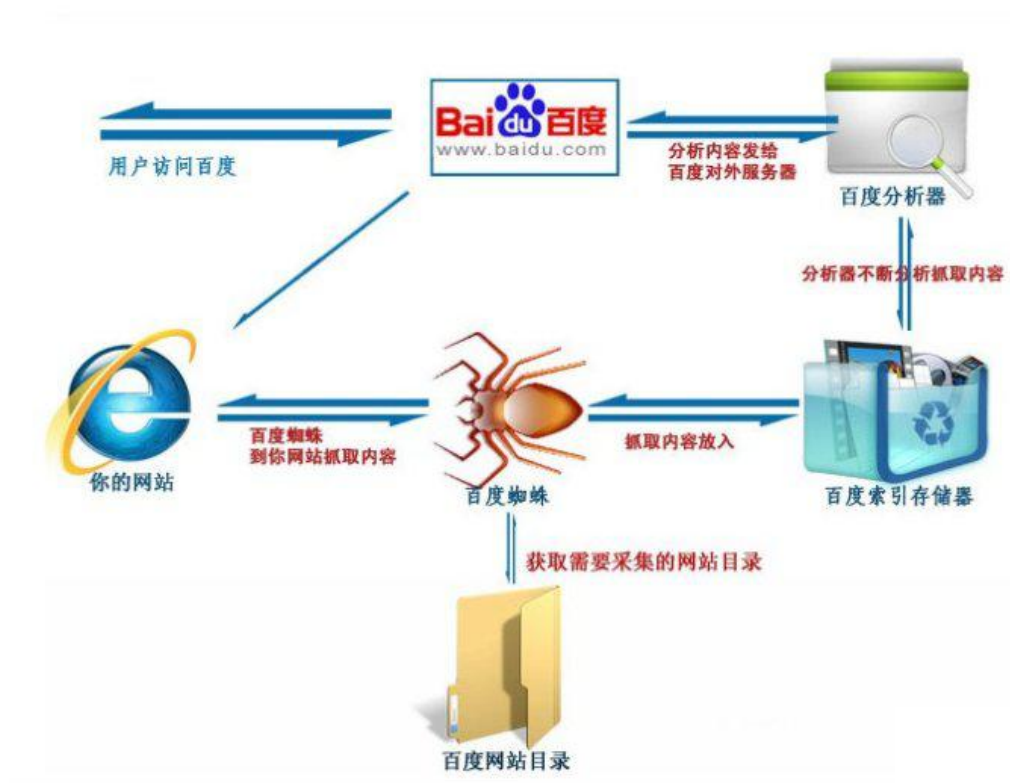


Http Tunnel 隧道的认证



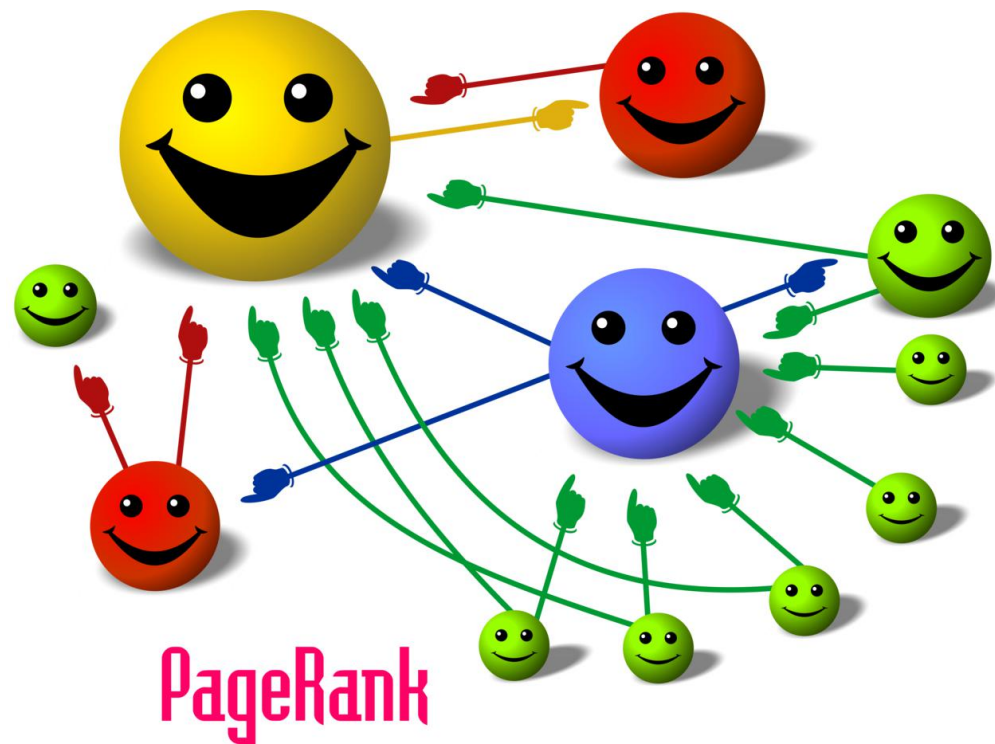
Web crawler: 网络爬虫

- 网络爬虫模拟人类使用浏览器浏览、操作页面的行为，对互联网的站点进行操作
- 网络爬虫获取到一个页面后，会分析出页面里的所有 URI，沿着这些 URI 路径递归的遍历所有页面，因此被称为爬虫（Web crawler）、蜘蛛（Spider）、网络机器人（spiderbot）



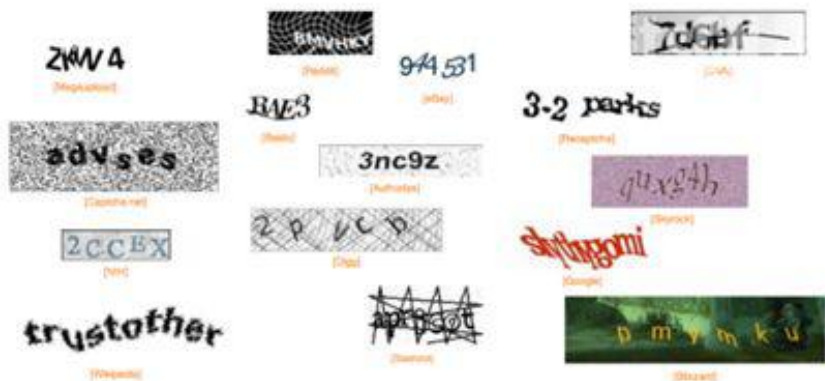
对待网络爬虫的 2 种态度：欢迎常光临

- SEO (Search Engine Optimization) , 搜索引擎优化
 - “合法”的优化: sitemap、title、keywords、https 等
 - “非法”的优化: 利用 PageRank 算法漏洞



对待网络爬虫的 2 种态度：拒绝访问

- 为了对抗网络爬虫而生的图形验证码



对待网络爬虫的 2 种态度：拒绝访问



- 为了对抗图形验证码而生的“打码平台(captcha human bypass)”

	通用字符识别 全面的图像内容识别服务，包括纯数字、纯英文、英文数字混合、纯汉字、数字、英文、汉字混合(含各类验证码，滑块，坐标题)。		复杂场景识别定制 适针对复杂场景，提供定制化的人工处理服务，可识别计算题、问答题、知识题
	大数据人工标注 适应于大规模图像、视频、语音、文本以及其他特殊数据的数据清洗、评估、提取以及特殊信息标注。专业的标注团队，高效、稳定提供数据标注服务		人工智能识别定制 采用先进的深度学习模型，为各行业提供专业的人工智能解决方案，为企业降低成本、提高效率助力

对待网络爬虫的 2 种态度：拒绝访问

- 升级图形验证码

 hello@geetest.com



 点击按钮进行验证 

登录

短信验证

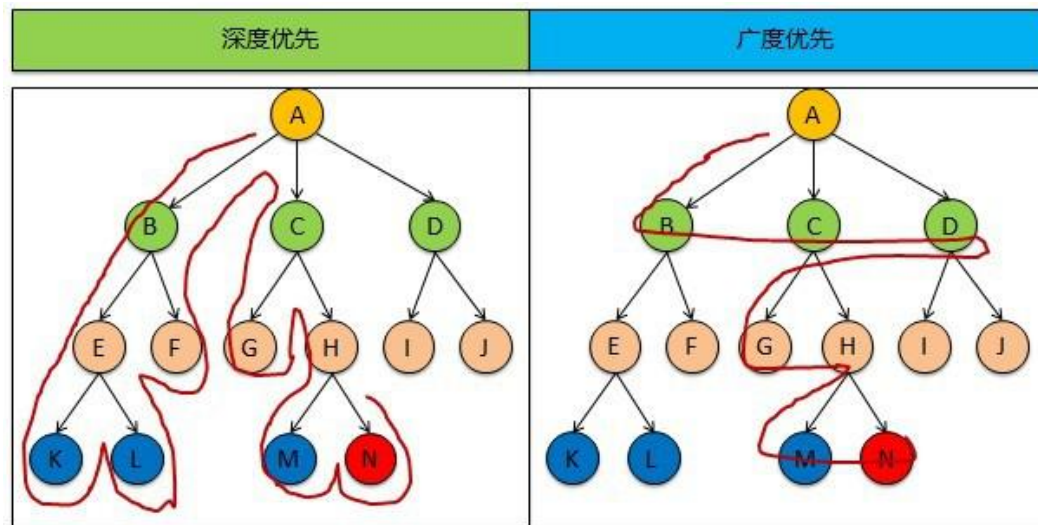
✓ 验证码短信已发送到您的手机 186*****10 [更换手机](#)

请填写验证码: 没有收到验证码 

确定

网络爬虫如何抓取数据？

- 模拟浏览器渲染引擎，需要对 JavaScript 文件分析执行、发起 Ajax 请求等
- 爬虫爬取数据的速度 VS 互联网生成信息的速度
 - 爬虫执行速度快，许多爬虫可以并发执行
 - 互联网生成信息的速度远大于爬取速度
- 优先爬取更重要的页面

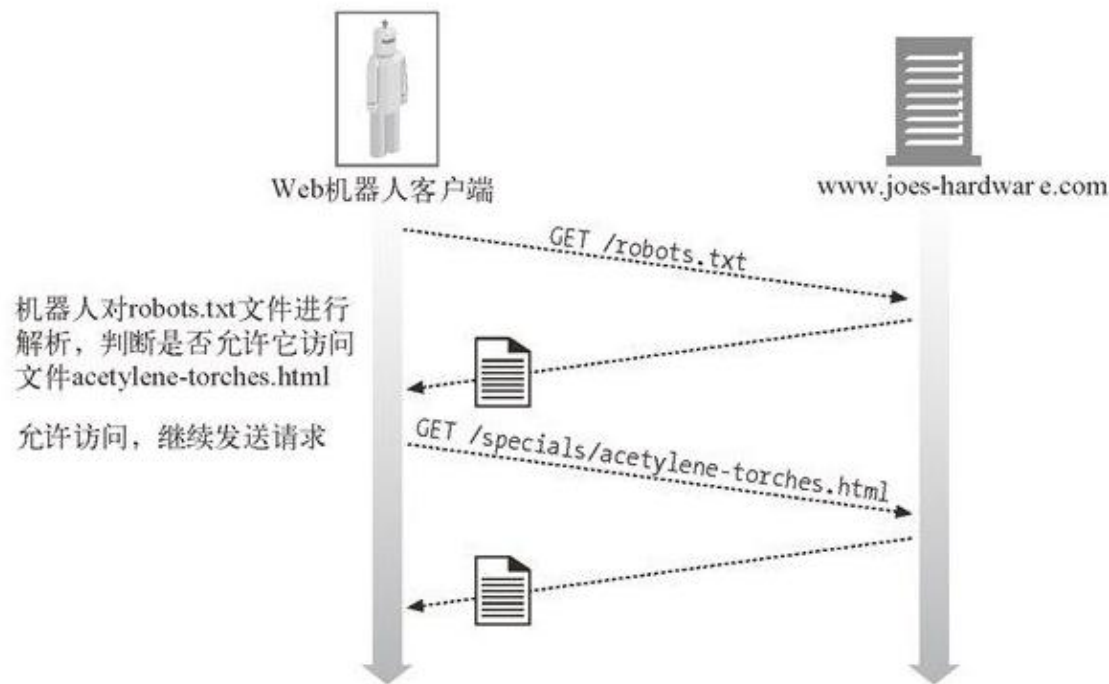


爬虫常见的请求头部

- User-Agent: 识别是哪类爬虫
- From: 提供爬虫机器人管理者的邮箱地址
- Accept: 告知服务器爬虫对哪些资源类型感兴趣
- Referer: 相当于包含了当前请求的页面 URI

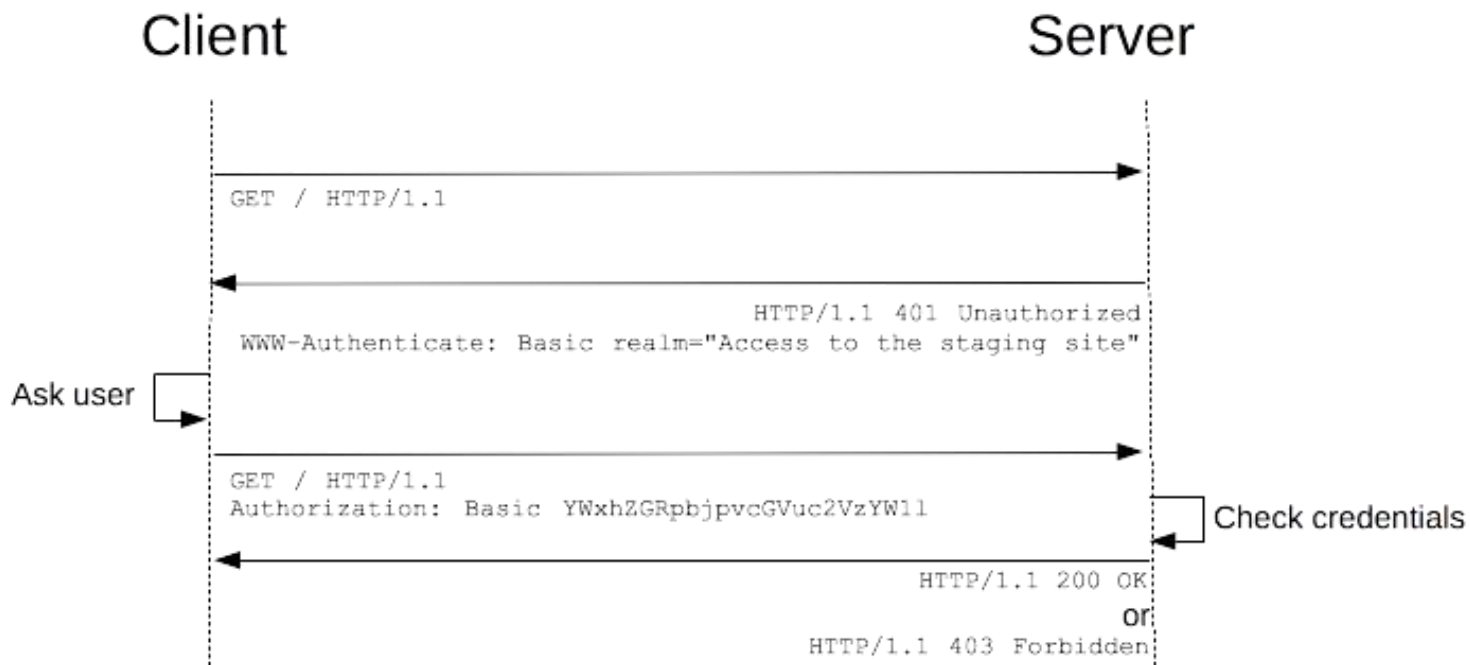
robots.txt：告知爬虫哪些内容不应爬取

- Robots exclusion protocol: <http://www.robotstxt.org/orig.html>
- robots.txt 文件内容
 - **User-agent**: 允许哪些机器人
 - **Disallow**: 禁止访问特定目录
 - **Crawl-delay**: 访问间隔秒数
 - **Allow**: 抵消 Disallow 指令
 - **Sitemap**: 指出站点地图的 URI



HTTP 基本认证

- RFC7235, 一种基本的验证框架, 被绝大部分浏览器所支持
- 明文传输, 如果不使用 TLS/SSL 传输则有安全问题



认证请求

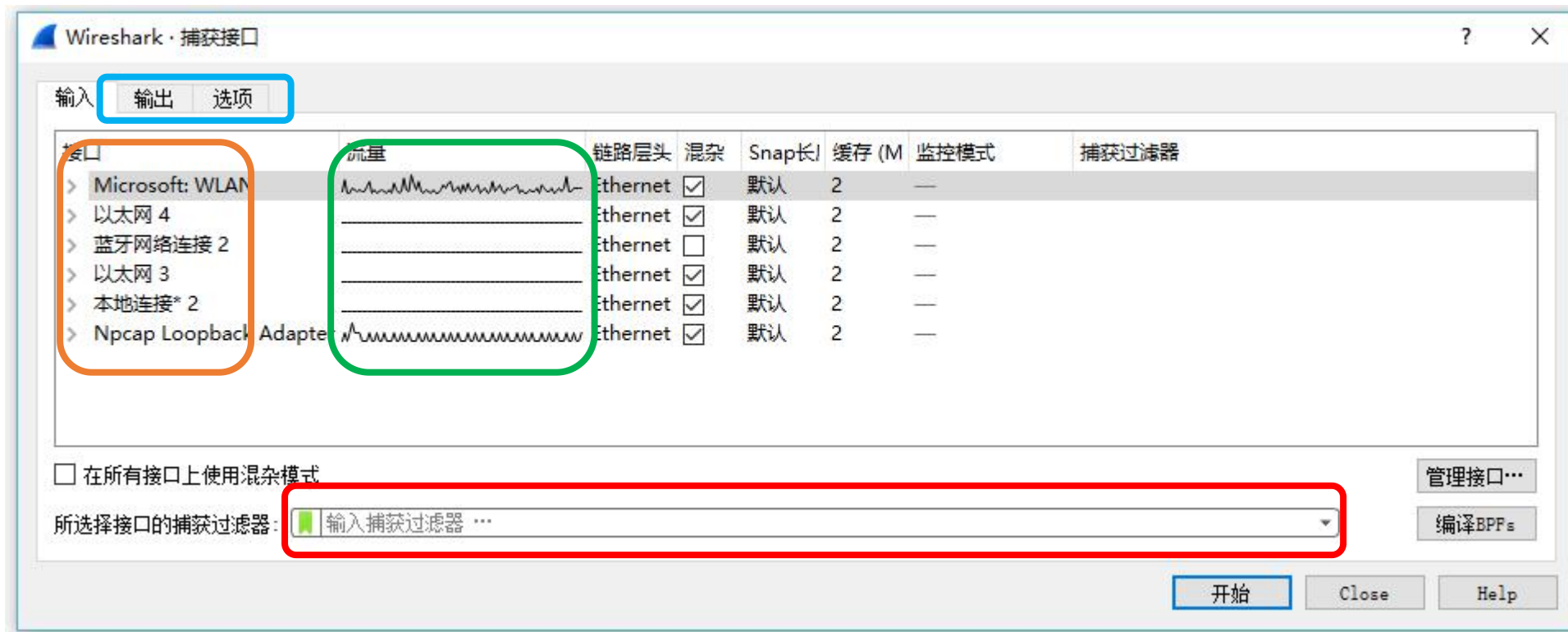
- 在请求中传递认证信息: Authorization = credentials
 - credentials = auth-scheme [1*SP (token68 / #auth-param)]
 - auth-scheme = token
 - token68 = 1*(ALPHA / DIGIT / "-" / "." / "_" / "~" / "+" / "/") * "="
 - auth-param = token BWS "=" BWS (token / quoted-string)
 - BWS = OWS
 - OWS = *(SP / HTAB)
 - 例如: authorization: Basic ZGQ6ZWU=
 - 实际 ZGQ6ZWU= 是 dd:ee 的 base64 编码, 表示用户名和密码
- 由代理服务器认证: Proxy-Authorization = credentials

认证响应

- 在响应头部中告知客户端需要认证: WWW-Authenticate = 1#challenge
 - challenge = auth-scheme [1*SP (token68 / #auth-param)]
 - auth-scheme = token
 - token68 = 1*(ALPHA / DIGIT / "-" / "." / "_" / "~" / "+" / "/") * "="
 - auth-param = token BWS "=" BWS (token / quoted-string)
 - BWS = OWS
 - OWS = *(SP / HTAB)
 - 例如: www_authenticate: Basic realm="test_auth_basic"
- 由代理服务器认证: Proxy-Authenticate = 1#challenge
- 认证响应码
 - 由源服务器告诉客户端需要传递认证信息: 401 Unauthorized
 - 由代理服务器认证: 407 Proxy Authentication Required
 - 认证失败: 403 Forbidden

如何捕获报文

- 点击捕获->选项，打开捕获窗口
 - 网卡设备/流量/捕获过滤器，点击“开始”按钮开始抓包
 - 输出（指定缓存文件）/选项（显示、名称解析、自动停止抓包条件） 面板
- 点击捕获->停止，停止抓包



Wireshark 面板

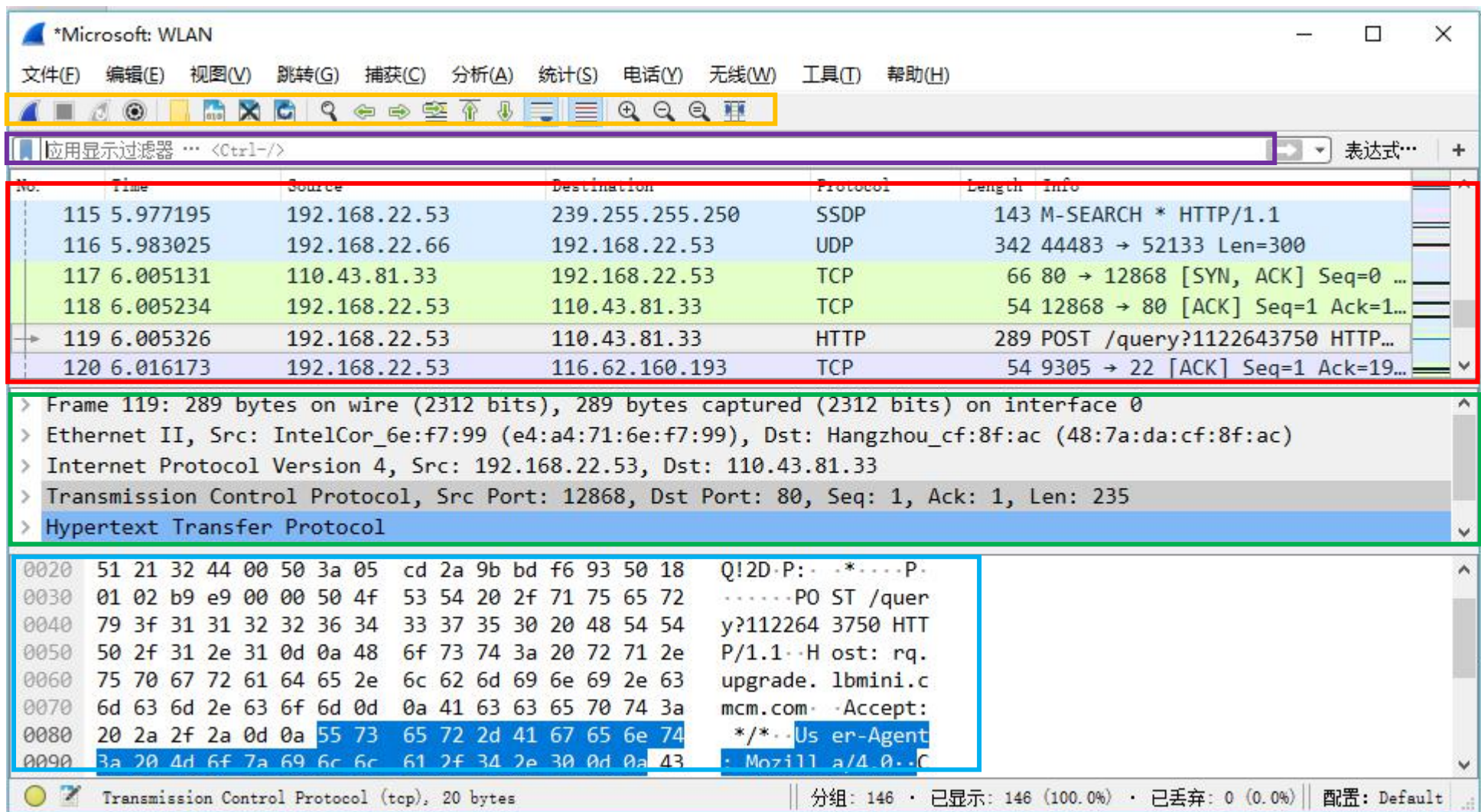
工具栏

显示过滤器

数据包列表

数据包细节

数据包字节流



快捷方式工具栏



- 开始抓包 (捕获->开始)



- 停止抓包 (捕获->停止)



- 重新开始抓包 (捕获->重新开始)



- 打开选项面板抓包 (捕获->选项)



- 查找报文 (编辑->查找分组)



- 到前一报文 (跳转->前一分组)



- 到后一报文 (跳转->后一分组)



- 到指定报文 (跳转->转至分组)



- 到第一个报文 (跳转->首个分组)



- 到最后一个报文 (跳转->最新分组)



- 实时抓包时自动滚动窗口至最新报文



- 打开抓包文件 (文件->打开)



- 另存为文件 (文件->另存为)



- 关闭当前抓包 (文件->关闭)



- 重新载入抓包文件 (视图->重新加载)



- 对报文着色 (视图->着色分组列表)



- 放大 (视图->缩放->放大)



- 缩小 (视图->缩放->缩小)



- 恢复正常尺寸 (视图->缩放->普通大小)



- 使列宽适配内容长度 (视图->调整列宽)

数据包的颜色（视图->着色规则）

名称	过滤器
☒ Bad TCP	tcp.analysis.flags && !tcp.analysis.window_update
☒ HSRP State Change	hsrp.state != 8 && hsrp.state != 16
☒ Spanning Tree...ology Change	stp.type == 0x80
☒ OSPF State Change	ospf.msg != 1
☒ ICMP errors	icmp.type eq 3 icmp.type eq 4 icmp.type eq 5 icmp.type eq 11 icmpv6.type eq 1 icmpv6.type eq 2 icmpv6.type eq 3
☒ ARP	arp
☒ ICMP	icmp icmpv6
☒ TCP RST	tcp.flags.reset eq 1
☒ SCTP ABORT	sctp.chunk_type eq ABORT
☒ TTL low or unexpected	(! ip.dst == 224.0.0.0/4 && ip.ttl < 5 && !pim && !ospf) (ip.dst == 224.0.0.0/24 && ip.dst != 224.0.0.251 && ip.ttl != 1)
☒ Checksum Errors	eth.fcs.status == "Bad" ip.checksum.status == "Bad" tcp.checksum.status == "Bad" udp.checksum.status == "Bad" sctp.checksum.status == "Bad"
☒ SMB	smb nbss nbns nbipx ipxsap netbios
☒ HTTP	http tcp.port == 80 http2
☒ IPX	ipx spx
☒ DCERPC	dcerpc
☒ Routing	hsrp eigrp ospf bgp cdp vrrp carp gvrp igmp ismp
☒ TCP SYN/FIN	tcp.flags & 0x02 tcp.flags.fin == 1
☒ TCP	tcp
☒ UDP	udp
☒ Broadcast	eth[0] & 1

设定时间显示格式

- 绝对时间

No.	Time
112	2019-05-07 16:45:42.632282
113	2019-05-07 16:45:42.633372
114	2019-05-07 16:45:42.643647
115	2019-05-07 16:45:42.644459
116	2019-05-07 16:45:42.650289
117	2019-05-07 16:45:42.672395
118	2019-05-07 16:45:42.672498
119	2019-05-07 16:45:42.672590

- 相对时间

No.	Time
112	5.965018
113	5.966108
114	5.976383
115	5.977195
116	5.983025
117	6.005131
118	6.005234
119	6.005326
120	6.016173

- 设定基于某一个数据包的相对时间

No.	Time
112	5.965018
113	5.966108
114	5.976383
115	5.977195
116	*REF*
117	0.022106
118	0.022209
119	0.022301

数据包列表面板的标记符号



- 会话中的第 1 个数据包



- 会话中选中的数据包



- 不属于所选会话中的数据包



- 会话中最后 1 个数据包



- 请求



- 响应



- 若当前选中的是 ACK 报文，则此报文是它确认的那个报文



- 若当前选中的是 ACK 报文，则此报文是它重复确认的那个报文



- 选中的报文与此报文有某种关联

四种流跟踪

- TCP
- UDP
- SSL
- HTTP

文件操作

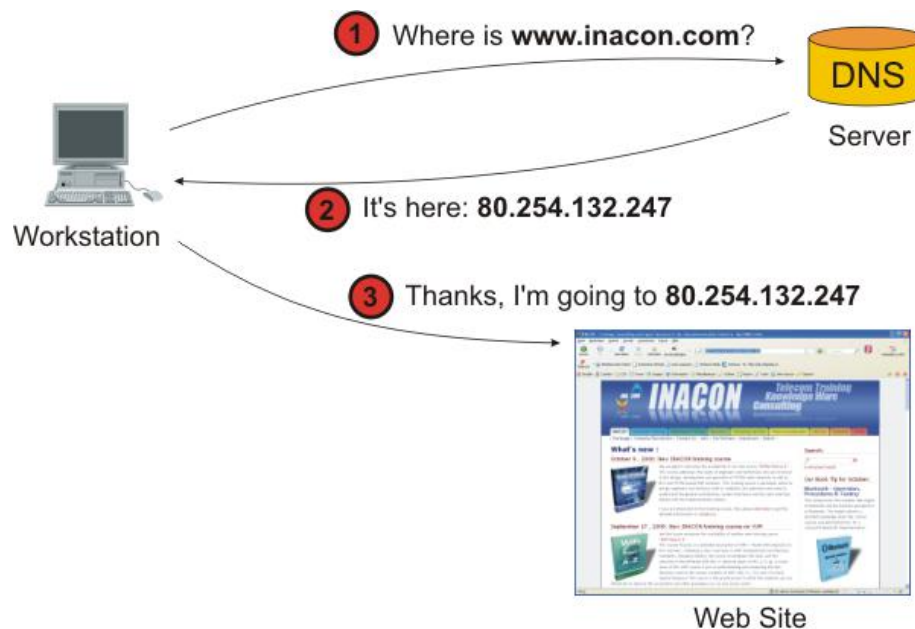
- 标记报文 Ctrl+M
- 导出标记报文（文件->导出特定分组），亦可按过滤器导出报文
- 合并读入多个报文（文件->合并）

如何快速抓取移动设备的报文？

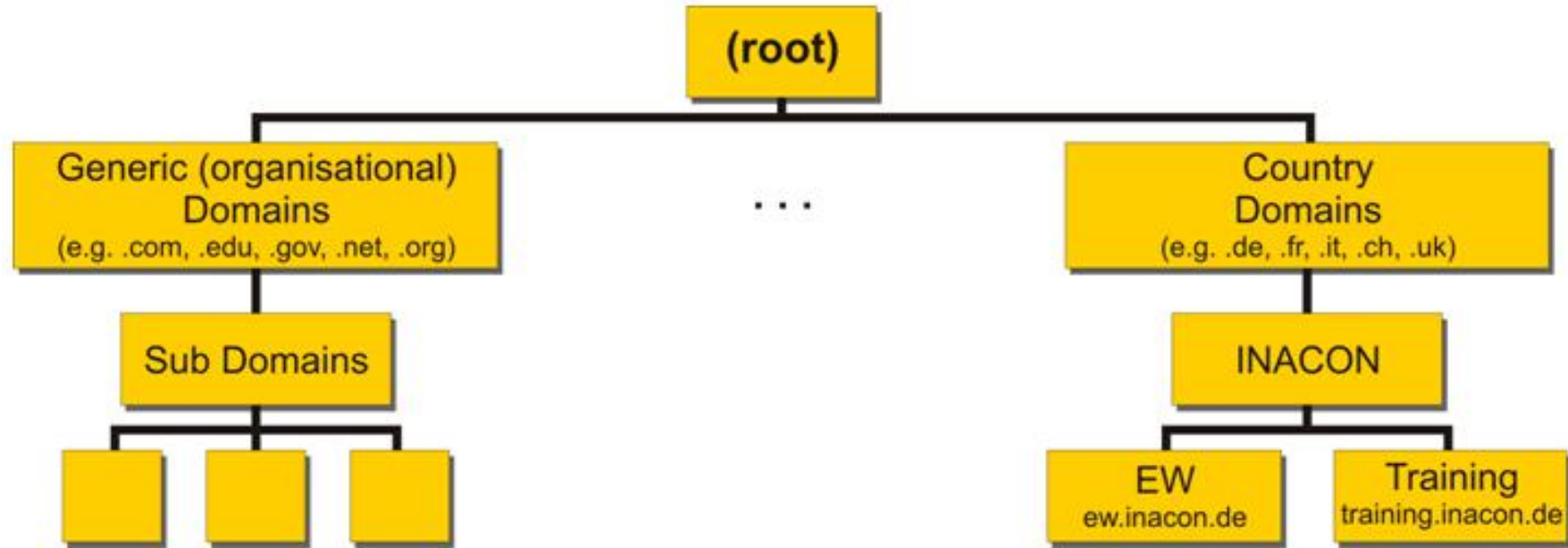
1. 在操作系统上打开 wifi 热点
2. 手机连接 wifi 热点
3. 用 Wireshark 打开捕获->选项面板，选择 wifi 热点对应的接口设备抓包

什么是 DNS?

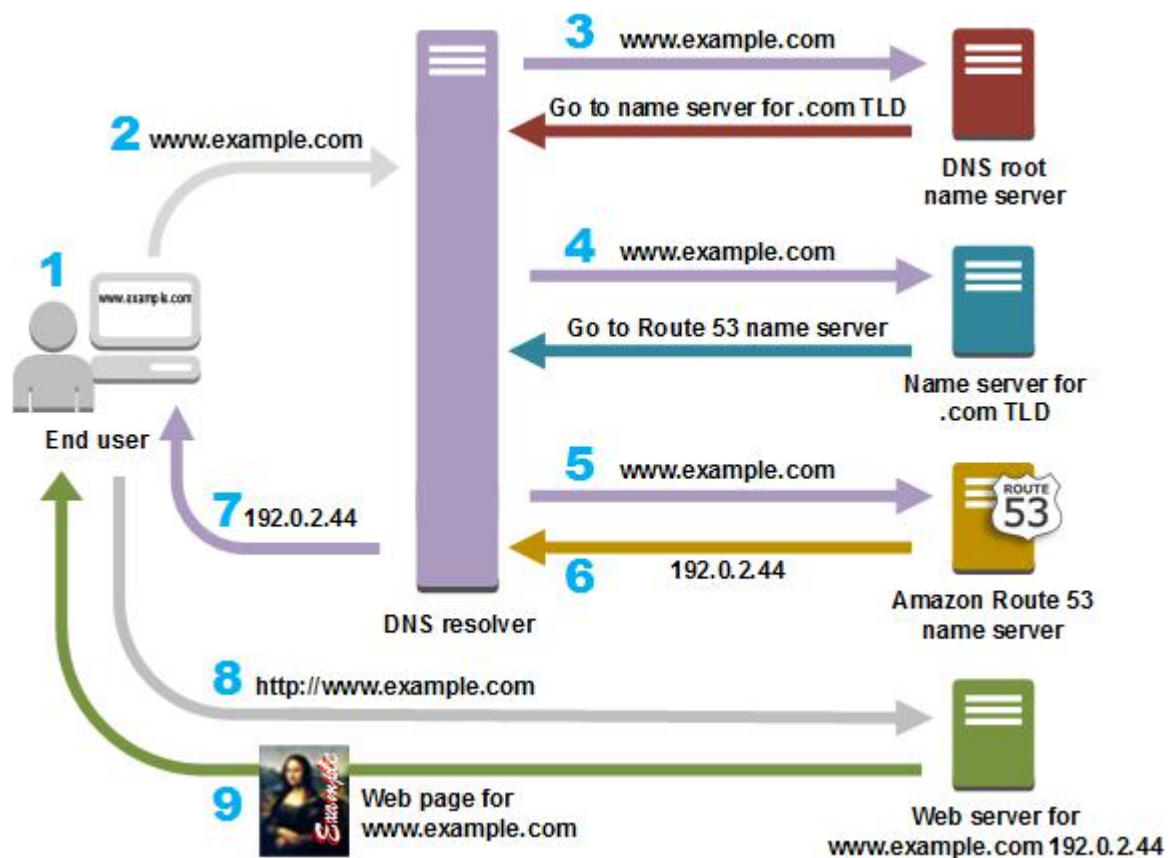
- 一个用于将人类可读的“域名”（例如 www.taohui.pub）与服务器的IP地址（例如 116.62.160.193）进行映射的数据库
- 递归查询
 - 根域名服务器
 - 权威服务器



DNS 域名结构

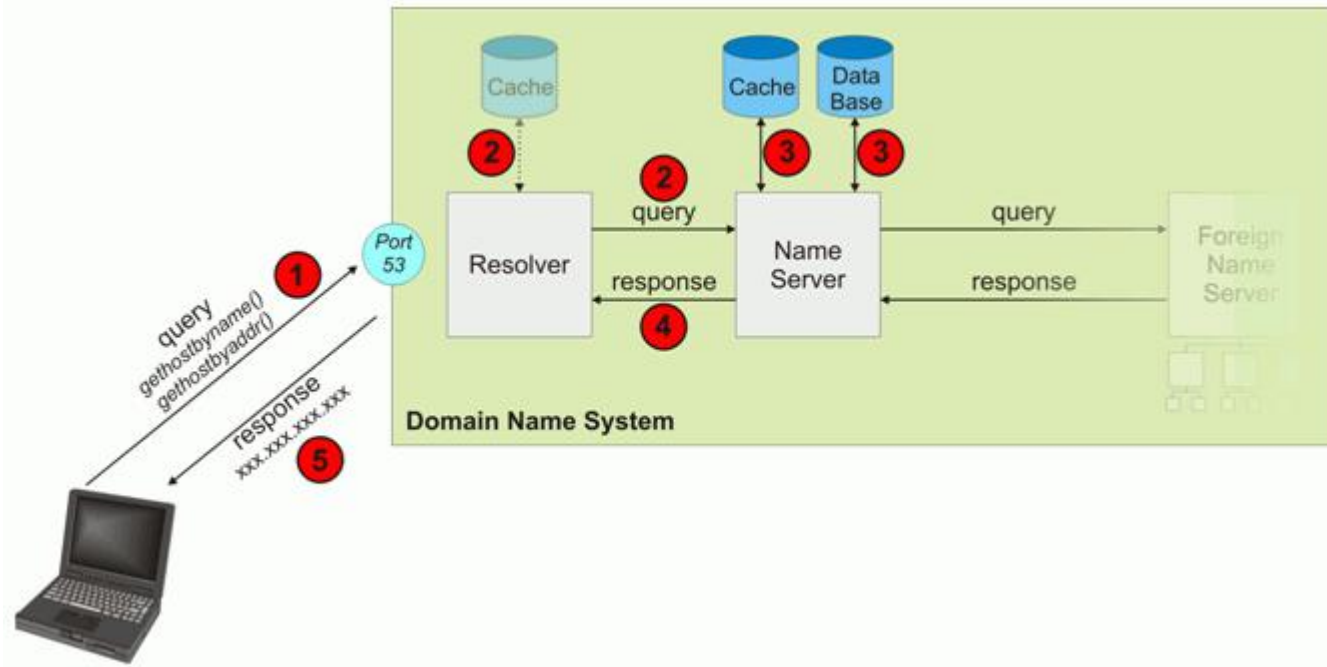


递归查询



DNS 报文：查询与响应

- query: 查询域名
- response: 返回 IP 地址



dig 工具查询

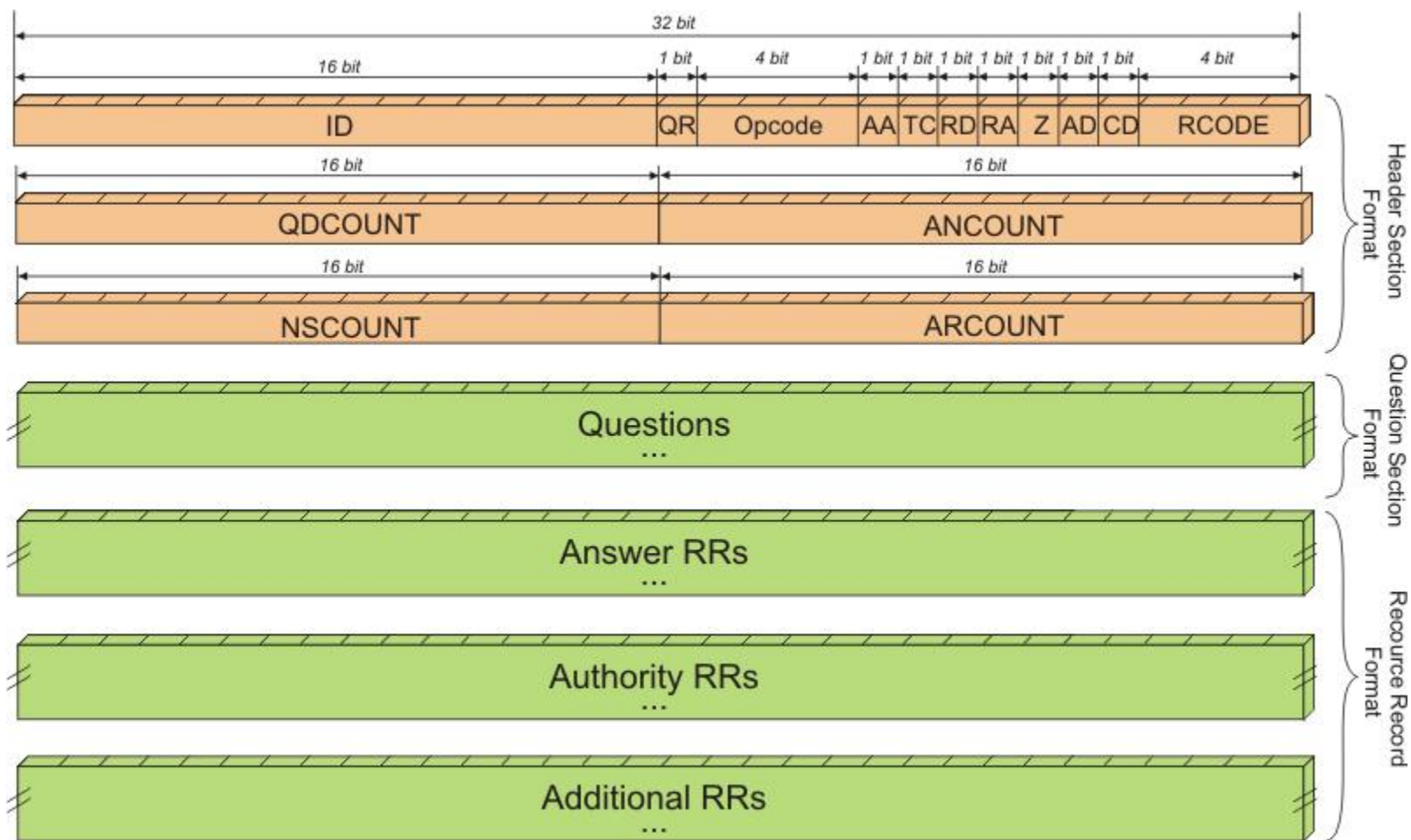
```
; <<>> DiG 9.9.4-RedHat-9.9.4-72.el7 <<>> image.baidu.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21015
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 4096
;; QUESTION SECTION:
;image.baidu.com.                IN      A

;; ANSWER SECTION:
image.baidu.com.                 749     IN      CNAME   image.n.shifen.com.
image.n.shifen.com.             300     IN      A       180.97.33.134
image.n.shifen.com.             300     IN      A       115.239.210.36

;; Query time: 9 msec
;; SERVER: 100.100.2.138#53(100.100.2.138)
;; WHEN: Wed May 08 20:51:01 CST 2019
;; MSG SIZE  rcvd: 105
```

DNS 报文



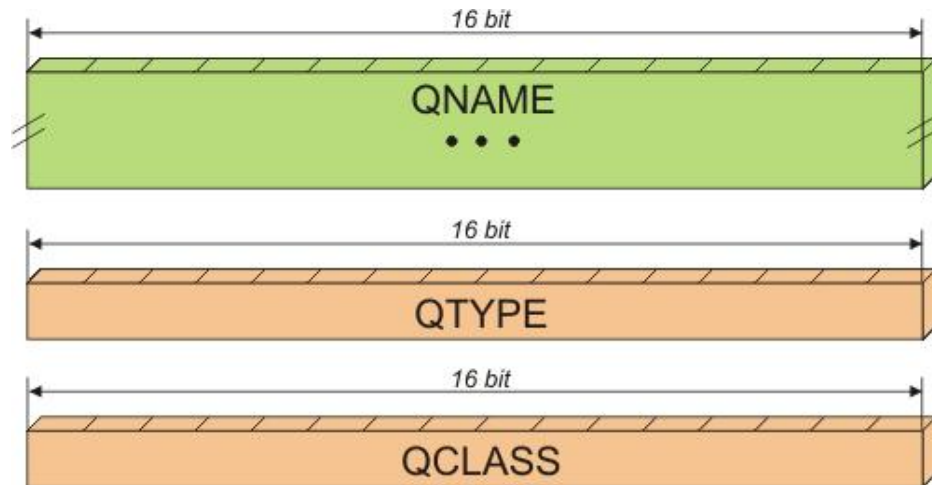
Questions 格式

- QNAME 编码规则:
 - 以.分隔为多段，每段以字节数打头
 - 单字节，前 2 比特必须为 00，只能表示 $2^6-1=63$ 字节
 - 在 ASCII 编码每段字符
 - 以 0 结尾

- QTYPE 常用类型

值	类型	意义
1	A	IPv4地址
2	NS	权威域名服务器
5	CNAME	别名
15	MX	邮件交换
16	TXT	文本字符串
28	AAAA	IPv6地址

- QCLASS: IN 表示 internet



Answer 格式

- NAME: 前 2 位为 11, 接引用 QNAME 偏移
 - 在 DNS 头部的字符偏移数
- TTL: Time To Live
- RDLENGTH: 指明 RDATA 的长度
- RDATA: 查询值, 如 IP 地址, 或者别名
 - 别名遵循 QNAME 编码规则

